

รายงานโครงงาน

เรื่อง

ระบบจัดการร้านซักรีดและซักผ้าหยอดเหรียญ (LaundryHub)

จัดทำโดย

673380425-2 วงศธร ธน.ยอด

673380395-5 คมชาญ น้อยเนียม

673380589-2 ปฏิภาณ มะนิลทิพย์

673380420-2 ภีมเดช กลั่นกิ่ง

อาจารย์ประจำวิชา: รศ. ดร.ปัญญาพล หอระตะ

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353002 หลักการพัฒนาซอฟต์แวร์ (Principles of Software Development)

สาขาวิชาวิทยาการคอมพิวเตอร์ ภาคเรียน 1 ปีการศึกษา 2569

วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

บทคัดย่อ

โครงงานนี้พัฒนา LaundryHub ระบบจัดการร้านซักรีดที่รองรับบริการสองรูปแบบในระบบเดียว ได้แก่ บริการฝากซัก (Full-Service) ที่คิดราคาจากน้ำหนักและประเภทบริการ และบริการซักผ้าหยอดเหรียญ (Self-Service) ที่ลูกค้าจองและใช้เครื่องเอง คิดราคาจากเวลาที่ใช้ ผู้ใช้ระบบมีสามบทบาท คือลูกค้า พนักงาน และผู้ดูแลระบบ

ระบบพัฒนาด้วย Spring Boot 3.5.7 และ Java 17 ตามสถาปัตยกรรมแบบชั้น (Layered Architecture) แยก Controller, Service, Repository และ Domain อย่างชัดเจน ใช้ PostgreSQL 16 พร้อม Flyway สำหรับจัดการ schema ใช้ Spring Security กำหนดสิทธิ์สามบทบาท และเปิดเผยบริการผ่าน REST API พร้อมเอกสาร Swagger/OpenAPI ส่วนการแสดงผลหน้าเว็บใช้ Thymeleaf โดยหน้าเว็บที่ทำเสร็จแล้ว ได้แก่ เข้าสู่ระบบและสมัครสมาชิก โปรไฟล์ จัดการสาขา ฝากซัก (ออเดอร์ของลูกค้าและบอร์ดพนักงาน) จองเครื่องและประวัติรอบใช้งาน ชำระเงิน รายการชำระเงิน และแจ้งเตือน

การออกแบบประยุกต์หลัก SOLID และ Design Pattern ที่มีเหตุผลรองรับ ได้แก่ Strategy (การคิดราคา และวิธีชำระเงิน), State (สถานะออเดอร์และเครื่อง), Observer (การแจ้งเตือน), Factory และ Facade (การชำระเงิน) รวมถึงรูปแบบเชิงสถาปัตยกรรม Repository, Service Layer, DTO และ Dependency Injection แบบ constructor ฐานข้อมูลมี 10 ตาราง ครอบคลุมความสัมพันธ์แบบ One-to-One และ One-to-Many

ผลการทดสอบ ชุดเทสต์อัตโนมัติรอบเต็มรัน 380 ข้อ ผ่านทั้งหมด (ผ่าน 343 และข้าม 37 ข้อที่ต้องเปิดฐานข้อมูลเอง) และเมื่อเปิดเทสต์ที่ต่อ PostgreSQL จริงรันครบ 390 ข้อ ผ่านทั้งหมด ระบบถูก deploy ใช้งานจริงบน Render และ Neon การพัฒนาใช้ Git workflow แบบแตกกิ่งส่วนตัวและรวมผ่าน Pull Request พร้อมการรีวิวโค้ดโดยเพื่อนร่วมทีม ซึ่งช่วยตรวจพบข้อบกพร่องหลายรายการก่อนรวมงาน

คำสำคัญ — Spring Boot, Layered Architecture, SOLID, Design Pattern, REST API, PostgreSQL, ระบบซักรีด

คำนำ

รายงานโครงงานฉบับนี้จัดทำขึ้นเป็นส่วนหนึ่งของการศึกษาวิชา CP353002 หลักการพัฒนาซอฟต์แวร์ (Principles of Software Development) โดยมีวัตถุประสงค์เพื่อนำเสนอการออกแบบและพัฒนาระบบ LaundryHub ซึ่งเป็นระบบจัดการร้านซักรีดที่รวมบริการฝากซักและซักผ้าหยอดเหรียญไว้ในระบบเดียว ตามหลักการออกแบบซอฟต์แวร์เชิงวัตถุ หลัก SOLID และ Design Pattern

เนื้อหาในรายงานแบ่งเป็น 5 บท ได้แก่ บทที่ 1 บทนำ บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง บทที่ 3 วิธีดำเนินการและการออกแบบระบบ บทที่ 4 ผลการพัฒนาและการทดสอบ และบทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ โดยสมาชิกแต่ละคนรับผิดชอบเขียนส่วนที่ตรงกับโมดูลของตนเอง และรวมเป็นเล่มเดียว

ผู้จัดทำขอขอบพระคุณอาจารย์ประจำวิชา รศ. ดร.ปัญญาพล หอระตะ ที่ให้ความรู้และคำแนะนำตลอดการทำโครงงาน และขอบคุณสมาชิกทุกคนที่ร่วมกันพัฒนาและตรวจทานงานจนแล้วเสร็จ

หากรายงานฉบับนี้มีข้อบกพร่องประการใด คณะผู้จัดทำขออภัยมา ณ ที่นี้ด้วย

คณะผู้จัดทำ

10 ตุลาคม 2569

สารบัญ

บทที่ 1 บทนำ 1
1.1 ที่มาและความสำคัญ 1
1.2 วัตถุประสงค์ 1
1.3 ขอบเขตของการศึกษา 2
1.4 ประโยชน์ที่คาดว่าจะได้รับ 3
1.5 นิยามศัพท์ 3
บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง 5
2.1 Spring Boot และ Spring Data JPA 5
2.2 การจัดการเวอร์ชันฐานข้อมูลด้วย Flyway 5
2.3 Spring Security และการเก็บรหัสผ่าน 6
2.4 Docker และ Container 6
2.5 CI/CD และเวิร์กโฟลว์ของ Git 6
2.6 State Pattern สำหรับสถานะออเดอร์ 7
2.7 Strategy Pattern สำหรับการคิดราคา 7
2.8 Builder Pattern สำหรับสร้างข้อมูลตอบกลับ 8
2.9 ความสัมพันธ์แบบ One-to-Many และการโหลดข้อมูลใน JPA 8
2.10 State Pattern สำหรับสถานะเครื่อง 8
2.11 Strategy สำหรับการคำนวณราคา 9
2.12 การตรวจช่วงเวลาซ้อนและการจองพร้อมกัน 9
2.13 Design Patterns 9
2.14 หลัก SOLID และสถาปัตยกรรมแบบชั้น 10
2.15 REST API, HTTP Status และ OpenAPI 10
2.16 ฐานข้อมูลและการควบคุมความถูกต้อง 10

2.17 การทดสอบซอฟต์แวร์ 10
บทที่ 3 วิธีดำเนินการและการออกแบบระบบ 11
3.1 สถาปัตยกรรมภาพรวม 11
3.2 แผนภาพ Use Case 12
3.3 แผนภาพ Component 14
3.4 แผนภาพ Deployment 14
3.5 การออกแบบระบบยืนยันตัวตนและความปลอดภัย 15
3.6 การออกแบบฐานข้อมูล 17
3.7 การออกแบบ API 20
3.8 การออกแบบหน้าเว็บ 21
3.9 การออกแบบการ Deploy และ CI/CD 21
3.10 วิธีทดสอบ 22
3.11 การออกแบบโมดูลออเดอร์ฝากซัก 23
3.12 การออกแบบฐานข้อมูลของโมดูลออเดอร์ 24
3.13 การคิดราคา 25
3.14 การออกแบบสถานะออเดอร์ 25
3.15 การออกแบบ API และสิทธิ์การเข้าถึง 28
3.16 การเชื่อมกับโมดูลชำระเงิน 29
3.17 โครงสร้างโมดูล Self-Service 29
3.18 ขั้นตอนการจองและการเปลี่ยนสถานะ 30
3.19 REST API และการตรวจสิทธิ์ 31
3.20 หลัก SOLID และการเชื่อมโมดูล 31
3.21 ภาพรวมการออกแบบ 32
3.22 การออกแบบกฎธุรกิจและความปลอดภัย 35
บทที่ 4 ผลการพัฒนาและการทดสอบ 37

4.1 ผลการพัฒนา 37
4.2 ผลหน้าเว็บ 37
4.3 ผล Unit Test 42
4.4 ผลทดสอบแอปจริงของโมดูลบัญชีผู้ใช้และสาขา 43
4.5 ผล Deploy และ CI 44
4.6 ผลการทำงานร่วมกันด้วย Git 44
4.7 ข้อบกพร่องที่พบและแก้ไข 45
4.8 ผลการพัฒนาโมดูลออเดอร์ 46
4.9 ผลการทดสอบอัตโนมัติ 48
4.10 ผลการทดสอบกับระบบจริง 49
4.11 ผลการทดสอบโมดูล Self-Service 50
4.12 ผลการรีวิวและข้อจำกัดของหลักฐาน 51
4.13 ผลทดสอบกับฐานข้อมูลจริง 51
4.14 ผลทดสอบแอปจริง (22 กรณี) 51
4.15 ผลหน้าเว็บชำระเงินและแจ้งเตือน 52
บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ 54
5.1 สรุปผลการดำเนินงาน 54
5.2 อภิปรายผล 54
5.3 ผลลัพธ์การเรียนรู้ 55
5.4 ปัญหาที่พบในการดำเนินงาน 55
5.5 ข้อจำกัด 56
5.6 ข้อเสนอแนะและแนวทางพัฒนาต่อ 58
5.7 สรุปผลโมดูลออเดอร์ 59
5.8 สรุปโมดูลที่ทำแล้ว 59
5.9 ข้อจำกัด งานตรวจรับที่เหลือ และแนวทางพัฒนาต่อ 59

เอกสารอ้างอิง..... 61

ภาคผนวก 63

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

ส่วนของวงศธร ธน.ยอด

ร้านซักรีดขนาดเล็กมักให้บริการสองรูปแบบพร้อมกัน รูปแบบแรกคือ **ฝากซัก** (Full-Service) ที่ลูกค้าส่งผ้าให้พนักงานซัก อบ รีด แล้วมารับภายหลัง รูปแบบที่สองคือ **ซักเอง** (Self-Service) ที่ลูกค้าใช้เครื่องซักและเครื่องอบหยอดเหรียญด้วยตนเอง การจัดการทั้งสองรูปแบบด้วยการจดบันทึกหรือโปรแกรมแยกกันทำให้ติดตามสถานะผ้า ตรวจสอบว่าเครื่องใดว่าง และสรุปการชำระเงินได้ยาก

โครงงานนี้พัฒนาระบบ LaundryHub เพื่อรวมการทำงานทั้งสองรูปแบบไว้ในระบบเดียว โดยใช้ Spring Boot ตามสถาปัตยกรรมแบบชั้น (Layered Architecture) และประยุกต์หลัก SOLID กับ Design Pattern ตามวัตถุประสงค์ของรายวิชา CP353002 Principles of Software Design and Development โครงงานดำเนินการโดยกลุ่มนักศึกษา 4 คน ระหว่างวันที่ 8 ถึง 10 ตุลาคม 2569 และนำระบบขึ้นให้บริการจริงบนอินเทอร์เน็ต

ส่วนของภีมเดช กลั่นกิ่ง

ระบบ LaundryHub รองรับร้านซักรีดสองรูปแบบในระบบเดียว คือฝากซักและซักหยอดเหรียญแบบบริการตนเอง ทั้งสองรูปแบบต้องมี **การชำระเงิน** (ค่าบริการต่างกัน คิดจากน้ำหนักหรือเวลาใช้เครื่อง) และ **การแจ้งเตือน** (สถานะออเดอร์/การจอง/การชำระเปลี่ยน) ที่เหมือนกัน การแยกสองเรื่องนี้ออกเป็นโมดูลกลางที่ไม่ผูกกับโมดูลออเดอร์หรือโมดูลเครื่องซัก ช่วยให้เพิ่มวิธีชำระหรือช่องทางแจ้งเตือนใหม่ได้โดยไม่แก้โมดูลต้นทาง นอกจากนี้ระบบ REST API ที่ผู้ใช้หลายบทบาทเรียกใช้ จำเป็นต้องตอบ error ในรูปแบบเดียวกันและไม่เปิดเผยข้อมูลภายใน

1.2 วัตถุประสงค์

ส่วนของวงศธร ธน.ยอด

1. พัฒนาระบบจัดการร้านซักรีดที่รองรับทั้งการฝากซักและการซักเอง พร้อมผู้ใช้ 3 บทบาท ได้แก่ ลูกค้า พนักงาน และผู้ดูแลระบบ
2. ออกแบบระบบตามสถาปัตยกรรมแบบชั้น โดย Controller ไม่เรียก Repository โดยตรง และประยุกต์ SOLID กับ Design Pattern อย่างมีเหตุผลรองรับ

3. ออกแบบฐานข้อมูลเชิงสัมพันธ์ที่มีความสัมพันธ์แบบ One-to-One และ One-to-Many และจัดการเวอร์ชันของโครงสร้างด้วย Flyway
4. พัฒนา REST API ที่มีเอกสาร Swagger รูปแบบ error มาตรฐาน และหน้าเว็บด้วย Thymeleaf
5. ทำงานเป็นทีมด้วย Git และ Pull Request ที่มีการรีวิวโค้ด ทดสอบอัตโนมัติด้วย CI และ deploy ระบบด้วย Docker ขึ้นบริการคลาวด์
6. (ส่วนของผู้เขียนคนนี้) พัฒนาโครงสร้างพื้นฐานของระบบ ได้แก่ ระบบยืนยันตัวตนและสิทธิ์ ข้อมูลผู้ใช้และสาขา โครงหน้าเว็บ การ deploy และ CI

ส่วนของภีมเดช กลั่นกิ่ง

1. พัฒนาระบบชำระเงินที่รองรับหลายวิธี (เงินสด, QR จำลอง, เหรียญ) และเพิ่มวิธีใหม่ได้โดยไม่แก้โค้ดเดิม
2. พัฒนาระบบแจ้งเตือนในระบบที่โมดูลอื่นไม่ต้องรู้จัก
3. กำหนดรูปแบบ error มาตรฐานของ REST API ทั้งระบบ และเอกสาร API ด้วย Swagger/OpenAPI
4. ทดสอบโมดูลด้วย Unit Test และการทดสอบกับฐานข้อมูลจริง

1.3 ขอบเขตของการศึกษา

ส่วนของวงศธร ธน.ยอด

- **ผู้ใช้และบทบาท**: ลูกค้า (CUSTOMER) พนักงาน (STAFF) และผู้ดูแลระบบ (ADMIN) โดยผู้ดูแลระบบทำได้ทุกอย่างที่พนักงานทำได้
- **ฟังก์ชันหลักที่พัฒนา**: สมัครสมาชิกและเข้าสู่ระบบ, จัดการโปรไฟล์ลูกค้า, จัดการสาขา, ฝากซักพร้อมคิดราคาและติดตามสถานะตามลำดับ, จัดการประเภทบริการและราคา, จัดการเครื่องซัก-อบ จองและใช้งานเครื่อง, ชำระเงิน (เงินสด QR จำลอง เหรียญ) และแจ้งเตือนในระบบ
- **เทคโนโลยี**: Java 17, Spring Boot 3.5.7, Spring Data JPA, Spring Security, PostgreSQL 16, Flyway, Thymeleaf กับ Bootstrap 5, JUnit 5 กับ Mockito, Docker, GitHub Actions, Render และ Neon
- **ไม่ทำ (ตัดตามขอบเขตเพื่อให้จบภายในเวลา)**: ไม่เชื่อมระบบชำระเงินจริง (ใช้การจำลอง), ไม่ส่ง SMS หรือ LINE จริง (แจ้งเตือนในระบบเท่านั้น),

ไม่ทำแอปมือถือแยก, ไม่มีหน้าจัดการบัญชีผู้ใช้สำหรับผู้ดูแลระบบ, ไม่มีรายงานยอดขายและแดชบอร์ด

- หน้าเว็บของทุกโมดูล (ออเดอร์ฝากซัก เครื่องซัก-อบ ชำระเงิน แจ้งเตือน) ทำเสร็จและรวมอยู่ใน develop แล้ว

ส่วนของภีมเดช กลั่นกิ่ง

- **ทำ**: `payments` (สร้าง/ดู/แบ่งหน้า/ยืนยันเงินสด), `notifications` (ดู/กรองยังไม่อ่าน/mark read), `GlobalExceptionHandler`, Swagger พร้อม Basic Auth, หน้าเว็บชำระเงิน รายการชำระเงิน/ยืนยันรับเงินสด (พนักงาน) และกล่องแจ้งเตือน (Thymeleaf)
- **ไม่ทำ (ตัดตามขอบเขตโปรเจค)**: ไม่ต่อ Payment Gateway จริง (ใช้การจำลอง), ไม่ส่ง SMS/LINE จริง (แจ้งเตือนในระบบเท่านั้น), ไม่ทำ Mobile App
- **ยังไม่ได้ทำ ณ วันที่เขียน**: รายงานยอดขาย (Admin), การวัด code coverage ด้วย Jacoco

1.4 ประโยชน์ที่คาดว่าจะได้รับ

ส่วนของวงศธร ธน.ยอด

1. ได้ระบบตัวอย่างที่ใช้งานได้จริงและเข้าถึงได้ผ่านอินเทอร์เน็ต สำหรับร้านซักรีดที่มีทั้งบริการฝากซักและเครื่องหยอดเหรียญ
2. ได้ตัวอย่างการประยุกต์ SOLID Design Pattern และสถาปัตยกรรมแบบชั้นกับโจทย์จริง พร้อมเอกสารอ้างอิงตำแหน่งในโค้ด
3. ได้ประสบการณ์ทำงานเป็นทีมด้วยกระบวนการที่ใช้ในงานจริง ได้แก่ Pull Request, Code Review, CI และการ deploy ด้วยคอนเทนเนอร์

ส่วนของภีมเดช กลั่นกิ่ง

ได้ตัวอย่างการใช้ Design Pattern และหลัก SOLID กับโจทย์จริง (ชำระเงินและแจ้งเตือน) ได้ API ที่มีรูปแบบ error สม่ำเสมอ และได้ชุดทดสอบที่ยืนยันกฎธุรกิจสำคัญทั้งในระดับโค้ดและระดับฐานข้อมูล

1.5 นิยามศัพท์

ส่วนของวงศธร ธน.ยอด

คำ	ความหมาย

Full-Service (ฝากซัก)	บริการที่ลูกค้าส่งผ้าให้พนักงานดำเนินการ คิดราคาจากน้ำหนักและประเภทบริการ
Self-Service (ซักเอง)	บริการที่ลูกค้าใช้เครื่องซักหรือเครื่องอบด้วยตนเอง คิดราคาจากเวลาที่ใช้เครื่อง
Entity	คลาสที่แทนแถวข้อมูลในตารางฐานข้อมูล ใช้ผ่าน JPA
DTO (Data Transfer Object)	คลาสที่ใช้รับหรือส่งข้อมูลผ่าน API โดยแยกจาก Entity เพื่อไม่เปิดเผยข้อมูลที่ไม่ควรเปิดเผย
ORM (Object-Relational Mapping)	เทคนิคแปลงข้อมูลระหว่างอ็อบเจ็กต์ในโปรแกรมกับตารางในฐานข้อมูล
Flyway	เครื่องมือจัดการเวอร์ชันของโครงสร้างฐานข้อมูลด้วยไฟล์ SQL เรียงลำดับ
BCrypt	ฟังก์ชันแฮชรหัสผ่านทางเดียวที่มีค่าสุ่ม (salt) และปรับความช้าได้
Authentication / Authorization	การยืนยันว่าผู้ใช้คือใคร / การตรวจว่าผู้ใช้มีสิทธิ์ทำสิ่งนั้นหรือไม่
HTTP Basic	วิธียืนยันตัวตนที่ส่งชื่อผู้ใช้และรหัสผ่านมากับทุกคำขอ ใช้กับการเรียก API
CI (Continuous Integration)	การรวมโค้ดและรันทดสอบอัตโนมัติทุกครั้งที่มีการเปลี่ยนแปลง
Container / Docker image	ชุดซอฟต์แวร์พร้อมสภาพแวดล้อมที่รันได้เหมือนกันทุกเครื่อง / ต้นแบบที่ใช้สร้าง container
Swagger / OpenAPI	มาตรฐานและเครื่องมือสร้างเอกสาร API ที่ทดลองเรียกได้ผ่านเบราว์เซอร์

ส่วนของภีมเดช กลั่นกิ่ง

คำ	**ความหมาย**
Payable	สิ่งที่ชำระเงินได้ (ออเดอร์ฝากซัก หรือรอบใช้เครื่อง) เป็น interface เล็ก 4 เมธอด
PayableProvider	ตัวค้นหา Payable ตามชนิด ทำให้โมดูลชำระเงินไม่ต้องรู้จักโมดูลอื่นตรงๆ
Facade	จุดเข้าเดียวที่ประสานหลายขั้นตอนเพื่อทำงานหนึ่งอย่าง
Domain Event	เหตุการณ์ที่โมดูลหนึ่งประกาศเพื่อให้โมดูลอื่นตอบสนอง
Constraint	กฎที่ฐานข้อมูลบังคับ (`UNIQUE`, `FK`, `CHECK`)

บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

2.1 Spring Boot และ Spring Data JPA

Spring Boot เป็นเฟรมเวิร์กที่ตั้งค่าส่วนประกอบของแอปพลิเคชัน Spring ให้อัตโนมัติ (auto-configuration) และฝังเว็บเซิร์ฟเวอร์ไว้ในตัวแอป ทำให้สร้างไฟล์ที่รันได้เองได้ และใช้ **Dependency Injection** ในการส่งต่อส่วนประกอบที่คลาสต้องใช้ผ่าน constructor จึงทดสอบแต่ละคลาสแยกกันได้ง่าย [Spring Boot Reference]

Spring Data JPA ทำงานบน Hibernate ซึ่งเป็น ORM สำหรับแปลงระหว่าง Entity กับตาราง ความสัมพันธ์ระหว่างตารางกำหนดด้วยแอนโนเทชัน เช่น `@OneToOne` และ `@OneToMany` โดยฝั่งที่ถือ foreign key เรียกว่า *owning side* ส่วนอีกฝั่งระบุ `mappedBy` ว่าอ้างจากฝั่งใด การโหลดข้อมูลมีแบบ `LAZY` (โหลดเมื่อใช้งาน) และ `EAGER` (โหลดทันที) ส่วน `cascade` กำหนดว่าการบันทึกหรือลบอ็อบเจ็กต์หลักจะส่งผลถึงอ็อบเจ็กต์ที่ผูกอยู่ด้วยหรือไม่ [Bauer, King & Gregory, 2015]

ธุรกรรม (Transaction) ทำให้การเขียนข้อมูลหลายขั้นตอนสำเร็จทั้งหมดหรือยกเลิกทั้งหมด (คุณสมบัติ ACID) ใน Spring ใช้ `@Transactional` โดย Spring สร้างตัวห่อ (proxy) ที่เปิดธุรกรรมก่อนเข้าเมธอด ยืนยัน (commit) เมื่อจบปกติ และย้อนกลับ (rollback) เมื่อเกิด `RuntimeException` นอกจากนี้ Hibernate ตรวจจับการแก้ไขอ็อบเจ็กต์ที่อยู่ในธุรกรรมและเขียนลงฐานข้อมูลให้เมื่อ commit (dirty checking)

2.2 การจัดการเวอร์ชันฐานข้อมูลด้วย Flyway

Flyway จัดการการเปลี่ยนแปลงโครงสร้างฐานข้อมูลด้วยไฟล์ SQL ที่มีหมายเลขเวอร์ชัน (เช่น `V1__init_schema.sql`) รันตามลำดับเพียงครั้งเดียวต่อไฟล์ และบันทึกประวัติพร้อมค่าตรวจสอบ (checksum) ในตาราง `flyway_schema_history` ถ้าไฟล์ที่รันไปแล้วถูกแก้ไข Flyway จะไม่ยอมทำงานต่อ จึงต้องเพิ่มการเปลี่ยนแปลงเป็นไฟล์เวอร์ชันใหม่ [Flyway Documentation] วิธีนี้ต่างจากการตั้ง `ddl-auto: update` ให้ Hibernate คาดเดาและแก้ตารางเอง ซึ่งไม่มีประวัติและให้ผลไม่แน่นอนระหว่างเครื่อง โปรเจคนี้ตั้ง `ddl-auto: validate` เพื่อให้ Hibernate เพียงตรวจว่า Entity ตรงกับตารางหรือไม่

2.3 Spring Security และการเก็บรหัสผ่าน

Spring Security ทำงานเป็นชุดตัวกรอง (filter chain) ที่ทุกคำขอต้องผ่านก่อนถึง Controller ขั้น Authentication ตรวจว่าผู้ใช้คือใคร โดยเรียก `UserDetailsService` ไปค้นผู้ใช้ แล้วให้ `PasswordEncoder` เทียบรหัสผ่านที่ส่งมากับค่าที่เก็บ ขั้น Authorization ตรวจว่ามีสิทธิ์หรือไม่ ทั้งแบบกำหนดตาม URL และแบบกำหนดที่เมธอดด้วย `@PreAuthorize` ซึ่งเขียนเงื่อนไขด้วยนิพจน์ได้ (SpEL) เช่น "เป็นแอดมินหรือเป็นเจ้าของข้อมูล" รหัสสถานะที่เกี่ยวข้องคือ 401 เมื่อยังไม่ยืนยันตัวตน และ 403 เมื่อยืนยันแล้วแต่ไม่มีสิทธิ์ [Spring Security Reference]

การเก็บรหัสผ่านต้องเก็บเป็นค่าแฮชทางเดียวและไม่เก็บข้อความจริง BCrypt เป็นอัลกอริทึมที่ออกแบบให้ปรับความช้าได้ (cost factor) และใส่ค่าสุ่ม (salt) ทุกครั้ง ทำให้รหัสผ่านเดียวกันได้ค่าแฮชต่างกัน และการเดารหัสด้วยการลองจำนวนมากทำได้ยากขึ้น [Provos & Mazières, 1999; OWASP] ระบบรองรับการยืนยันตัวตนสองช่องทาง คือ **ฟอร์มล็อกอินพร้อมเซสชัน** สำหรับหน้าเว็บ และ HTTP Basic สำหรับ REST API นอกจากนี้ หน้าเว็บที่ใช้เซสชันต้องป้องกัน CSRF (การหลอกให้เบราว์เซอร์ส่งคำขอโดยใช้เซสชันที่ล็อกอินค้างอยู่) ด้วยโทเค็นในแบบฟอร์ม

2.4 Docker และ Container

Docker แพ็กแอปพลิเคชันพร้อมสภาพแวดล้อมที่ต้องใช้เป็น image แล้วรันเป็น container ที่ใช้ทรัพยากรน้อยกว่าเครื่องเสมือนเพราะแบ่งปันเคอร์เนลของระบบปฏิบัติการ ทำให้แอปทำงานเหมือนกันในเครื่องนักพัฒนาและบนเซิร์ฟเวอร์ [Merkel, 2014] ไฟล์ `Dockerfile` ระบุขั้นตอนสร้าง image แบบ multi-stage ใช้ชั้นแรกที่มีเครื่องมือ build สร้างไฟล์แอป แล้วคัดลอกเฉพาะผลลัพธ์ไปยังชั้นที่สองซึ่งมีเพียงตัวรัน ทำให้ image สุดท้ายเล็กและไม่พกซอร์สโค้ดหรือเครื่องมือ build ส่วน `docker-compose` ใช้กำหนดและรันหลาย container ร่วมกัน เช่น แอปและฐานข้อมูล [Docker Documentation]

2.5 CI/CD และเวิร์กโฟลว์ของ Git

Continuous Integration คือการรวมงานของสมาชิกเข้าด้วยกันบ่อยๆ พร้อมสร้างและทดสอบโดยอัตโนมัติทุกครั้ง เพื่อพบข้อผิดพลาดตั้งแต่เนิ่นๆ [Fowler, 2006] โปรเจคนี้ใช้ GitHub Actions ซึ่งนิยาม *workflow* เป็นไฟล์ YAML ประกอบด้วย *job* ที่รันบนเครื่องรันที่ GitHub จัดให้ และตั้ง *required status checks* ใน branch protection เพื่อห้าม merge ถ้า job ไม่ผ่าน [GitHub Docs] ส่วน Continuous Deployment

ทำโดยผูกที่เก็บโค้ดกับบริการ Render ที่ build image จาก Dockerfile และปล่อยเวอร์ชันใหม่โดยอัตโนมัติเมื่อมีการ push

การทำงานร่วมกันใช้ Git แบบแตกสาขา (branching) โดย `main` เก็บเวอร์ชันส่งมอบ `develop` ใช้รวมงาน และแต่ละคนทำงานบนสาขาของตนเอง การรวมงานทุกครั้งผ่าน **Pull Request** ที่ต้องมีผู้รีวิวอย่างน้อยหนึ่งคนอนุมัติ [Chacon & Straub, 2014]

2.6 State Pattern สำหรับสถานะออเดอร์

State Pattern ให้วัตถุเปลี่ยนพฤติกรรมตามสถานะภายใน โดยแยกพฤติกรรมของแต่ละสถานะเป็นคลาสของตัวเองที่ implement interface เดียวกัน ผู้เรียกจึงไม่ต้องเขียน `if/switch` ตามสถานะทุกครั้งที่ต้องตัดสินใจ [Gamma et al., 1994]

ในโมดูลออเดอร์ interface `OrderState` มี 3 เมธอด คือ `status()`, `next()` และ `canCancel()` และมีคลาส State 7 คลาสตามสถานะในตาราง `laundry_orders` ได้แก่ `ReceivedState`, `WashingState`, `DryingState`, `IroningState`, `ReadyState`, `PickedUpState` และ `CancelledState`

เมธอด `next()` คืนค่า `Optional<OrderState>` สถานะสุดท้ายจึงคืน `Optional.empty()` แทนการโยน `UnsupportedOperationException` ทุก State ใช้แทนกันได้ตามหลัก Liskov Substitution

2.7 Strategy Pattern สำหรับการคิดราคา

Strategy Pattern แยกกลุ่มอัลกอริทึมที่ทำงานเรื่องเดียวกันออกเป็นคลาสที่สลับใช้แทนกันได้ผ่าน interface เดียว ผู้ใช้อัลกอริทึมขึ้นกับ interface ไม่ใช่คลาสจริง [Gamma et al., 1994]

ทีมใช้ interface กลาง `PricingStrategy<I>` ที่มีเมธอดเดียว `calculate(I input)` โมดูลฝากซักใช้ `FullServicePricing` (คิดตามน้ำหนัก) ส่วนโมดูลซักเองใช้ `SelfServicePricing` (คิดตามเวลา) การเพิ่มวิธีคิดราคาใหม่ทำได้ด้วยการเพิ่มคลาส ไม่ต้องแก้ Service เดิม ตามหลัก Open/Closed

2.8 Builder Pattern สำหรับสร้างข้อมูลตอบกลับ

Builder Pattern แยกขั้นตอนการสร้างวัตถุที่มีหลายส่วนออกจากตัววัตถุ ช่วยให้สร้างวัตถุที่มีพารามิเตอร์จำนวนมากได้อ่านง่ายและไม่สลับตำแหน่งผิด [Gamma et al., 1994; Bloch, 2018]

`OrderResponse` มี 11 field และมีหลาย field เป็นชนิด `Long` เหมือนกัน (`id`, `customerId`, `branchId`) ถ้าสร้างด้วย constructor ธรรมดาแล้วสลับตำแหน่งโปรแกรมยัง compile ผ่าน โมดูลนี้จึงใช้ `@Builder` ของ Lombok และสร้างผ่าน `OrderResponse.builder().customerId(..).branchId(..).build()` ส่วน DTO ที่มี field น้อยยังใช้ constructor ปกติ

2.9 ความสัมพันธ์แบบ One-to-Many และการโหลดข้อมูลใน JPA

ออเดอร์หนึ่งใบมีรายการผ้าหลายรายการ จึงใช้ `@OneToMany(mappedBy = "order", cascade = ALL, orphanRemoval = true)` ฝั่งออเดอร์ และ `@ManyToOne(fetch = LAZY)` ฝั่งรายการ `cascade = ALL` ทำให้บันทึกหรือลบออเดอร์แล้วรายการถูกบันทึกหรือลบตาม ส่วน `orphanRemoval` ลบแถวของรายการที่ถูกนำออกจาก list ขณะแก้ไขออเดอร์

การโหลดแบบ `LAZY` ช่วยไม่ให้ดึงข้อมูลที่ไม่ได้ใช้ แต่ทำให้เกิดปัญหา N+1 query ได้เมื่อวนอ่านความสัมพันธ์ของหลายแถว ซึ่งแก้ได้ด้วย `@EntityGraph` (ดึงพร้อมกันในคำสั่งเดียว) หรือ `@BatchSize` (ดึงเป็นชุดด้วย `IN (...)`) [Hibernate ORM, 2024]

2.10 State Pattern สำหรับสถานะเครื่อง

การตัดสินใจว่าเครื่องสามารถเริ่ม จบ หรือเข้าสถานะหยุดบริการได้ขึ้นกับสถานะปัจจุบัน ระบบแยกกฎนี้เป็น interface MachineState ซึ่งมี status(), canStart(), canFinish() และ canSetOutOfService() แต่ละ implementation คืนค่าความสามารถของตนเอง Service ใช้ผลดังกล่าวเพื่อตัดสินใจอนุญาตหรือปฏิเสธคำสั่ง ไม่ใช้ UnsupportedOperationException ใน state ที่ไม่รองรับการทำงาน จึงมีสัญญาการเรียกที่สม่ำเสมอ

สถานะเครื่อง	เริ่มได้	จบได้	ตั้งหยุดบริการได้
AVAILABLE	ได้	ไม่ได้	ได้
RESERVED	ไม่ได้	ไม่ได้	ได้
IN_USE	ไม่ได้	ได้	ไม่ได้

OUT_OF_SERVICE	ไม่ได้	ไม่ได้	ได้

MachineStateFactory รับ List ของ state จาก Spring ผ่าน constructor แล้วเก็บใน EnumMap เพื่อค้นหาตาม MachineStatus เป็น registry สำหรับเลือก state ไม่ใช่ GoF Factory Method ตามตำรา ค่า RESERVED ของเครื่องมีไว้เผื่อขยาย ส่วน flow ปัจจุบันใช้ RESERVED กับ Session และยังคงให้เครื่อง AVAILABLE จนเริ่มใช้งาน

2.11 Strategy สำหรับการคำนวณราคา

SelfServicePricing implements PricingStrategy ของข้อมูล SelfServicePricingInput ทำให้ SessionService ขึ้นกับ interface การคำนวณราคาแทนรายละเอียดสูตรโดยตรง สูตรคือ basePrice + pricePerMinute × durationMinutes ใช้ BigDecimal สำหรับราคา และตรวจว่าราคาไม่เป็นลบและระยะเวลาอยู่ระหว่าง 10 ถึง 180 นาที ตัวอย่าง ค่าเริ่มต้น 20 บาท ราคาต่อนาที 1.50 บาท ใช้ 30 นาที ได้ยอด 65 บาท

2.12 การตรวจช่วงเวลาซ้อนและการจองพร้อมกัน

ช่วงเวลาเป็นแบบรวมจุดเริ่มและไม่รวมจุดจบ จึงถือว่าซ้อนเมื่อ existing.startTime < new.endTime และ existing.endTime > new.startTime สำหรับเครื่องเดียวกันและ Session ที่มีสถานะ RESERVED หรือ IN_USE หากการจองหนึ่งจบตรงกับเวลาเริ่มของอีกการจองจะไม่ถือว่าซ้อน

การตรวจ query ก่อน insert แม้อยู่ใน transaction ยังไม่เพียงพอเมื่อมีสองคำขอพร้อมกัน เพราะทั้งสองอาจเห็นช่วงว่างเดียวกัน ระบบจึงใช้ PESSIMISTIC_WRITE บนแถวเครื่องก่อนตรวจ overlap และถือ lock จน commit คำขอถัดมารอแล้วตรวจข้อมูลที่บันทึกไปแล้ว การใช้งาน PostgreSQL ตามค่า isolation ของระบบจึงทำให้เหลือเพียงหนึ่ง reservation เมื่อคำขอซ้อนกัน

2.13 Design Patterns

- **Strategy** กำหนดกลุ่มอัลกอริทึมให้สลับใช้แทนกันได้ผ่าน interface เดียว ใช้กับวิธีชำระเงิน (`PaymentProcessor`) [Gamma et al., 1994]
- **Factory** รวมการเลือกวัตถุที่ถูกต้องไว้จุดเดียว ในโปรเจคเป็น *Simple Factory แบบ registry* (Spring ฉีด processor ทุกตัวแล้วค้นด้วย Map ตาม enum) ไม่ใช่ GoF *Factory Method* ตามตำรา

- **Facade** ให้ interface เดียวแก่ระบบย่อยหลายส่วน ใช้กับขั้นตอน Checkout (`CheckoutFacade`)
- **Observer** ให้ผู้ประกาศเหตุการณ์ไม่ต้องรู้จักผู้รับ ใช้ผ่าน `ApplicationEventPublisher` และ `@EventListener` ของ Spring

2.14 หลัก SOLID และสถาปัตยกรรมแบบชั้น

Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion [Martin, 2003] และ Layered Architecture (Controller ⟶ Service ⟶ Repository) รวมถึง DTO + Mapper และ Dependency Injection แบบ constructor [Fowler, 2002]

2.15 REST API, HTTP Status และ OpenAPI

ความหมายของรหัสสถานะ HTTP ตาม RFC 9110 (400, 401, 403, 404, 409, 500) และ OpenAPI/Swagger สำหรับเอกสาร API ที่โต้ตอบได้ (springdoc-openapi)

2.16 ฐานข้อมูลและการควบคุมความถูกต้อง

PostgreSQL กับ constraint (`PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `CHECK`) และ Flyway สำหรับจัดการเวอร์ชันของ schema

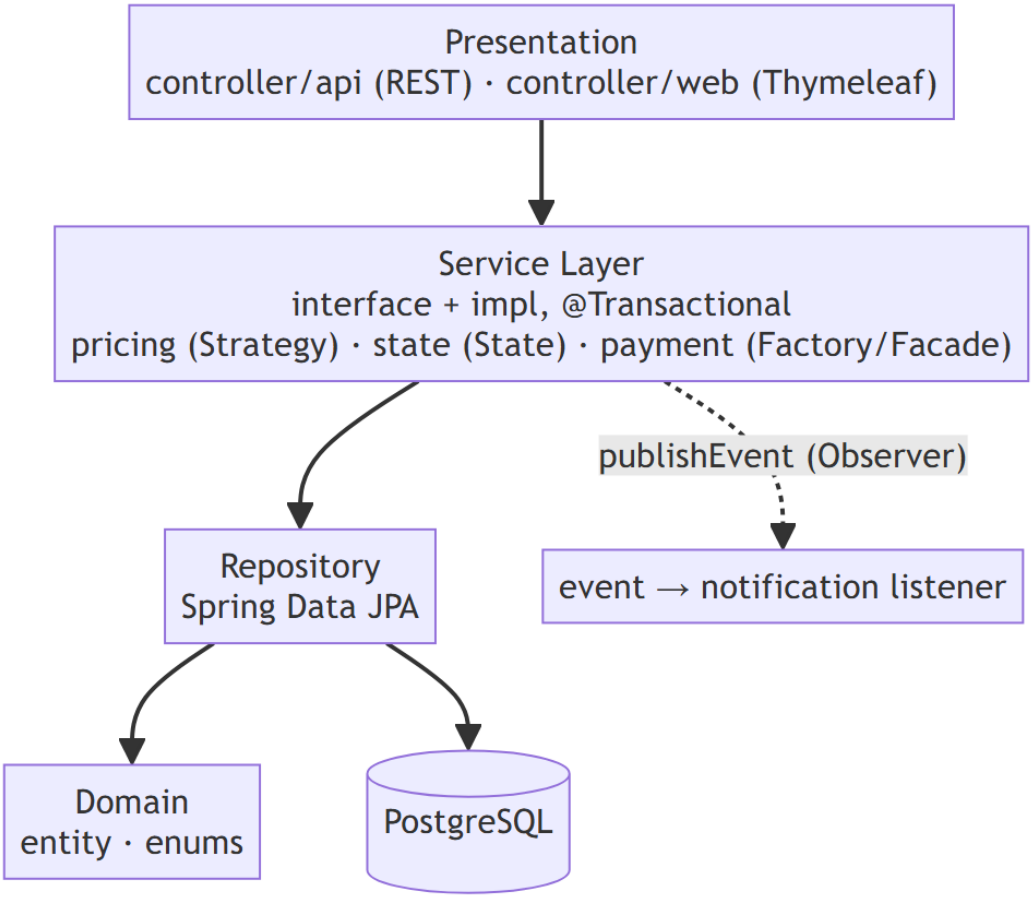
2.17 การทดสอบซอฟต์แวร์

JUnit 5 และ Mockito สำหรับ Unit Test, Spring MockMvc (standalone) สำหรับทดสอบ controller โดยไม่ต้องรันแอปทั้งตัว และการทดสอบ repository กับฐานข้อมูลจริงด้วย `@DataJpaTest` พร้อม schema ชั่วคราว

บทที่ 3 วิธีดำเนินการและการออกแบบระบบ

3.1 สถาปัตยกรรมภาพรวม

ระบบแบ่งเป็นชั้น (Layered Architecture) ตามภาพด้านล่าง คำขอผ่าน Presentation ไปยัง Service แล้วจึงถึง Repository และฐานข้อมูล ตามกฎที่ตรวจได้คือ **Controller ไม่ import Repository** (ตรวจด้วยคำสั่งค้นหาทั้งโปรเจคแล้วไม่พบ) และทุกคลาสรับ dependency ผ่าน constructor เท่านั้น เมื่อสถานะในระบบเปลี่ยน Service ประกาศ Domain Event ผ่าน `ApplicationEventPublisher` ให้ส่วนแจ้งเตือนรับไปทำต่อ (Observer) โดยไม่ผูกกับผู้ประกาศ



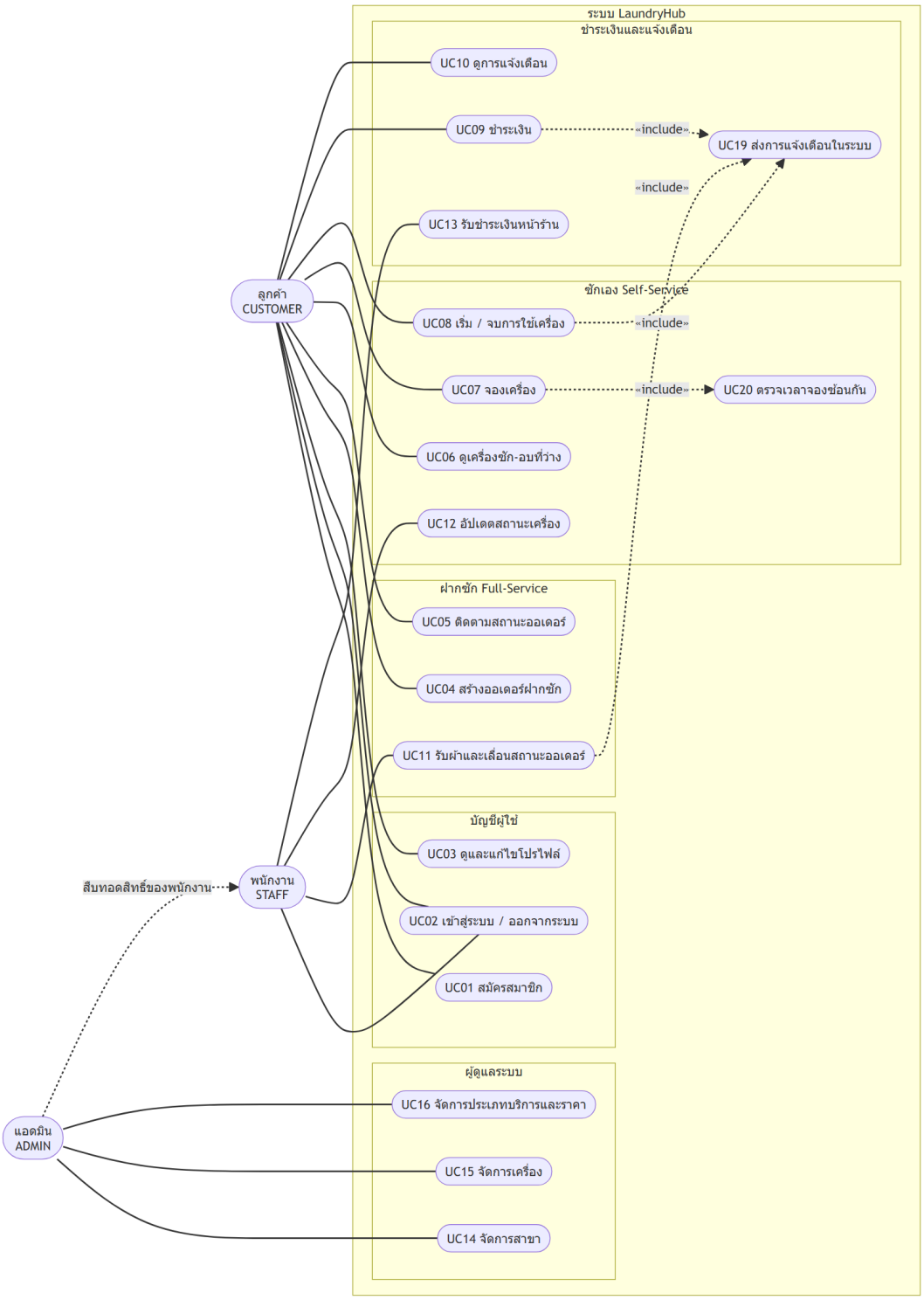
ภาพที่ 1 ภาพรวมสถาปัตยกรรมแบบชั้นของระบบ LaundryHub

ชั้น / แพ็กเกจ	หน้าที่	ตัวอย่างในส่วนของผู้เขียน
`controller/api`, `controller/web`	รับคำขอ ตรวจข้อมูล (`@Valid`) และสิทธิ์ ส่งต่อให้	`AuthApiController`, `BranchApiController`,

	Service	AuthWebController
service, service/impl	กฎธุรกิจและธุรกรรม	AuthServiceImpl, UserServiceImpl, BranchServiceImpl
repository	เข้าถึงฐานข้อมูลด้วย Spring Data JPA	UserRepository, BranchRepository
domain/entity, domain/enums	Entity และค่าคงที่ที่ตรงกับ CHECK ในฐานข้อมูล	User, CustomerProfile, Branch, Role
dto, mapper	แยกข้อมูลเข้า/ออก API จาก Entity	RegisterRequest, UserResponse, UserMapper
security, config, common	ความปลอดภัย การตั้งค่า และเครื่องมือกลาง	SecurityConfig, AppUserDetails, SecurityUtils

3.2 แผนภาพ Use Case

ผู้ใช้มี 3 บทบาท ผู้ดูแลระบบสืบทอดสิทธิ์ของพนักงาน และมี use case ที่ระบบทำเองคือการส่งแจ้งเตือน ดังภาพด้านล่าง รายละเอียดขั้นตอนหลัก ขั้นตอนทางเลือก และรหัสสถานะของ use case หลัก 8 รายการ อยู่ที่ภาคผนวก ข



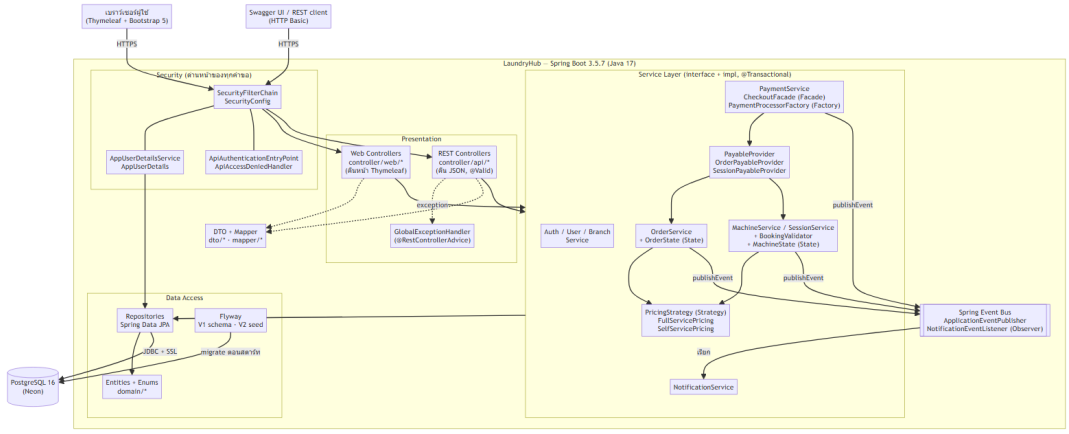
ภาพที่ 2 แผนภาพ Use Case ของระบบ LaundryHub

บทบาท	**Use case หลัก**
ลูกค้า	สมัครสมาชิก เข้าสู่ระบบ จัดการโปรไฟล์ สร้างและติดตามออเดอร์ฝากซัก

	ดูเครื่องว่างและจองเครื่อง เริ่ม/จบการใช้เครื่อง ชำระเงิน ดูการแจ้งเตือน
พนักงาน	เลื่อนสถานะออเดอร์ อัปเดตสถานะเครื่อง รับชำระเงินหน้าร้าน
ผู้ดูแลระบบ	จัดการสาขา จัดการเครื่อง จัดการประเภทบริการและราคา (และทำได้ทุกอย่างของพนักงาน)

3.3 แผนภาพ Component

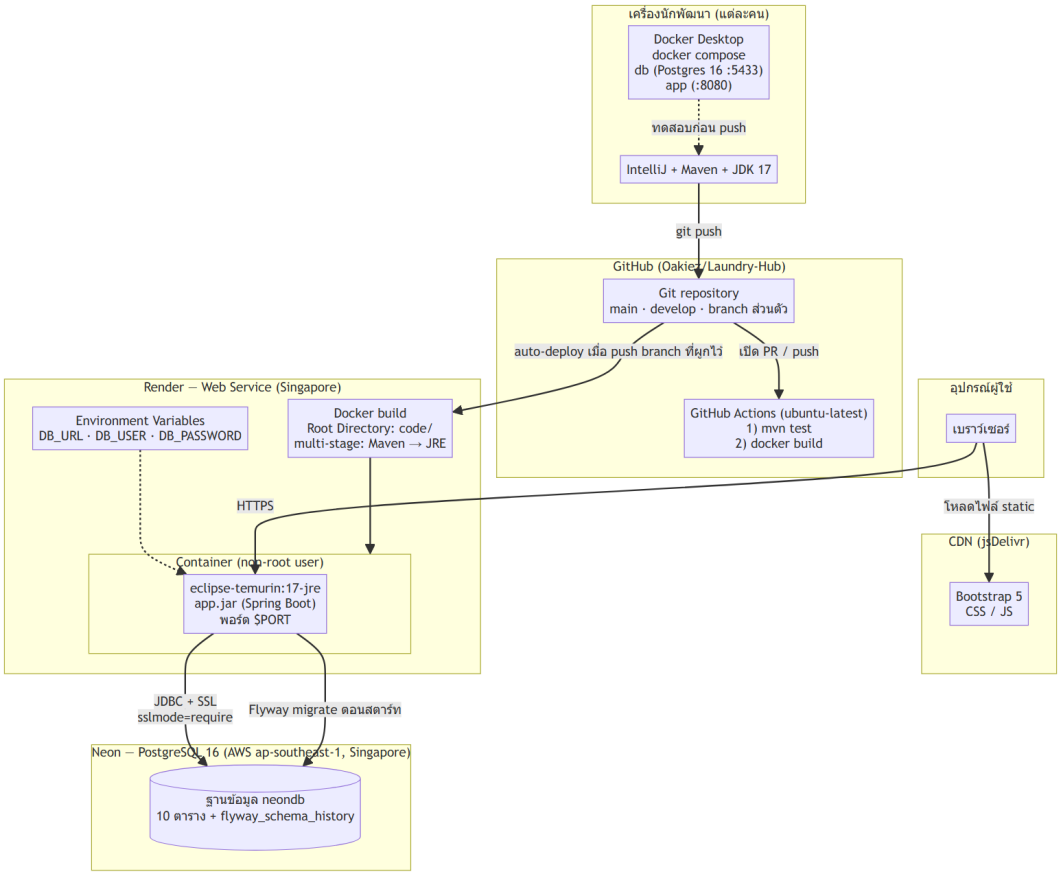
ภาพด้านล่างแสดงส่วนประกอบหลักและทิศทางการพึ่งพา ด่านความปลอดภัยอยู่หน้าสุดของทุกคำขอ ชั้น Presentation เรียก Service ผ่าน interface Service บางส่วนพึ่งพาตัวคำนวณราคา (Strategy) และตัวควบคุมสถานะ (State) ส่วน Payment หาสิ่งที่ต้องชำระผ่าน `PayableProvider` จึงไม่ต้องรู้จักโมดูลออเดอร์หรือเครื่องซักโดยตรง



ภาพที่ 3 แผนภาพ Component ของระบบ

3.4 แผนภาพ Deployment

ภาพด้านล่างแสดงเส้นทางตั้งแต่โค้ดจนถึงผู้ใช้ นักพัฒนา push โค้ดขึ้น GitHub ซึ่งรัน CI และแจ้ง Render ให้ build image จาก `code/Dockerfile` แล้วรันเป็น container ที่เชื่อมต่อฐานข้อมูลบน Neon ผ่าน JDBC แบบเข้ารหัส (`sslmode=require`) ค่าเชื่อมต่อฐานข้อมูลอยู่ในตัวแปรสภาพแวดล้อมบน Render ไม่อยู่ในโค้ด



ภาพที่ 4 แผนภาพ Deployment ของระบบ

3.5 การออกแบบระบบยืนยันตัวตนและความปลอดภัย

3.5.1 ช่องทางการยืนยันตัวตน

- **หน้าเว็บ**: ฟอร์มที่ `/login` ใช้เซสชัน เมื่อสำเร็จ Spring Security เก็บผู้ใช้ไว้ในเซสชัน ต่อมาจึงไม่ต้องกรอกซ้ำ ฟอร์มทุกอันมีโทเค็น CSRF
- **REST API และ Swagger**: ใช้ HTTP Basic ส่งชื่อและรหัสผ่านมากับทุกคำขอ ผู้ทดสอบกดปุ่ม Authorize ใน Swagger จึงไม่จำเป็นต้องมี endpoint เข้าสู่ระบบแยก (ปิด CSRF เฉพาะ `/api/**`)
- `AppUserDetailsService` ค้นผู้ใช้จากฐานข้อมูลแล้วคืน `AppUserDetails` ที่เก็บค่าแฮช บทบาท (นำหน้าด้วย `ROLE_`) และ id ของผู้ใช้ไว้ในตัว เพื่อให้เงื่อนไขสิทธิ์อ้าง id ได้โดยไม่ต้องค้นฐานข้อมูลซ้ำ

3.5.2 การเก็บรหัสผ่านและการป้องกันข้อมูลหลุด

รหัสผ่านถูกเข้ารหัสด้วย BCrypt ก่อนบันทึก และจำกัดความยาวไม่เกิน 72 ตัวอักษรเพราะ BCrypt ใช้เพียง 72 ไบต์แรก แบบฟอร์มสมัครสมาชิก (`RegisterRequest`) **ไม่มีช่อง** `role` จึงสมัครได้เฉพาะบัญชีลูกค้า ส่วนข้อมูลที่ตอบกลับ (`UserResponse`) **ไม่มีช่องรหัสผ่าน** ชื่อผู้ใช้และอีเมลตรวจความซ้ำโดยไม่สนตัวพิมพ์เล็ก-ใหญ่

3.5.3 การควบคุมสิทธิ์

ทรัพยากร	ผู้ที่ไม่ล็อกอิน	ลูกค้า	พนักงาน	ผู้ดูแลระบบ
สมัครสมาชิก, หน้า login, Swagger	ทำได้	ทำได้	ทำได้	ทำได้
ข้อมูลผู้ใช้และโปรไฟล์ของ id หนึ่ง	401	เฉพาะเจ้าของ	เฉพาะเจ้าของ	ทำได้ทุก id
ดูสาขา	401	ทำได้	ทำได้	ทำได้
เพิ่ม/แก้/ลบสาขา	401	403	403	ทำได้
หน้า `/admin/**`	เด้งไปหน้า login	403	403	ทำได้
หน้า `/staff/**`	เด้งไปหน้า login	403	ทำได้	ทำได้

กฎระดับโซน (`/admin/**`, `/staff/**`) อยู่ใน `SecurityConfig` ส่วนกฎระดับ endpoint ให้แต่ละ Controller ประกาศเองด้วย `@PreAuthorize` เช่น `hasRole('ADMIN') or #id == authentication.principal.id` (เจ้าของหรือแอดมิน) เพื่อให้โมดูลใหม่เพิ่มกฎได้โดยไม่ต้องแก้ไฟล์กลาง ผลคือเมื่อสิทธิ์ไม่พอ ระบบตอบ 403 ก่อนตรวจว่ามีข้อมูลนั้นหรือไม่ เพื่อไม่ให้ผู้ไม่มีสิทธิ์ไล่เดา id ได้

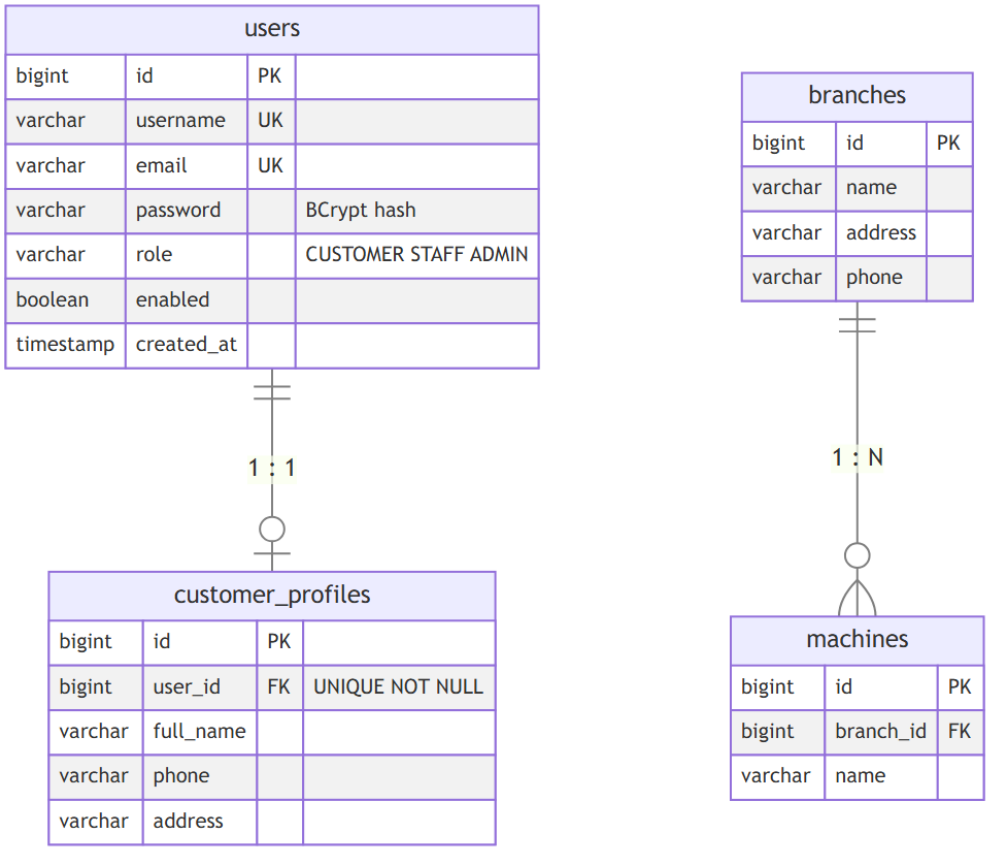
3.5.4 รูปแบบ error จากชั้นความปลอดภัย

`GlobalExceptionHandler` ดักได้เฉพาะ exception ที่เกิดใน Controller ส่วน 401/403 ที่ตัวกรองความปลอดภัยตัดสินเกิดก่อนถึงจุดนั้น จึงเพิ่ม `ApiAuthenticationEntryPoint` และ `ApiAccessDeniedHandler` ที่เขียน `ApiErrorResponse` (`timestamp`, `status`, `error`, `message`, `path`, `fieldErrors`) ออกมาเป็น JSON รูปแบบเดียวกับ error อื่นของระบบ สำหรับ `/api/**` ส่วนหน้าเว็บยังเปลี่ยนเส้นทางไปหน้า login ตามปกติ

3.6 การออกแบบฐานข้อมูล

ส่วนของวงศธร ธน.ยอด

ส่วนของผู้เขียนคือตาราง `users`, `customer_profiles` และ `branches` (ดังภาพด้านล่าง) ที่สร้างด้วย `V1__init_schema.sql` และมีข้อมูลตัวอย่างจาก `V2__seed_data.sql`



ภาพที่ 5 ER Diagram ของตาราง users, customer_profiles, branches และ machines

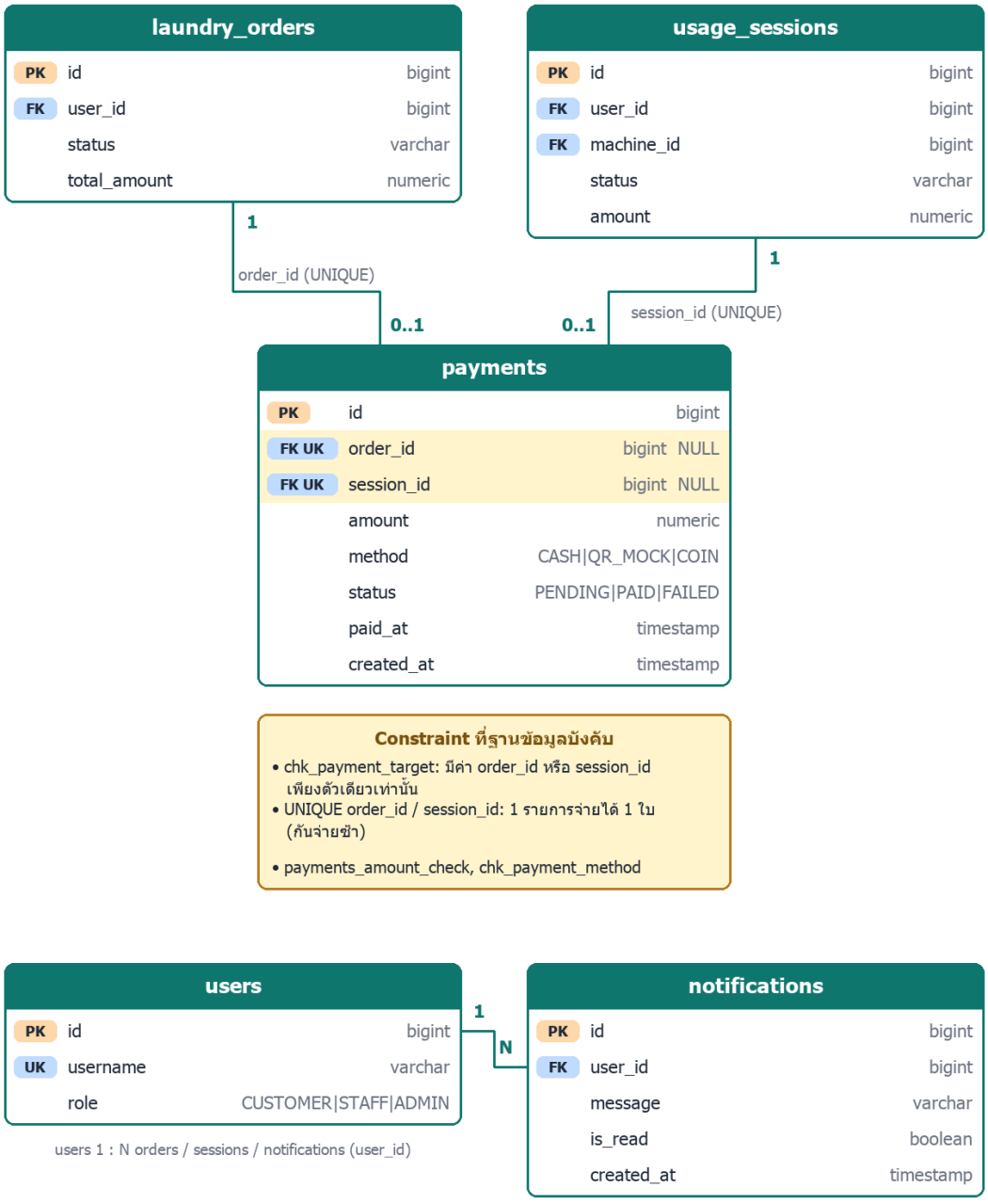
ตาราง	ข้อกำหนดสำคัญ
`users`	`username` และ `email` เป็น UNIQUE, `role` มี CHECK ให้เป็น CUSTOMER/STAFF/ADMIN, `password` เก็บค่าแฮช BCrypt, `enabled` ค่าเริ่มต้นจริง
`customer_profiles`	`user_id` เป็น NOT NULL และ UNIQUE พร้อม FK ไป `users` แบบ ON DELETE CASCADE
`branches`	ชื่อ ที่อยู่ เบอร์โทร และเป็นฝั่ง "หนึ่ง"

	ของความสัมพันธ์ One-to-Many กับ machines

ความสัมพันธ์ One-to-One ระหว่าง users กับ customer_profiles บังคับที่ฐานข้อมูลด้วย UNIQUE บน user_id (ถ้าไม่มี หนึ่งผู้ใช้จะมีได้หลายโปรไฟล์) และแมปใน JPA โดยฝั่ง CustomerProfile เป็นเจ้าของ FK (@OneToOne + @JoinColumn(unique = true)) ฝั่ง User ใช้ @OneToOne(mappedBy = "user", cascade = ALL) เพื่อให้บันทึกผู้ใช้แล้วโปรไฟล์ถูกบันทึกตามในธุรกรรมเดียว ตั้ง fetch = LAZY และเมธอด attachProfile() ตั้งค่าความสัมพันธ์ทั้งสองทิศให้ตรงกัน บัญชีพนักงานและผู้ดูแลระบบไม่มีโปรไฟล์ จึงเป็นความสัมพันธ์ "หนึ่งต่อศูนย์หรือหนึ่ง"

ส่วนของภีมเดช กลั่นกิ่ง

ตาราง payments และ notifications อยู่ใน V1__init_schema.sql (ดู ER Diagram ฉบับเต็ม 10 ตารางที่ doc/diagrams/er-diagram.md และ doc/data-dictionary.md) ภาพด้านล่างแสดงเฉพาะส่วนที่เกี่ยวกับการชำระเงินและแจ้งเตือน



ภาพที่ 6 ER Diagram ของตาราง payments และ notifications พร้อมความสัมพันธ์กับออเดอร์ รอบใช้เครื่อง และผู้ใช้

- `payments` มี FK สองตัว (`order_id`, `session_id`) ที่ nullable ทั้งคู่ เพราะหนึ่งรายการจ่ายให้ออเดอร์**หรือ**รอบใช้เครื่องอย่างใดอย่างหนึ่ง จึงควบคุมด้วย `CHECK chk_payment_target` (ต้องมีค่าเพียงตัวเดียว) และ `UNIQUE` บนทั้งสองคอลัมน์ (หนึ่งรายการมีการชำระได้ครั้งเดียว)

- Payment และ Notification เก็บ FK เป็นค่า Long ไม่ผูก @OneToOne/@ManyToOne กับ entity ของโมดูลอื่น เพื่อไม่ให้ขึ้นกับโมดูลเหล่านั้นโดยตรง (Dependency Inversion) ขณะที่ฐานข้อมูลยังบังคับ FK

3.7 การออกแบบ API

ส่วนของวงศธร ธน.ยอด

Method	Endpoint	สิทธิ์	ผลที่ออกแบบ
POST	/api/v1/auth/register	ทุกคน	201, ข้อมูลผิด 400, ชื่อหรืออีเมลซ้ำ 409
GET	/api/v1/users/{id}	เจ้าของ / ADMIN	200, ไม่มีสิทธิ์ 403, ไม่พบ 404
GET, PUT	/api/v1/users/{id}/profile	เจ้าของ / ADMIN	200, ข้อมูลผิด 400, ไม่มีโปรไฟล์ 404
GET	/api/v1/branches, /{id}	ผู้ที่ล็อกอิน	200, ไม่พบ 404
POST, PUT, DELETE	/api/v1/branches, /{id}	ADMIN	201 / 200 / 204, ข้อมูลผิด 400, สาขาที่ยังมีข้อมูลอ้างอิง 409

การสมัครสมาชิกสร้าง User และ CustomerProfile ใน @Transactional เดียวกัน หากบันทึกโปรไฟล์ไม่สำเร็จ บัญชีก็ถูกย้อนกลับ จึงไม่เกิดบัญชีที่ไม่มีโปรไฟล์ การลบสาขาสั่ง flush() ทันทีหลังลบ เพื่อให้ข้อผิดพลาดจาก foreign key (สาขายังมีเครื่องหรือออเดอร์อ้างอิง) เกิดภายในเมธอดแล้วถูกแปลงเป็น 409 ไม่หลุดเป็น 500

ส่วนของภีมเดช กลั่นกิ่ง

Method	Endpoint	สิทธิ์	ผลสำเร็จ
POST	/api/v1/payments	เจ้าของ / STAFF	201
GET	/api/v1/payments/{id}	เจ้าของ / STAFF	200

GET	/api/v1/payments?status=&page=&size=&sort=	STAFF / ADMIN	200 (แบ่งหน้า)
PATCH	/api/v1/payments/{id}/confirm	STAFF / ADMIN	200
GET	/api/v1/users/{userId}/notifications?unread=	เจ้าของ / ADMIN	200 (แบ่งหน้า)
PATCH	/api/v1/notifications/{id}/read	เจ้าของ	200

รูปแบบ error มาตรฐาน (ApiErrorResponse): timestamp, status, error, message, path, fieldErrors[]

3.8 การออกแบบหน้าเว็บ

หน้าเว็บใช้ Thymeleaf และ Bootstrap 5 โดยทุกหน้าใช้ส่วนประกอบร่วมจาก templates/fragments/layout.html ได้แก่ ส่วนหัว แถบเมนูที่แสดงตามบทบาทด้วยคำสั่ง sec:authorize ส่วนแสดงข้อความแจ้งผล (success/error/info) และส่วนท้าย โมดูลอื่นเพิ่มเมนูของตนในจุดที่กำหนดไว้ในไฟล์เดียวกัน การส่งฟอร์มใช้รูปแบบ Post/Redirect/Get คือหลังบันทึกสำเร็จจะเปลี่ยนเส้นทางไปหน้าแสดงผลพร้อมข้อความ (flash message) เพื่อกันการส่งฟอร์มซ้ำเมื่อรีเฟรช หน้าของผู้เขียนมี หน้าแรก เข้าสู่ระบบ สมัครสมาชิก โปรไฟล์ของฉัน และจัดการสาขา (รายการ/ฟอร์ม)

3.9 การออกแบบการ Deploy และ CI/CD

1. **Dockerfile แบบ multi-stage**: ขั้น build ใช้ Maven กับ JDK 17 คัดลอก pom.xml และดาวน์โหลด dependency ก่อนคัดลอกโค้ด เพื่อให้ Docker จำชั้นนี้ไว้ได้เมื่อแก้โค้ด ชั้นรันใช้เฉพาะ JRE 17 คัดลอกไฟล์ .jar มา รันด้วยผู้ใช้ที่ไม่ใช่ root และตั้ง -XX:MaxRAMPercentage=75 เพื่อให้เหมาะกับหน่วยความจำ 512 MB

ของแผนฟรี ขั้น build ข้ามเทสต์เพราะโฟลเดอร์ test/ อยู่นอก code/ ตามข้อกำหนดของรายวิชา

2. **docker-compose:** มีบริการฐานข้อมูล PostgreSQL 16 พร้อม healthcheck และบริการแอปที่รอจนฐานข้อมูลพร้อมรับการเชื่อมต่อก่อนสตาร์ท
3. **Render และ Neon:** แอปรันบน Render แบบ Docker (ภูมิภาคสิงคโปร์ Southeast Asia) ฐานข้อมูลอยู่บน Neon (ภูมิภาคสิงคโปร์) แยกจากแอป ข้อมูลจึงไม่หายเมื่อ deploy ใหม่ ค่า DB_URL, DB_USER, DB_PASSWORD ตั้งเป็นตัวแปรสภาพแวดล้อมบน Render และใช้ที่อยู่แบบเชื่อมต่อตรง (ไม่ผ่าน connection pooler) เพราะ Flyway ต้องการการเชื่อมต่อโดยตรงขณะรัน migration
4. **GitHub Actions:** ไฟล์ .github/workflows/ci.yml มี 2 งาน คือ Test (mvn test) และ Docker build รันทุก Pull Request ที่เข้า develop หรือ main และทุกการ push เข้าสองสาขานี้
5. **Branch protection:** ตั้ง ruleset บน main และ develop ห้ามลบและห้าม force push ต้องผ่าน Pull Request ที่ได้รับอนุมัติอย่างน้อย 1 คน และต้องผ่าน CI ทั้งสองงาน

3.10 วิธีทดสอบ

ส่วนของวงศธร ธน.ยอด

1. **Unit Test ชั้น Service** ด้วย JUnit 5 และ Mockito โดยจำลอง Repository จึงไม่ต้องใช้ฐานข้อมูล ทำได้เพราะทุกคลาสรับ dependency ผ่าน constructor
2. **ทดสอบกฎสิทธิ์ระดับ Controller** (SecurityRulesTest) ด้วย @WebMvcTest และ MockMvc โดยนำ SecurityConfig จริงมาใช้ ตรวจว่าใครเรียก endpoint ใดได้ และรูปแบบ error ถูกต้อง
3. **พิสูจน์ว่าเทสต์จับข้อผิดพลาดได้จริง:** ถอดการแก้ที่เกี่ยวข้องออกชั่วคราวแล้วรันเทสต์ ต้องล้มตามที่คาด จากนั้นใส่กลับแล้วผ่าน
4. **ทดสอบกับระบบจริง:** รันแอปบน PostgreSQL ใน Docker แล้วเรียก API ด้วย curl ใช้บัญชีตัวอย่างจากไฟล์ seed และทดสอบหน้าเว็บ ได้แก่ สมัคร เข้าสู่ระบบ ออกจากระบบ ด้วยการจำลองการทำงานของเบราว์เซอร์ รวมถึงบันทึกภาพหน้าจอ
5. **ตรวจเอกสาร:** ใช้สคริปต์ตรวจเลขบรรทัดอ้างอิงในเอกสาร SOLID และ Design Pattern กับโค้ดจริง ตรวจลิงก์ และตรวจไวยากรณ์แผนภาพ

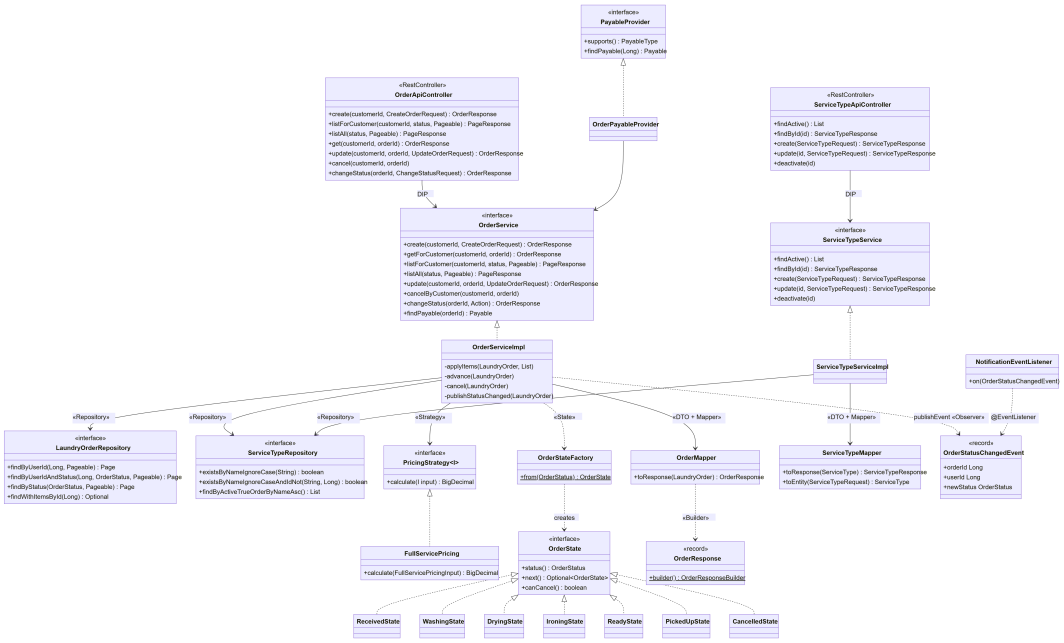
ส่วนของภีมเดช กลั่นกิ่ง

1. **Unit Test** (JUnit 5 + Mockito) ของ Factory, processor, service, facade, listener และ controller (MockMvc แบบ standalone)
2. **ทดสอบกับ PostgreSQL จริง** (`PaymentRepositoryDbTest`) ใช้ schema ชั่วคราวชื่อสุ่ม รัน Flyway แล้วลบทิ้ง เปิดใช้ด้วยตัวแปร `LAUNDRY_DB_TESTS=true` และตรวจชื่อ constraint ที่ฐานข้อมูลตอบกลับ
3. **ทดสอบแอปจริงแบบ manual** ด้วย `curl` และ Swagger UI กับผู้ใช้ตัวอย่างจากไฟล์ seed

3.11 การออกแบบโมดูลออเดอร์ฝากซัก

โมดูลออเดอร์ทำงานตามสถาปัตยกรรมแบบชั้น Controller เรียก Service ผ่าน interface และ Service เรียก Repository โดย Controller ไม่ import Repository เลย

ชั้น	คลาส	หน้าที่
Presentation	`OrderApiController`, `ServiceTypeApiController` (REST) · `OrderWebController` (Thymeleaf)	รับคำขอ ตรวจสิทธิ์ด้วย `@PreAuthorize` ตรวจข้อมูลด้วย `@Valid` แล้วส่งต่อ
Service	`OrderService` / `OrderServiceImpl`, `ServiceTypeService` / `ServiceTypeServiceImpl`, `OrderPayableProvider`	กฎธุรกิจ คิดราคา เปลี่ยนสถานะ ตรวจเจ้าของ ยิง event
Pattern	`PricingStrategy` + `FullServicePricing` · `OrderState` + 7 State + `OrderStateFactory`	คิดราคา (Strategy) และควบคุมลำดับสถานะ (State)
Data Access	`LaundryOrderRepository`, `ServiceTypeRepository`	Spring Data JPA ไม่มีการเขียน SQL เอง
DTO + Mapper	`CreateOrderRequest`, `OrderResponse`, `PageResponse`, `OrderMapper`, `ServiceTypeMapper`	แยก Entity ออกจากรูปแบบข้อมูลของ API



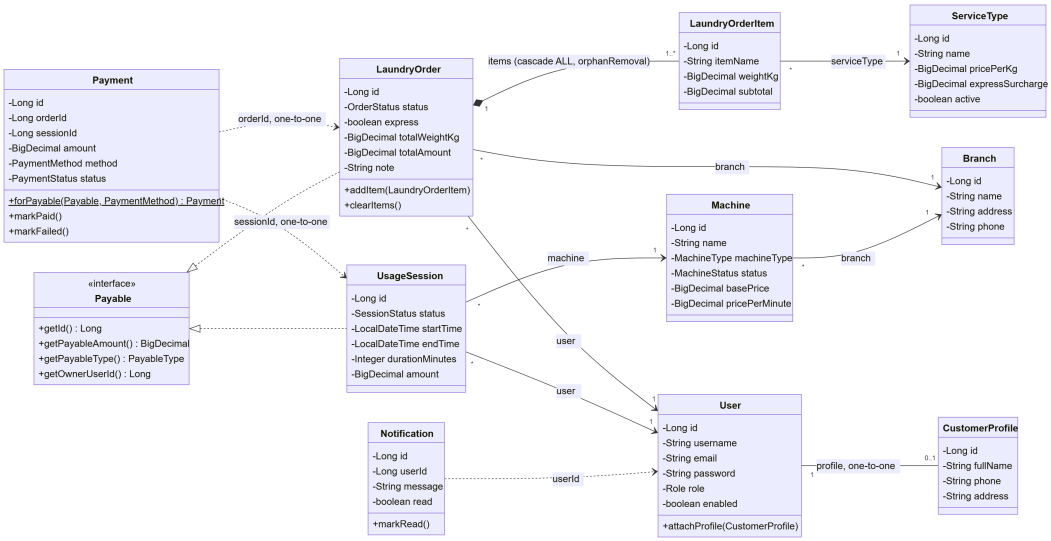
ภาพที่ 7 Class Diagram ของโมดูลออเดอร์ฝากซักและประเภทบริการ พร้อมตำแหน่ง Design Pattern

3.12 การออกแบบฐานข้อมูลของโมดูลออเดอร์

โมดูลนี้ใช้ 3 ตารางจาก `V1__init_schema.sql` ได้แก่ `service_types`, `laundry_orders` และ `laundry_order_items`

ความสัมพันธ์	ชนิด	การตั้งค่าใน Entity
`users` → `laundry_orders`	One-to-Many	`LaundryOrder.user` เป็น `@ManyToOne(fetch = LAZY)`
`branches` → `laundry_orders`	One-to-Many	`LaundryOrder.branch` เป็น `@ManyToOne(fetch = LAZY)`
`laundry_orders` → `laundry_order_items`	One-to-Many (composition)	`cascade = ALL, orphanRemoval = true`
`service_types` → `laundry_order_items`	One-to-Many	`LaundryOrderItem.serviceType` เป็น `@ManyToOne(fetch = LAZY)`

ประเภทบริการลบแบบ soft delete (`active = false`) แทนการลบแถวจริง เพราะรายการผ้าของออเดอร์เก่ายังอ้างถึงผ่าน foreign key `service_type_id`



ภาพที่ 8 Domain Model ทั้งระบบ

3.13 การคิดราคา

ราคาคิดที่ฝั่งเซิร์ฟเวอร์เสมอ คำขอสร้างออเดอร์ (`CreateOrderRequest`) ไม่มีช่องราคาหรือยอดรวม ลูกค้าจึงตั้งราคาเองไม่ได้

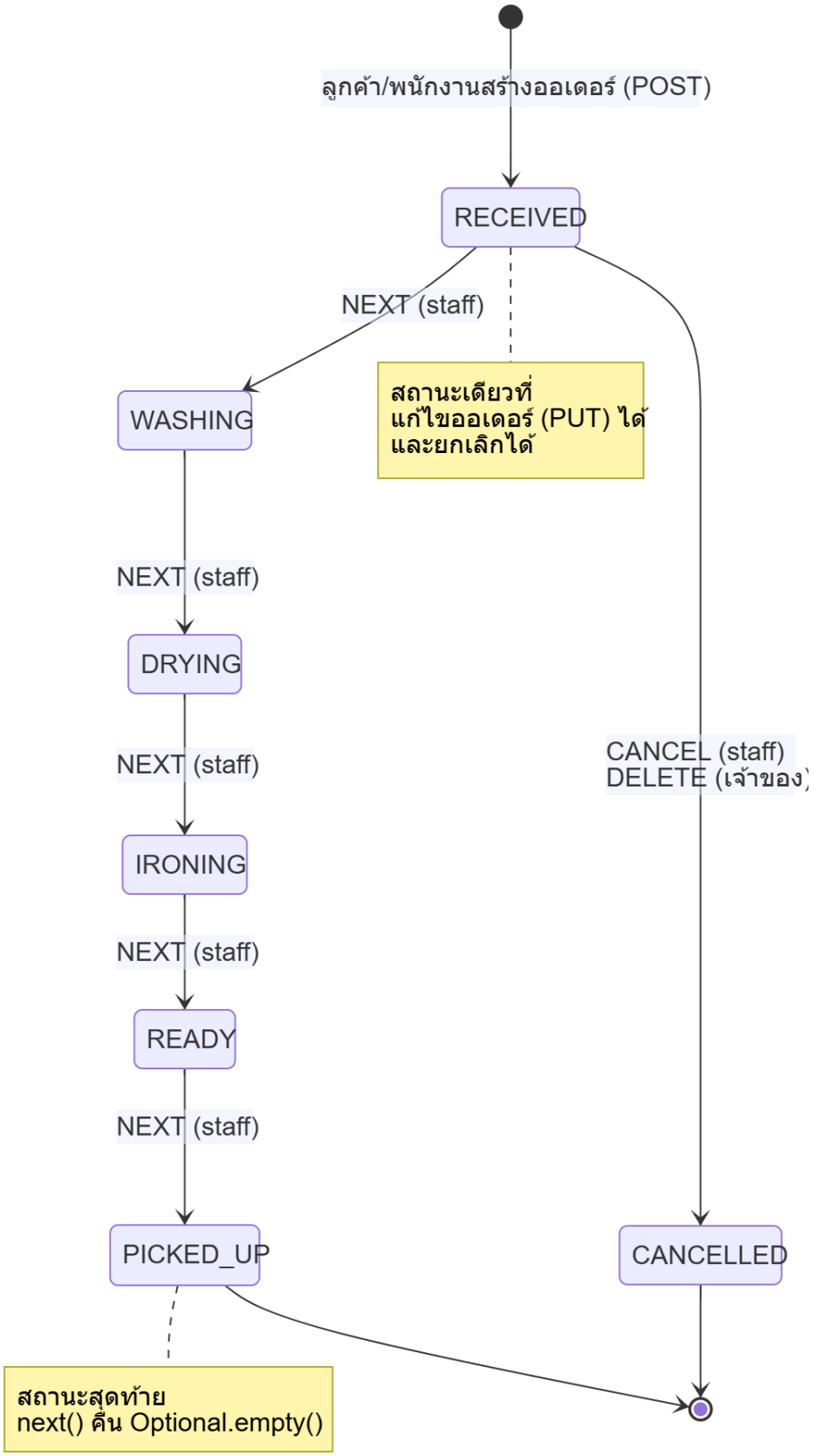
- ราคาต่อรายการ = น้ำหนัก × (ราคาต่อกก. + ค่าด่วนต่อกก. ถ้าเลือกงานด่วน) ปัดทศนิยม 2 ตำแหน่งแบบ HALF_UP
- ยอดรวมและน้ำหนักรวม = ผลรวมของทุกรายการ คำนวณใน `OrderServiceImpl.applyItems()`
- ข้อมูลที่ใช้คิดราคาถูกตรวจใน compact constructor ของ `FullServicePricingInput` (น้ำหนักต้องมากกว่า 0 ราคาห้ามว่าง) ข้อมูลผิดได้ 400 แทน 500
- ตัวอย่าง: ซักธรรมดา 25 บาท/กก. ค่าด่วน 10 บาท/กก. น้ำหนัก 2 กก. แบบด่วน = 2 × (25 + 10) = 70.00 บาท

3.14 การออกแบบสถานะออเดอร์

ออเดอร์เริ่มที่ RECEIVED และเดินหน้าได้ทีละขั้นจนถึง PICKED_UP ยกเลิกได้เฉพาะตอน RECEIVED และแก้ไขรายการได้เฉพาะตอน RECEIVED เช่นกัน การทำผิดกฎได้ `BusinessRuleException` (400) และสถานะไม่เปลี่ยน

`OrderServiceImpl.advance()` เรียก `OrderStateFactory.from(status).next()` และ `cancel()` เรียก `canCancel()` จึงไม่มี `if/switch` ตามสถานะใน Service ปุ่ม "ขั้นถัดไป" บนบอร์ดพนักงานก็ถาม `next()` จากคลาส State เช่นกัน ลำดับสถานะจึงกำหนดไว้ที่เดียว

ทุกครั้งที่สถานะเปลี่ยน Service ยิง `OrderStatusChangedEvent` ผ่าน `ApplicationEventPublisher` แล้ว `NotificationEventListener` ของโมดูลแจ้งเตือนสร้างข้อความให้ลูกค้า (Observer) โดย Service ไม่รู้จักระบบแจ้งเตือน


ลูกค้า/พนักงานสร้างออเดอร์ (POST)
RECEIVED
NEXT (staff)
สถานะเดียวที่
แก้ไขออเดอร์ (PUT) ได้
และยกเลิกได้
WASHING
NEXT (staff)
DRYING
NEXT (staff)
IRONING
NEXT (staff)
READY
NEXT (staff)
PICKED_UP
CANCEL (staff)
DELETE (เจ้าของ)
CANCELLED
สถานะสุดท้าย
next() คืน Optional.empty()

ภาพที่ 9 State Diagram ของออเดอร์ฝากซัก

3.15 การออกแบบ API และสิทธิ์การเข้าถึง

Method	Endpoint	สิทธิ์	ผลลัพธ์
POST	/api/v1/customers/{customerId}/orders	เจ้าของ / STAFF	201 / 400 / 403 / 404
GET	/api/v1/customers/{customerId}/orders?page&size&sort&status	เจ้าของ / STAFF	200 (แบ่งหน้า) / 400 / 403
GET	/api/v1/customers/{customerId}/orders/{orderId}	เจ้าของ / STAFF	200 / 403 / 404
PUT	/api/v1/customers/{customerId}/orders/{orderId}	เจ้าของ	200 / 400 / 403
DELETE	/api/v1/customers/{customerId}/orders/{orderId}	เจ้าของ (ยกเลิก)	204 / 400 / 403
PATCH	/api/v1/orders/{orderId}/status {"action":"NEXT" **หรือ** "CANCEL"}	STAFF / ADMIN	200 / 400 / 404
GET	/api/v1/orders?status&page&size&sort	STAFF / ADMIN	200 (บอร์ดพนักงาน)
GET	/api/v1/service-types, /{id}	ผู้ที่ล็อกอิน	200 / 404
POST, PUT, DELETE	/api/v1/service-types, /{id}	ADMIN	201 / 200 / 204 / 400 / 404 / 409

- สิทธิ์ตรวจ 2 ชั้น ชั้นแรก `@PreAuthorize` ตรวจว่า `customerId` ใน URL เป็นของผู้ที่ล็อกอิน (ไม่ใช่ได้ 403) ชั้นที่สองใน Service ตรวจว่าออเดอร์เป็นของลูกค้าคนนั้นจริง (ไม่ใช่ได้ 404) เพื่อไม่เปิดเผยว่ามีออเดอร์เลขนั้นอยู่
- การเรียงลำดับรับเฉพาะฟิลด์ที่อนุญาต (`id`, `createdAt`, `updatedAt`, `status`, `totalAmount`, `totalWeightKg`) ฟิลด์อื่นได้ 400 แทนที่จะเป็น 500
- ผลลัพธ์แบบแบ่งหน้าตอบเป็น `PageResponse<T>` (`content`, `page`, `size`, `totalElements`, `totalPages`)

3.16 การเชื่อมกับโมดูลชำระเงิน

`LaundryOrder` implement interface `Payable` (4 เมธอด) และ `OrderPayableProvider` ส่งออเดอร์ให้ `CheckoutFacade` ผ่าน interface `PayableProvider` โมดูลชำระเงินจึงจ่ายเงินค่าออเดอร์ได้โดยไม่รู้จัก `LaundryOrder` หรือ `OrderService` โดยตรง (Dependency Inversion)

3.17 โครงสร้างโมดูล Self-Service

โมดูลแยก Controller ⟶ Service ⟶ Repository โดย Controller รับ DTO ตรวจ validation และสิทธิ์ ส่วน Service ประสานกฎธุรกิจ การคำนวณราคา transaction และ event Repository รับผิดชอบ query และ lock Mapper แปลง Entity เป็น DTO ภายใน transaction เพื่อไม่ส่ง Entity หรือ LAZY association ไปยัง HTTP client

ส่วน	คลาสหลัก	หน้าที่
Entity	Machine, UsageSession	เครื่องและรอบใช้งาน โดย UsageSession implements Payable
Repository	MachineRepository, UsageSessionRepository	ค้นหา แบ่งหน้า ตรวจ overlap และล็อกแถว
Service	MachineServiceImpl, SessionServiceImpl	จัดการเครื่องและ flow จอง/เริ่ม/จบ/ยกเลิก
กฎแยก	BookingValidator, SelfServicePricing, MachineState	ตรวจเวลา คิดราคา และความสามารถตามสถานะ
Controller	MachineApiController,	REST API และสิทธิ์ระดับ

	SessionApiController	endpoint
DTO/Mapper	MachineRequest, BookSessionRequest, SessionResponse, MachineMapper, SessionMapper	สัญญาข้อมูลเข้า/ออก
เชื่อมชำระเงิน	SessionPayableProvider	ค้น Session ผ่าน PayableProvider กลาง

Machine อ้าง Branch และ UsageSession อ้าง Machine/User ด้วย ManyToOne แบบ LAZY โดยไม่กำหนด cascade remove ไปยังข้อมูลร่วม Foreign key ของ migration ตรวจความถูกต้องของรหัสอ้างอิง และ unique(branch_id, name) ป้องกันชื่อเครื่องซ้ำในสาขา ฐานข้อมูลเดิมมี index สำหรับ machine_id, start_time, end_time ซึ่งสนับสนุนการค้นช่วงเวลา แต่ไม่ได้แทนที่กฎป้องกัน overlap หรือ lock

3.18 ขั้นตอนการจองและการเปลี่ยนสถานะ

Service ตรวจเจ้าของคำขอและผู้ใช้ที่เป็น CUSTOMER และ enabled จากนั้นล็อก Machine เรียก BookingValidator คำนวณ endTime เรียก PricingStrategy คำนวณ amount และบันทึก Session สถานะ RESERVED ก่อนประกาศ SessionStatusChangedEvent การจองยังไม่เปลี่ยนสถานะเครื่องเป็น RESERVED

Start ตรวจ Session ว่าเป็น RESERVED และ Machine ว่าเริ่มได้ แล้วเปลี่ยนทั้งสองเป็น IN_USE Finish ตรวจ Session ว่า IN_USE และ Machine ว่าจบได้ แล้วเปลี่ยน Session เป็น COMPLETED และ Machine เป็น AVAILABLE Cancel อนุญาตเฉพาะ RESERVED และเปลี่ยนเฉพาะ Session เป็น CANCELLED เพื่อไม่ปล่อยเครื่องที่อาจกำลังใช้งานโดย Session อื่น

Lifecycle อ่าน machineId แบบ scalar ก่อนล็อก Machine แล้วจึงล็อก Session เพื่อใช้ลำดับล็อกเดียวกันทุก operation และลดการล็อกสลับลำดับ การเปลี่ยนสถานะประกาศ event หลัง flush ภายใน transaction เดียวกัน Listener ของ Session ซึ่งทีมชำระเงินจัดทำเป็น synchronous จึง rollback งานหลักด้วยเมื่อบันทึกแจ้งเตือนไม่สำเร็จ MachineStatusChangedEvent มี publisher แล้วแต่ยังไม่มี listener

3.19 REST API และการตรวจสิทธิ์

Endpoint	การทำงานและสิทธิ์
GET /api/v1/machines และ /{id}	ค้นหา/อ่านสำหรับผู้ล็อกอิน
POST, PUT, DELETE /api/v1/machines และ /{id}	CRUD สำหรับ ADMIN
PATCH /api/v1/machines/{id}/status	ตั้ง AVAILABLE/OUT_OF_SERVICE สำหรับ STAFF/ADMIN
POST /api/v1/machines/{machineId}/sessions	จองโดยเจ้าของหรือ STAFF/ADMIN ที่ทำแทนลูกค้า
GET /api/v1/machines/{machineId}/sessions	ประวัติเครื่องสำหรับ STAFF/ADMIN
GET /api/v1/users/{userId}/sessions	ประวัติของผู้ล็อกอินเอง
GET /api/v1/sessions/{id}	อ่านสำหรับเจ้าของหรือ STAFF/ADMIN
PATCH /api/v1/sessions/{id}/start, /finish, /cancel	เปลี่ยนสถานะสำหรับเจ้าของหรือ STAFF/ADMIN

Controller ใช้ PreAuthorize และอ่าน currentUserId/staff จาก authenticated principal ผ่าน SecurityUtils ไม่รับสิทธิ์เหล่านี้จาก JSON หรือ query parameter SessionService ตรวจเจ้าของอีกชั้นเมื่อเข้าถึง Session รายตัว Request ใช้ Bean Validation และผลแบ่งหน้าใช้ PageResponse ที่มี content, page, size, totalElements และ totalPages มี allowlist ของ sort fields ก่อนส่ง query

สถานะสำเร็จใช้ 201 สำหรับสร้าง 200 สำหรับอ่าน/แก้ไข และ 204 สำหรับลบ การจองซ้อนและชื่อซ้ำตอบ 409 ไม่พบตอบ 404 ไม่มีสิทธิ์ตอบ 403 และยังไม่ล็อกอินตอบ 401 Validation และผิดสถานะ lifecycle ตอบ 400 ตาม BusinessRuleException handler กลาง ซึ่งต่างจาก 409 ในตาราง API ของ brief แต่ตรงกับชนิด exception ที่ brief ระบุในเทสต์ ข้อแตกต่างนี้บันทึกไว้ให้ทีมตรวจรับ ส่วนการอ่านเครื่องต้องล็อกอินตาม Security กลางโดยไม่ได้แก้ requestMatchers

3.20 หลัก SOLID และการเชื่อมโมดูล

SRP แยก BookingValidator สำหรับกฎเวลา PricingStrategy สำหรับราคา และ Mapper สำหรับแปลงข้อมูล DIP ใช้ constructor injection และ interface ของ Service/PricingStrategy ISP ใช้ Payable ที่ให้ข้อมูลเฉพาะรหัส ยอด ประเภท และเจ้าของแก่ระบบชำระเงิน LSP ของ MachineState ใช้ผล boolean แทน exception สำหรับความสามารถที่ไม่รองรับ OCP ของระบบชำระเงินรองรับ SessionPayableProvider เพิ่มผ่าน registry เดิมโดยไม่แก้ CheckoutFacade เพื่อรู้จัก UsageSession โดยตรง

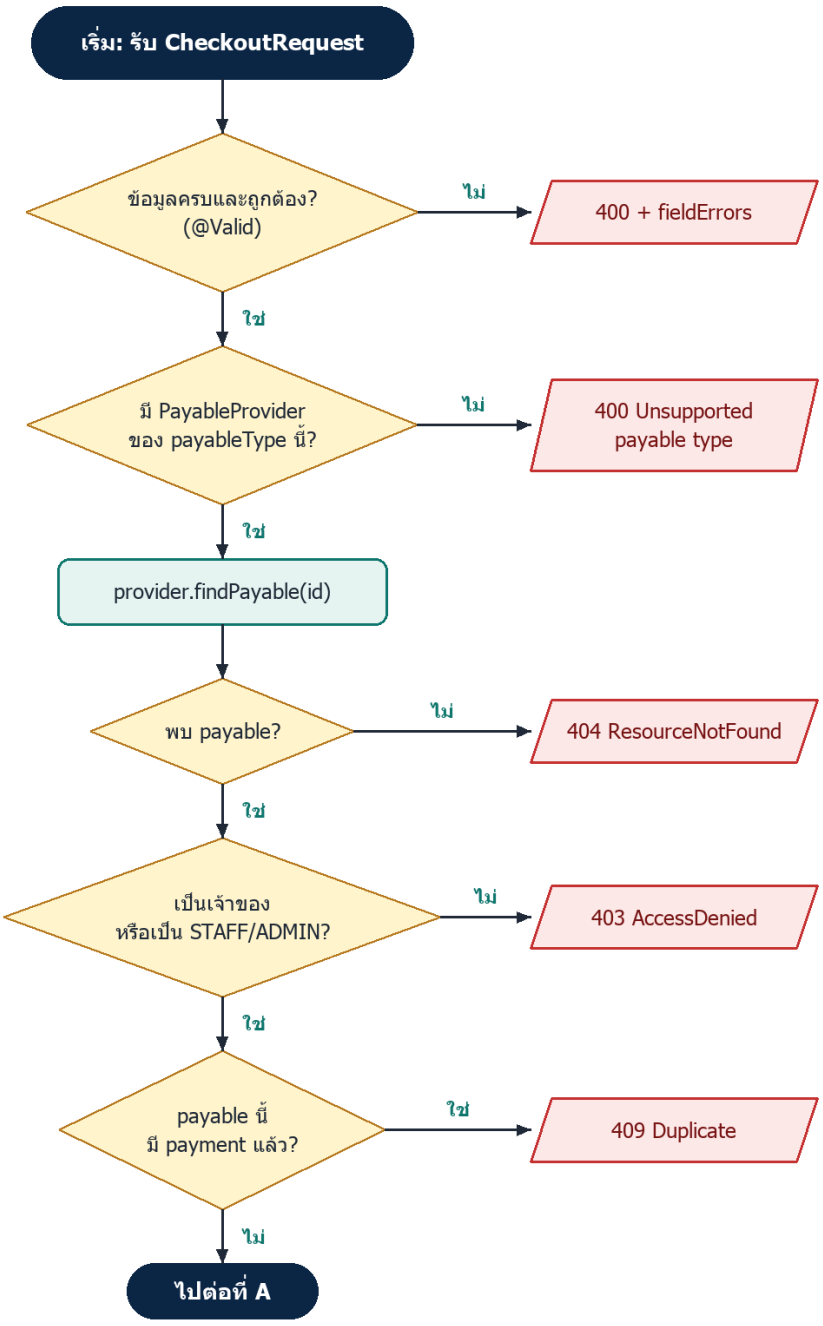
SessionService.findPayable คืนข้อมูลตาม contract แล้ว CheckoutFacade ตรวจเจ้าของใน payment transaction รอบนอก ยอด payment มาจาก amount ที่บันทึกฝั่ง server การเพิ่ม provider ไม่ได้เพิ่มการตรวจสิทธิ์ลงใน getter ของ Entity

3.21 ภาพรวมการออกแบบ

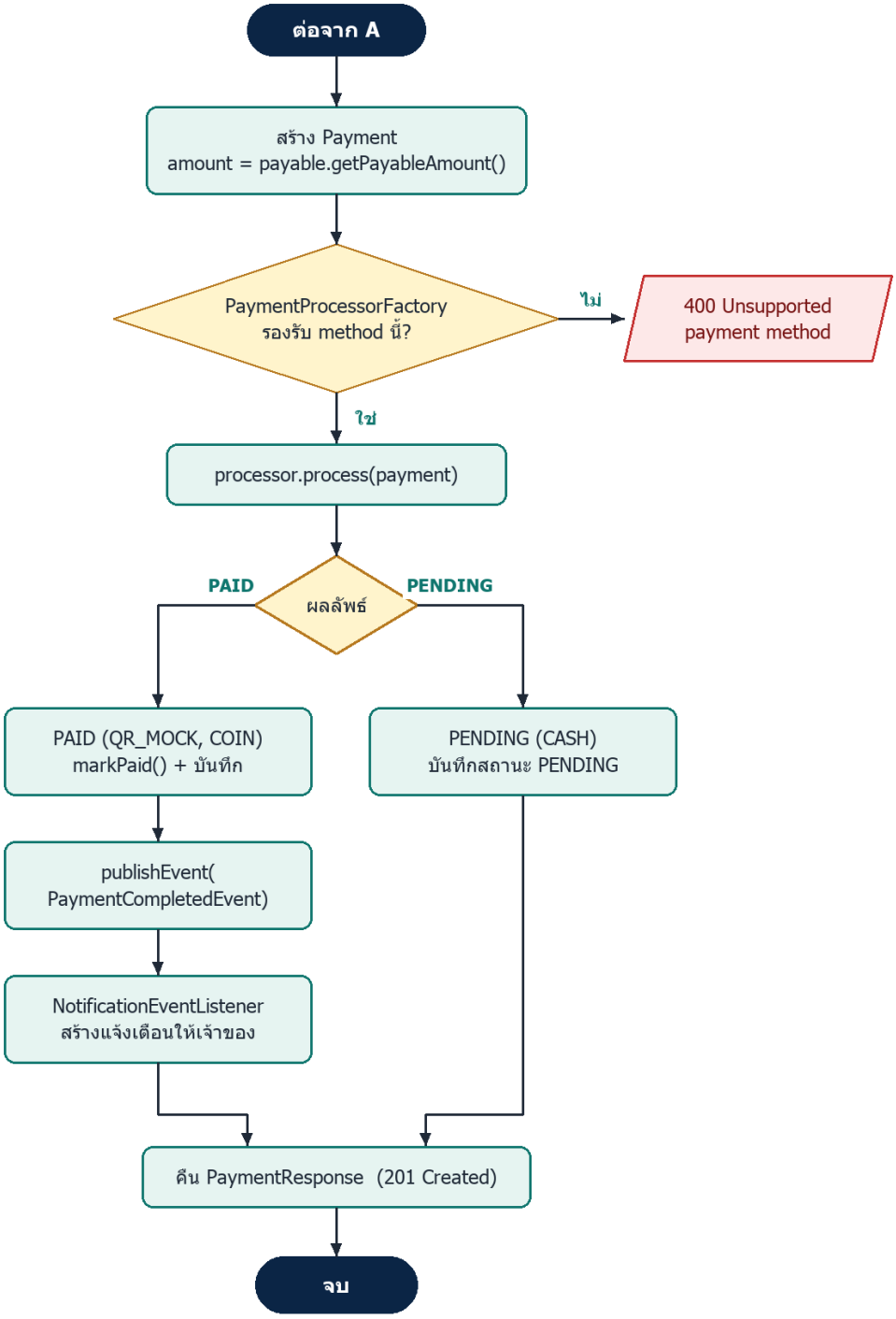
โมดูลชำระเงินและแจ้งเตือนอยู่ตรงกลางระหว่างโมดูลออเดอร์กับโมดูลเครื่องซัก โดยพึ่งพากันผ่าน interface (`Payable`, `PayableProvider`) และ event (`PaymentCompletedEvent`) เท่านั้น

องค์ประกอบ	ตำแหน่ง	หน้าที่
`PaymentApiController`, `NotificationApiController`	`…/controller/api/`	รับคำขอ ตรวจสิทธิ์เบื้องต้น แบ่งหน้า
`CheckoutFacade`	`…/service/payment/`	ประสานขั้นตอนชำระเงิน ตรวจเจ้าของ ยิง event
`PaymentServiceImpl`	`…/service/impl/`	กฎการสร้าง/ยืนยันการชำระ กันจ่ายซ้ำ
`PaymentProcessor` + `CashProcessor`, `QrMockProcessor`, `CoinProcessor`	`…/service/payment/`	วิธีชำระแต่ละแบบ (Strategy)
`PaymentProcessorFactory`	`…/service/payment/`	เลือก processor ตามวิธีชำระ
`NotificationEventListener`, `NotificationServiceImpl`	`…/event/`, `…/service/impl/`	Observer สร้างแจ้งเตือนจาก event
`GlobalExceptionHandler`, `ApiErrorResponse`	`…/exception/`, `…/dto/response/`	แปลง exception เป็นรูปแบบ error มาตรฐาน

ขั้นตอนชำระเงินแสดงเป็น Activity Diagram สองส่วน ส่วนแรกคือการตรวจข้อมูลและสิทธิ์ก่อนสร้างรายการ (ตรวจข้อมูล ⟶ มี provider ของประเภทนี้ ⟶ พบรายการ ⟶ เป็นเจ้าของหรือพนักงาน ⟶ ยังไม่เคยชำระ) ส่วนที่สองคือการสร้าง Payment เลือก processor ตามวิธีชำระ แล้วยิง event ให้ผู้ฟังสร้างแจ้งเตือน เส้นสีแดงคือกรณีผิดพลาดและรหัส HTTP ที่ตอบกลับ

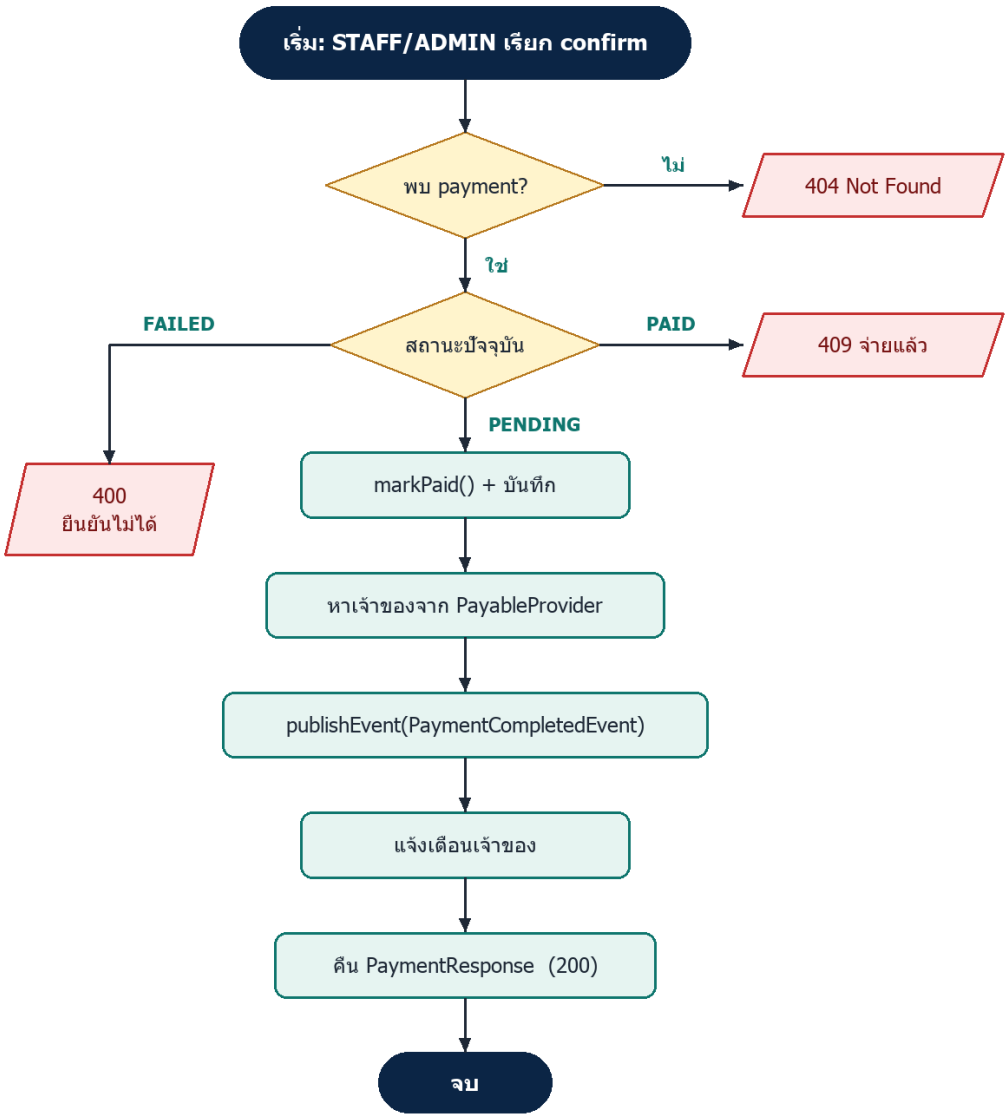


ภาพที่ 10 Activity Diagram ของ checkout ส่วนที่ 1: ตรวจข้อมูลและสิทธิ์



ภาพที่ 11 Activity Diagram ของ checkout ส่วนที่ 2: สร้าง Payment เลือก processor และยิง event

พนักงานยืนยันรับเงินสดมีขั้นตอนสั้นกว่า โดยตรวจสถานะปัจจุบันก่อน (จ่ายแล้วได้ 409, ล้มเหลวได้ 400, รอยืนยันจึงดำเนินการต่อ)



ภาพที่ 12 Activity Diagram ของการยืนยันรับเงินสดโดยพนักงาน

ตำแหน่ง Design Pattern ของโมดูลนี้ดูที่ doc/design-patterns.md และรายละเอียดต้นฉบับของแผนภาพอยู่ที่ doc/diagrams/activity-checkout.md

3.22 การออกแบบกฎธุรกิจและความปลอดภัย

1. **ยอดเงินมาจากเซิร์ฟเวอร์**: CheckoutRequest ไม่มีฟิลด์ amount ยอดมาจาก Payable.getPayableAmount() เท่านั้น

2. **ตรวจสิทธิ์สองชั้น**: `@PreAuthorize` สำหรับรายการทั้งหมดและการยืนยัน (STAFF/ADMIN) และการตรวจ "เป็นเจ้าของ payable" ใน `CheckoutFacade`
3. **กันจ่ายซ้ำสองชั้น**: ตรวจในโค้ดเพื่อให้ข้อความอ่านง่าย และ `UNIQUE` ในฐานข้อมูลเพื่อกันสองคำขอพร้อมกัน
4. **เงินสดรอยืนยัน**: เงินสดมีสถานะ `PENDING` จนกว่าพนักงานยืนยัน ส่วน QR จำลองและเหรียญได้ `PAID` ทันที (เหรียญใช้ได้เฉพาะรอบใช้เครื่อง)
5. **แจ้งเตือนแบบ synchronous**: `@EventListener` ทำงานในธุรกรรมเดียวกับผู้ยิง เพื่อให้ข้อมูลสอดคล้อง (ไม่มีกรณีจ่ายแล้วแต่ไม่มีแจ้งเตือน) โดยยอมรับว่าแจ้งเตือนที่ล้มเหลวจะกระทบธุรกรรมหลัก

บทที่ 4 ผลการพัฒนาและการทดสอบ

4.1 ผลการพัฒนา

ส่วนของวงศธร ธน.ยอด

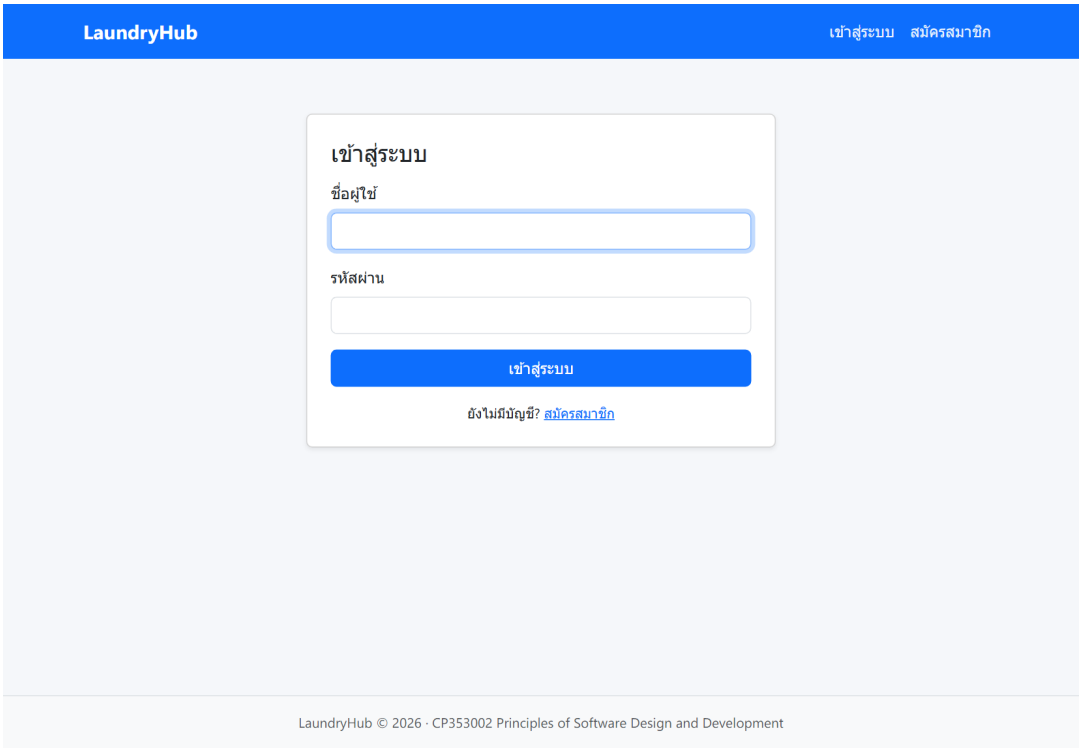
ส่วนของผู้เขียนพัฒนาครบตามบทที่ 3 ได้แก่ ระบบสมัครสมาชิกและยืนยันตัวตนทั้งฟอร์มและ HTTP Basic, การควบคุมสิทธิ์ 3 บทบาท, ข้อมูลผู้ใช้และโปรไฟล์, การจัดการสาขา, โครงหน้าเว็บกลางพร้อมหน้าที่เกี่ยวข้อง, Dockerfile และ docker-compose, การ deploy บน Render กับ Neon, และ CI ด้วย GitHub Actions ระบบที่ deploy แล้วเข้าถึงได้ที่ https://laundry-hub-1ltc.onrender.com (เอกสาร API ที่ `/swagger-ui.html`)

ส่วนของภีมเดช กลั่นกิ่ง

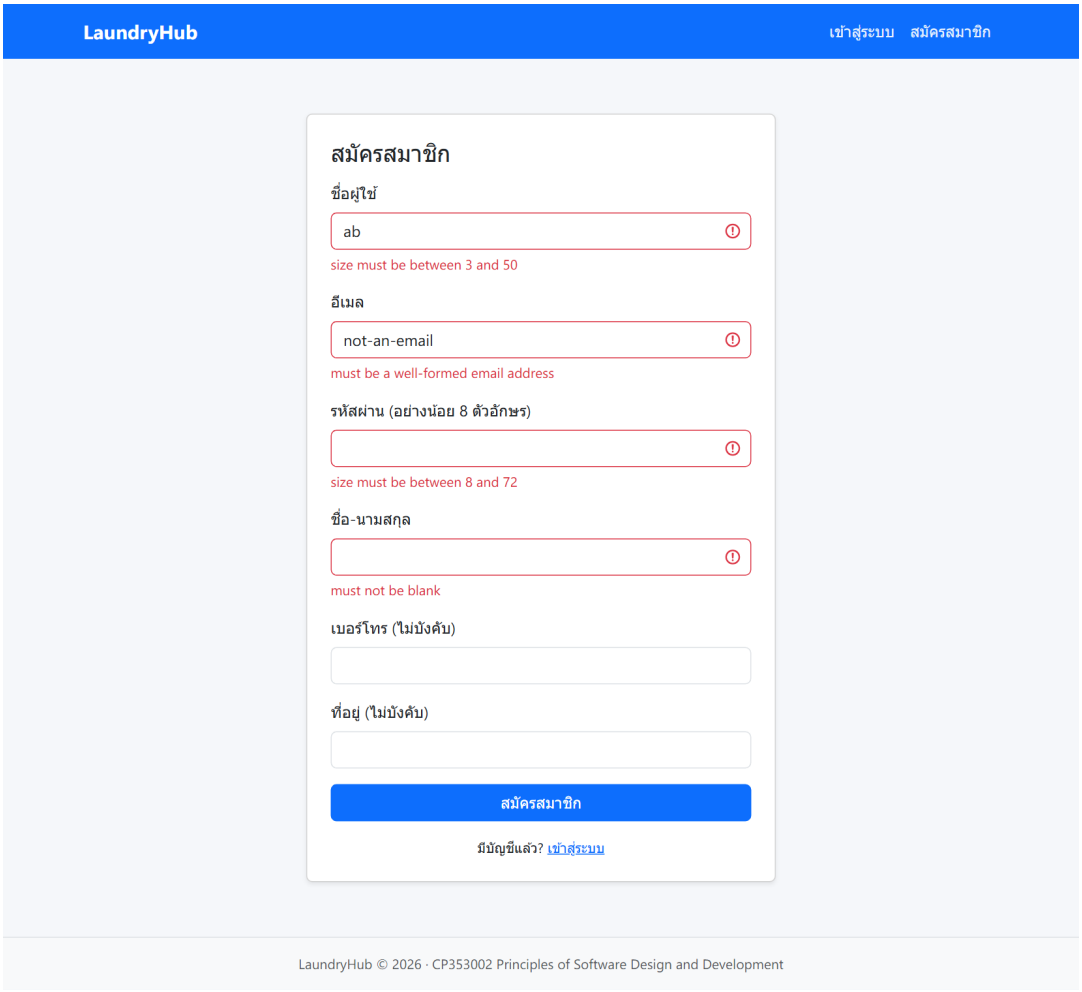
พัฒนาครบตามตาราง API ในบทที่ 3 พร้อมเอกสาร Swagger (ปุ่ม Authorize แบบ Basic Auth) และเว็บที่ deploy ที่ https://laundry-hub-1ltc.onrender.com (ตรวจพบ endpoint ของ payments/notifications ในเอกสาร API และตอบ 401 รูปแบบ `ApiErrorResponse` เมื่อไม่ล็อกอิน)

4.2 ผลหน้าเว็บ

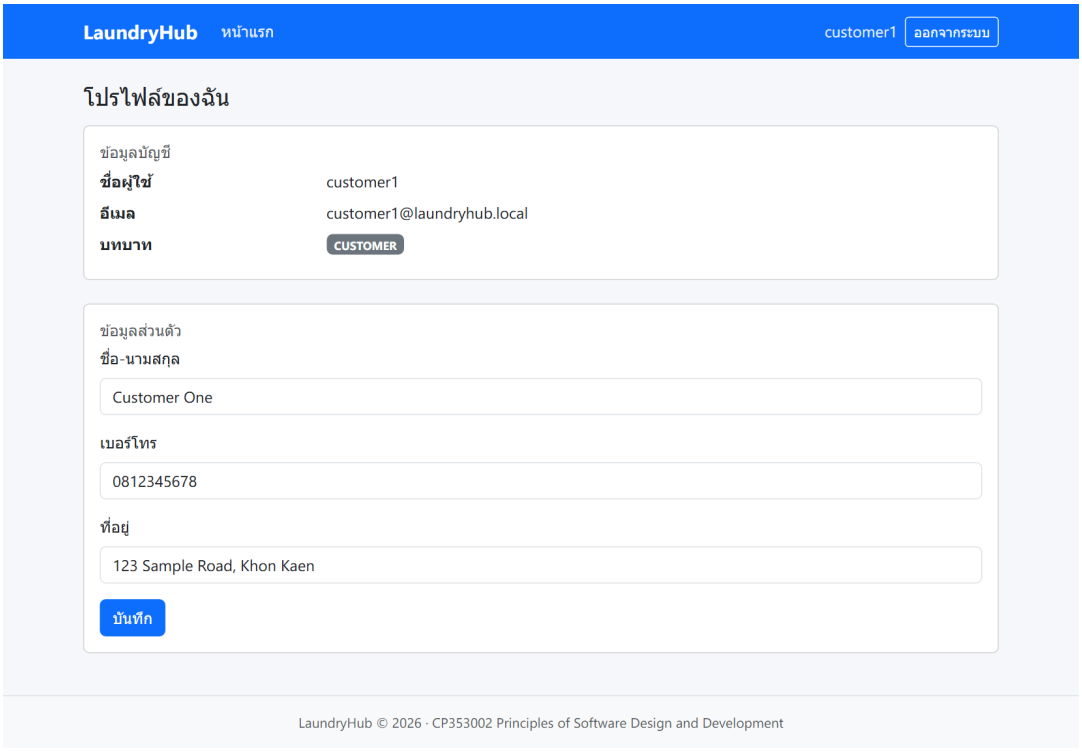
ภาพต่อไปนี้เป็นภาพหน้าจอจากแอปที่รันจริง ใช้ข้อมูลตัวอย่างจากไฟล์ seed



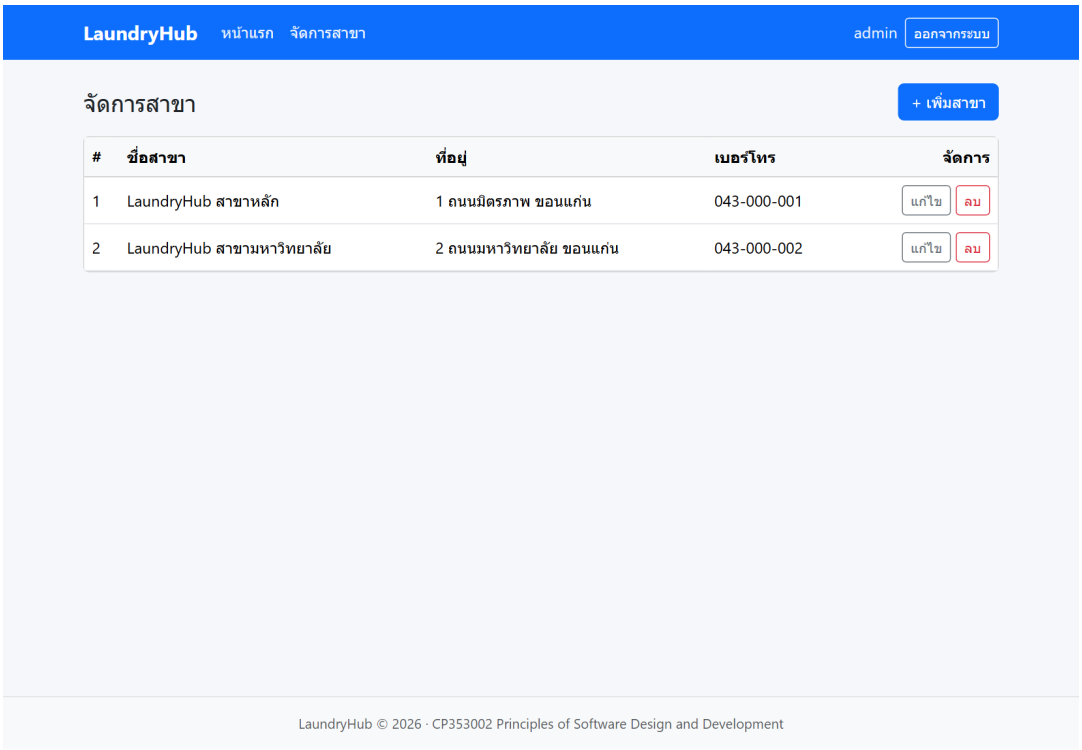
ภาพที่ 13 หน้าเข้าสู่ระบบ



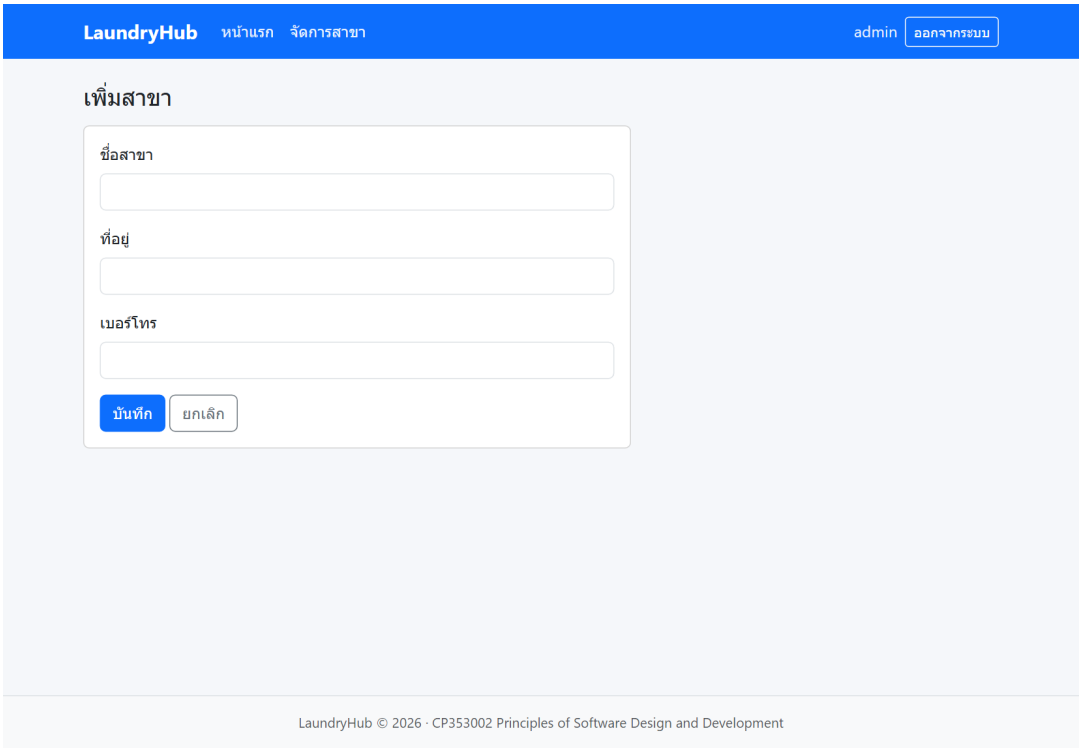
ภาพที่ 14 หน้าสมัครสมาชิกเมื่อกรอกข้อมูลไม่ถูกต้อง แสดงข้อความใต้แต่ละช่อง



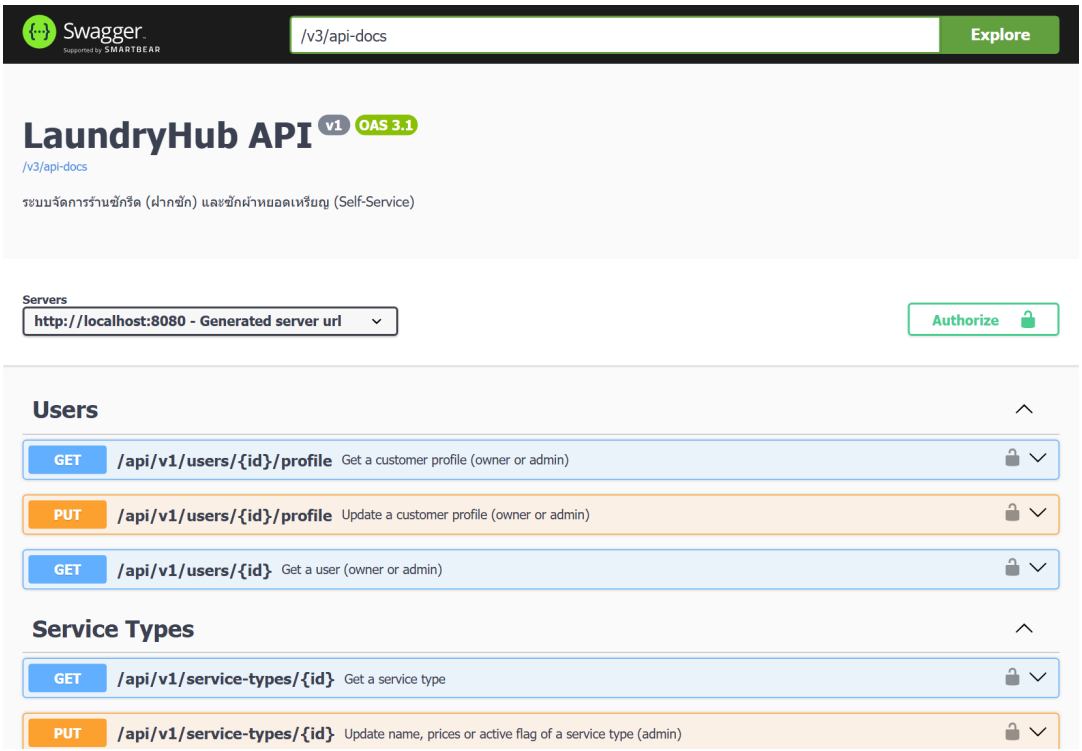
ภาพที่ 15 หน้าโปรไฟล์ของลูกค้า



ภาพที่ 16 หน้าจัดการสาขาของผู้ดูแลระบบ



ภาพที่ 17 ฟอร์มเพิ่มสาขา



ภาพที่ 18 หน้าเอกสาร API (Swagger UI) พร้อมปุ่ม Authorize

4.3 ผล Unit Test

ส่วนของวงศธร ธน.ยอด

รันเมื่อ 10 ตุลาคม 2569 ด้วย `mvn test` ผลของส่วนที่ผู้เขียนรับผิดชอบ:

คลาสเทสต์	ทดสอบอะไร	จำนวน	ผ่าน	ล้มเหลว
`AuthServiceTest`	สมัครสำเร็จ (เข้ารหัสผ่าน, role เป็นลูกค้า, ผูกโปรไฟล์), ชื่อซ้ำ, อีเมลซ้ำ ต้องไม่บันทึก	3	3	0
`BranchServiceTest`	ดู/สร้าง/แก้/ลบ, ไม่พบ, ลำดับลบแล้ว flush	8	8	0
`UserServiceTest`	ดูผู้ใช้ ดู/แก้โปรไฟล์ ไม่พบ ผลตอบกลับไม่มีรหัสผ่าน	5	5	0
`SecurityRulesTest`	สิทธิ์ตามบทบาท, เจ้าของหรือแอดมิน, รูปแบบ error 401/403/400	13	13	0
`AuthWebControllerTest`	ฟอร์มสมัครต้องยืนยันรหัสผ่าน: ไม่ตรงกัน/ไม่ส่งช่องที่สอง ต้องไม่สมัครและแสดงข้อความภาษาไทย, ตรงกันต้องสมัครแล้วไปหน้า login	3	3	0
รวม		**32**	**32**	**0**

ตัวเลขนี้เป็นของคลาสเทสต์ที่ผู้เขียนเป็นเจ้าของเท่านั้น ในช่วงปรับหน้าจอรอบสุดท้ายผู้เขียนยังเพิ่มเทสต์ในคลาสของโมดูลอื่นอีก 8 ข้อ (ซ่อนปุ่มชำระเงิน 1 ข้อในโมดูลออเดอร์, ป้ายสถานะภาษาไทยและซ่อนปุ่ม 3 ข้อในโมดูลเครื่องซัก-อบ, และ 3 ข้อระดับ service กับ 1 ข้อระดับฐานข้อมูลของโมดูลชำระเงิน) ซึ่งนับอยู่ในผลรวมทั้งโปรเจค ผลรันล่าสุดของผู้เขียนเมื่อ 10 ตุลาคม 2569 บน `develop` commit `3d4e79b` (โค้ดชุดเดียวกับ commit `399b351` ที่ต่างกันเฉพาะไฟล์เอกสาร) เปิดเทสต์ฐานข้อมูลด้วย `LAUNDRY_DB_TESTS=true` กับ PostgreSQL 16 ใน Docker ได้ 390 เทสต์ ผ่าน 390 ล้มเหลว 0 ข้าม 0 ส่วนตัวเลขในรายงานทดสอบรวม (doc/test-report/test-report.md) เป็นผลของรอบก่อนหน้า ให้ถือผลรันนี้เป็นหลักฐานล่าสุด

การพิสูจน์ว่าเทสต์สิทธิ์จับข้อผิดพลาดได้จริง: เมื่อถอดการแก้รูปแบบ error 401 ออกชั่วคราว เทสต์ `api_withoutLogin_returns401WithStandardErrorBody` ล้ม และผ่านเมื่อใส่กลับ

ส่วนของภีมเดช กลั่นกิ่ง

โมดูล	จำนวน	ผ่าน	ล้มเหลว
Payment / Notification / API Quality / หน้าเว็บ (ของโมดูลนี้)	118	118	0
รอบเต็มทั้งโปรเจค (รันบน `develop`)	380	343	0 (ข้าม 37 เทสต์ที่ต้องต่อฐานข้อมูล; เมื่อเปิดเทสต์เหล่านั้นรันได้ 390 ข้อ ผ่านทั้งหมด)

รายละเอียดรายคลาสดู `doc/test-report/test-report.md` หัวข้อ 2 และ 5

4.4 ผลทดสอบแอปจริงของโมดูลบัญชีผู้ใช้และสาขา

ทดสอบด้วย `curl` กับแอปที่รันบน PostgreSQL ใน Docker และใช้บัญชีตัวอย่างจากไฟล์ seed

กรณีทดสอบ	ผลที่ได้	ผลที่คาด
สมัครด้วยข้อมูลถูกต้อง	201 ได้บัญชีลูกค้าพร้อมโปรไฟล์ในฐานข้อมูล	ตรง
ชื่อผู้ใช้ซ้ำ / อีเมลซ้ำ (รวมกรณีตัวพิมพ์ต่าง)	409	ตรง
ข้อมูลสมัครไม่ถูกต้อง	400 พร้อมรายการ `fieldErrors` 4 ช่อง	ตรง
ส่ง `"role":"ADMIN"` มาตอนสมัคร	ได้บัญชี CUSTOMER (ไม่รับค่า role)	ตรง
รหัสผ่านถูกเก็บเป็นแฮช	ในฐานข้อมูลขึ้นต้นด้วย `$2a$10$`	ตรง
เจ้าของอ่านและแก้โปรไฟล์ตนเอง	200 และอ่านกลับได้ค่าใหม่	ตรง
ลูกค้าอ่านข้อมูลของผู้ใช้คนอื่น	403	ตรง
แอดมินอ่านโปรไฟล์ลูกค้า	200	ตรง
ขอโปรไฟล์ของพนักงาน / ผู้ใช้ที่ไม่มีอยู่	404	ตรง
ไม่ล็อกอินเรียก API	401 พร้อม JSON มาตรฐาน	ตรง
ใช้รหัสผ่านผิดเรียก API	401 พร้อม JSON มาตรฐาน ไม่มี header ที่ทำให้เบราว์เซอร์เด้งหน้าต่างล็อกอิน	ตรง (หลังแก้ ดูตารางข้อบกพร่อง)

ลูกค้าเพิ่มสาขา	403 พร้อม JSON มาตรฐาน	ตรง
แอดมินเพิ่ม / แก้ / ลบสาขา	201 / 200 / 204	ตรง
ลบสาขาที่มีเครื่องอ้างอิง	409 และข้อมูลไม่ถูกลบ	ตรง
เปิดหน้า `/admin/branches` ด้วยบัญชีลูกค้า	403	ตรง
ทดสอบหน้าเว็บ: กรอกฟอร์มสมัครผิด / ถูก / ซ้ำ	แสดงข้อความใต้ช่อง / พาไปหน้า login / แจ้งชื่อซ้ำ	ตรง
ล็อกอินผิด / ถูก / ออกจากระบบ	กลับ `/login?error` / ไปหน้าแรก / กลับ `/login?logout` และเข้าหน้าแรกไม่ได้อีก	ตรง

4.5 ผล Deploy และ CI

ตรวจระบบที่ deploy บน Render ผ่านอินเทอร์เน็ต:

รายการ	**ผล**
`/login`, `/register`, `/swagger-ui.html`, `/v3/api-docs`	200 ทั้งหมด
เรียก API โดยไม่ล็อกอิน	401
เรียก API ด้วยบัญชีแอดมิน	ได้ข้อมูลสาขาจากฐานข้อมูลบน Neon (Flyway สร้างตารางและข้อมูลตัวอย่างให้เองตอนสตาร์ทครั้งแรก)
เวลาสตาร์ทครั้งแรกบน Render แผนฟรี	ประมาณ 137 วินาที (เทียบกับ 4–5 วินาทีบนเครื่องพัฒนา เนื่องจาก CPU 0.1 คอร์)
ขนาด Docker image	ประมาณ 558 MB

ผลของ CI ที่รันบน GitHub: งาน `Test (mvn test)` ใช้เวลา 39 วินาที (รอบแรก) และ 23 วินาที (รอบหลัง ซึ่งใช้ cache) งาน `Docker build` ใช้เวลาประมาณ 1 นาที 28 วินาที และผ่านทั้งคู่ จากนั้นตั้งทั้งสองงานเป็น required check ทำให้ Pull Request ที่เทสต์หรือการ build ไม่ผ่านจะ merge ไม่ได้

4.6 ผลการทำงานร่วมกันด้วย Git

ผู้เขียนมี commit ของตนเอง 57 รายการ (ไม่นับ merge commit นับถึง commit `399b351` ของวันที่ 10 ตุลาคม 2569) กระจายตลอดช่วงพัฒนา และเปิด Pull Request เข้า `develop` และ merge แล้ว 9 รายการ (นับก่อน Pull Request ที่แก้ตัวเลขในรายงานนี้) ได้แก่ โครงโปรเจค,

ระบบยืนยันตัวตนกับสาขากับหน้าเว็บ, Dockerfile, CI กับการแก้ error 401/403, แผนภาพ, เอกสารรวม SOLID/Pattern, ส่วนรายงานพร้อมภาพหน้าจอ, ปรับหน้าจอ (สถานะภาษาไทย ยืนยันรหัสผ่าน ซ่อนปุ่มชำระเงินเมื่อมีรายการชำระแล้ว), และแก้ตามรีวิว (ป้ายสถานะ เทสต์เพิ่ม เลขบรรทัดอ้างอิงในเอกสาร) ทุกรายการผ่านการรีวิวและอนุมัติก่อน merge รวมถึงทำหน้าที่รีวิว Pull Request ของเพื่อนสมาชิก

4.7 ข้อบกพร่องที่พบและแก้ไข

ส่วนของวงศธร ธน.ยอด

#	พบจาก	ปัญหา	การแก้ไข
1	ทดสอบเริ่มระบบ	PostgreSQL ที่ติดตั้งในเครื่องใช้พอร์ต 5432 ชนกับฐานข้อมูลใน Docker ทำให้แอปเชื่อมต่อผิดตัวและยืนยันตัวตนฐานข้อมูลไม่ผ่าน	ยืนยันสาเหตุด้วย container แยกที่พอร์ตอื่น แล้วทีมเปลี่ยนพอร์ตของฐานข้อมูลใน Docker เป็น 5433
2	ตั้งค่า Deploy	สตริงเชื่อมต่อ Neon ที่ได้มามี `-pooler` ในชื่อโฮสต์และพารามิเตอร์ `channel_binding` ซึ่ง JDBC ไม่รู้จัก	ใช้ที่อยู่แบบเชื่อมต่อตรงและตัดพารามิเตอร์ที่ไม่ใช้ออก
3	ทดสอบกับระบบจริง	รหัสผ่านผิดในการเรียก API ได้ 401 แบบไม่มี JSON เพราะตัวกรอง Basic ตอบเองโดยไม่ผ่านตัวจัดการ error	ตั้ง entry point ของ HTTP Basic ให้ตอบ `ApiErrorResponse` และเพิ่มเทสต์
4	รีวิวโค้ด (เพื่อนสมาชิก)	ตรวจชื่อผู้ใช้ซ้ำแบบสนตัวพิมพ์ ทำให้ `Admin` กับ `admin` สมัครได้ทั้งคู่	เปลี่ยนเป็นการตรวจแบบไม่สนตัวพิมพ์ทั้งชื่อผู้ใช้และอีเมล
5	รีวิวโค้ด (เพื่อนสมาชิก)	กฎสิทธิ์รายสาขาอยู่ในไฟล์ `SecurityConfig` กลาง เสี่ยงที่หลายคนต้องแก้ไฟล์เดียวกัน	ย้ายไปประกาศด้วย `@PreAuthorize` ที่ Controller ของแต่ละโมดูล
6	รีวิวโค้ด (เพื่อนสมาชิก)	เมนูมีลิงก์ไปหน้าโปรไฟล์และจัดการสาขาที่ยังไม่มี	พัฒนาสองหน้านี้ก่อน merge
7	ตรวจภาพหน้าจอ	ลิงก์ในแถบเมนูมีสีชีดจนอ่านยากบนพื้นน้ำเงิน	ปรับสีตัวอักษรของลิงก์ด้วย CSS
8	ผลของ CI	คำเตือนว่าเวอร์ชันของ action ที่ใช้อิง Node.js 20 ซึ่งเลิกสนับสนุน	ปรับเป็นเวอร์ชันที่รองรับ
9	ตรวจเอกสาร	เลขบรรทัดอ้างอิงในเอกสารเพี้ยนเมื่อมีการแก้โค้ดของโมดูลอื่นระหว่างทาง	เขียนสคริปต์ตรวจเลขบรรทัดและลิงก์ทั้งหมดกับโค้ดจริง และรันซ้ำก่อนส่ง

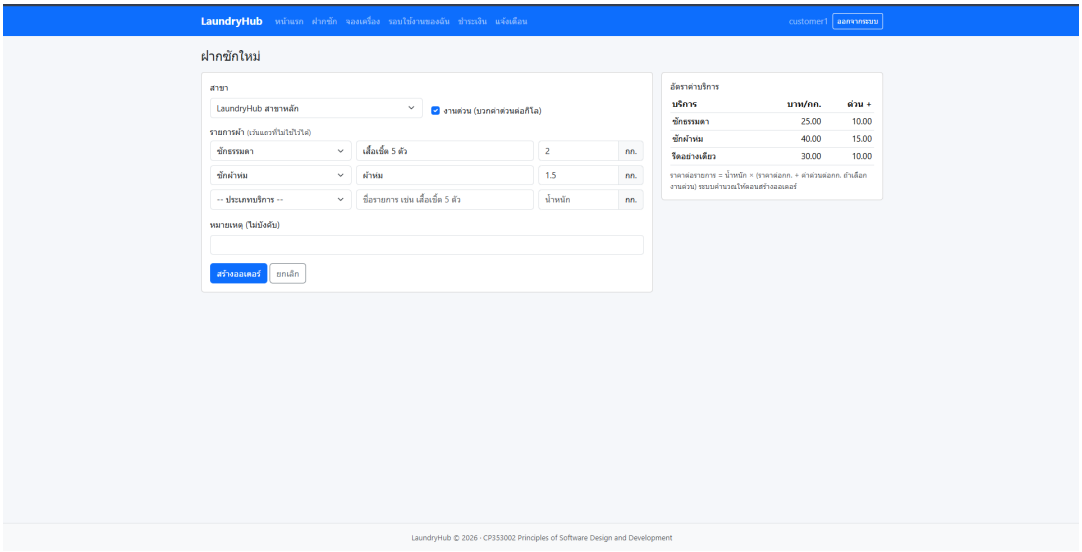
ส่วนของภีมเดช กลั่นกิ่ง

#	พบจาก	ปัญหา	การแก้ไข
1	เทสต์ล้ม 8 ข้อ	`@RestControllerAdvice` จำกัดขอบเขตผิด handler ไม่ทำงาน	ปรับตัวกรองขอบเขต
2	รีวิวโค้ด	handler ของ `Exception` กลืน error ของ Spring MVC ทำให้ 405/400/415 เป็น 500	เพิ่ม handler เฉพาะ + เทสต์
3	เทสต์เพิ่ม	405 เกิดก่อนเลือก controller ตัวกรองของ advice จึงไม่ทำงาน	เอาตัวกรองออกให้ครอบทุก controller
4	รีวิวโค้ด	`markPaid()` เรียกซ้ำเขียนทับเวลารับเงิน	ทำให้ idempotent
5	รีวิวโค้ด	`notifyUser` ไม่ตรวจ `userId`/ข้อความ null	ตรวจและโยน `BusinessRuleException`
6	ทดสอบแอปจริง	`?sort=abc` ได้ 500	whitelist ฟิลด์ที่เรียงได้ ⟶ 400
7	ทดสอบแอปจริง	Swagger แสดงพารามิเตอร์แบ่งหน้าเป็นก้อนเดียว	เพิ่ม `@ParameterObject`
8	ทดสอบหน้าเว็บจริง	URL ที่ไม่มีอยู่ (เช่น `/api/v1/nothing-here`) ได้ 500	เพิ่ม handler `NoResourceFoundException` ⟶ 404 + เทสต์
9	ทดสอบหน้าเว็บจริง	dropdown ว่างเปล่า (`map[t]` ใน SpEL ตีความเป็นข้อความ)	เปลี่ยนเป็น `map.get(t)`

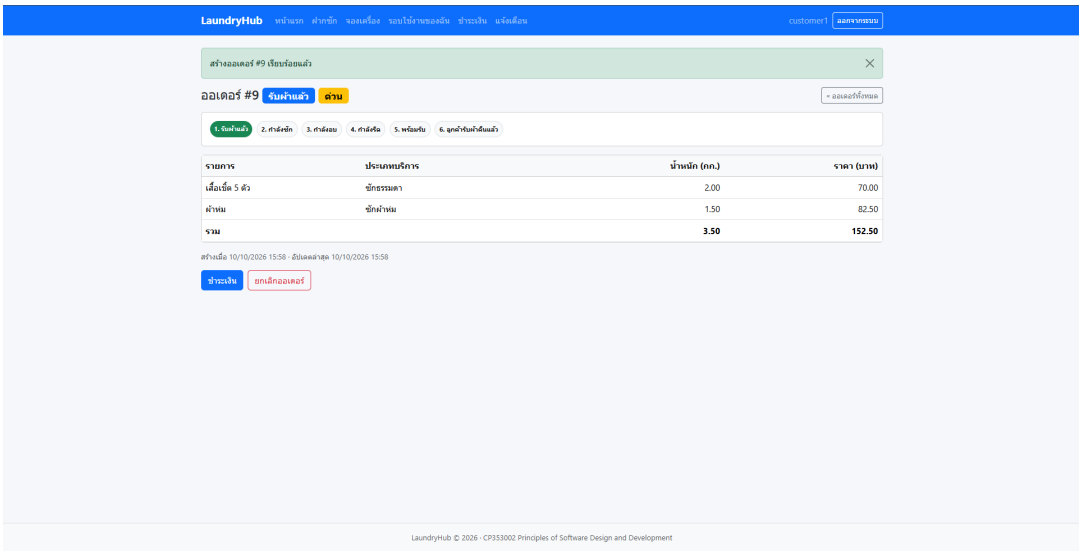
4.8 ผลการพัฒนาโมดูลออเดอร์

- REST API ของออเดอร์ 7 endpoint และประเภทบริการ 5 endpoint ใช้งานได้ผ่าน Swagger UI
- หน้าเว็บสำหรับลูกค้า: รายการออเดอร์ของฉัน (แบ่งหน้า กรองสถานะ), ฟอร์มฝากซักใหม่พร้อมตารางราคา, หน้ารายละเอียดพร้อมแถบขั้นตอน ปุ่มชำระเงิน และปุ่มยกเลิก (แสดงเฉพาะตอน `RECEIVED`)
- หน้าเว็บสำหรับพนักงาน: บอร์ดออเดอร์ เรียงจากออเดอร์ที่รอนานที่สุด มีปุ่มเลื่อนไปขั้นถัดไปและปุ่มยกเลิก
- แก้ปัญหา N+1 ของหน้ารายการด้วย `@BatchSize(size = 50)` ที่ …/domain/entity/LaundryOrder.java:60 และ ระ

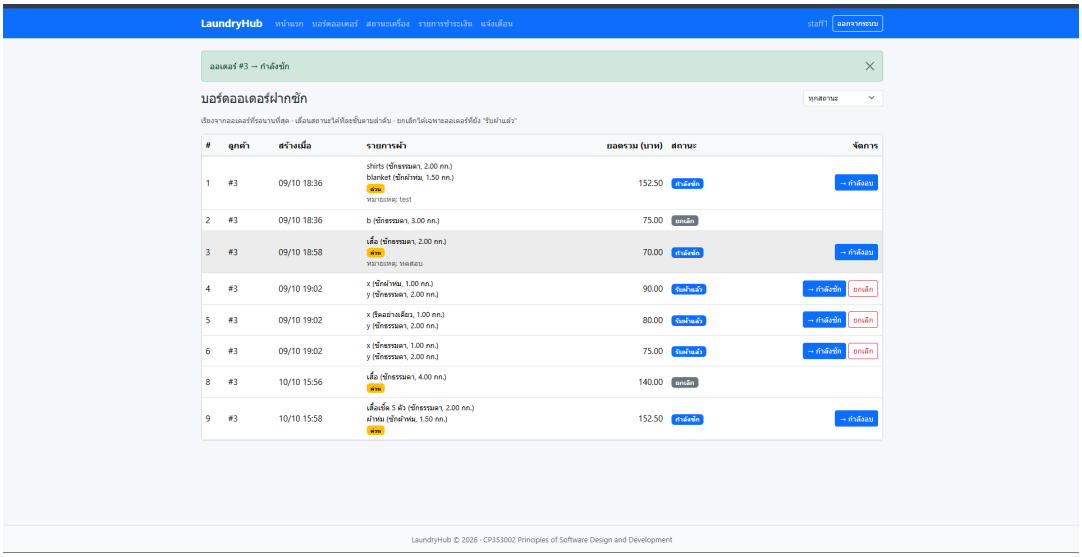
…/domain/entity/ServiceType.java:12 จากการวัดด้วย log org.hibernate.SQL ขณะดึงออเดอร์ 6 ใบ เดิมใช้คำสั่ง SQL ของออเดอร์ 10 คำสั่ง (ออเดอร์ 1 + รายการ 6 + ประเภทบริการ 3) หลังแก้เหลือ 3 คำสั่ง



ภาพที่ 19 หน้าฟอร์มฝากซักใหม่



ภาพที่ 20 หน้ารายละเอียดออเดอร์ แสดงแถบขั้นตอนและราคาแต่ละรายการ



ภาพที่ 21 บอร์ดออเดอร์ของพนักงาน

4.9 ผลการทดสอบอัตโนมัติ

เทสต์ของโมดูลออเดอร์มี 8 คลาส รวม 78 เทสต์ ผ่านทั้งหมด (ตรงกับ doc/test-report/test-report.md รอบ 10 ต.ค. 2569)

คลาสทดสอบ	จำนวน	สิ่งที่ทดสอบ
`FullServicePricingTest`	6	ราคาปกติ, ด่วนคิดต่อกิโล (2 กก. = 70.00), ปัดทศนิยม, น้ำหนัก 0, ราคาว่าง, input ว่าง
`OrderStateTest`	15	ทุกสถานะไปขั้นถัดไปถูก, สถานะสุดท้ายไม่มีขั้นต่อไปและไม่ throw, ยกเลิกได้เฉพาะ `RECEIVED`, Factory ครบทุก enum
`OrderServiceTest` (Mockito)	14	คิดยอดฝั่งเซิร์ฟเวอร์ (122.50), ไม่พบ/ปิดใช้งานประเภทบริการ, ดูออเดอร์คนอื่นได้ 404, เลื่อนสถานะยิง event 1 ครั้ง, ยกเลิกหลังเริ่มซักไม่ได้, แก้ไข, แบ่งหน้า, ฟิลด์ sort ที่ไม่อนุญาต
`OrderPayableProviderTest`	3	ส่งออเดอร์เป็น `Payable` ครบ 4 ค่า และไม่พบได้ 404
`OrderSecurityTest` (`@WebMvcTest`)	17	401 ไม่ล็อกอิน, เจ้าของ/คนอื่น/พนักงานในทุก endpoint, 403 และ 404

		แยกกรณี, validation 400
ServiceTypeServiceTest	7	ชื่อซ้ำแบบไม่สนตัวพิมพ์ได้ 409, แก้ไข, soft delete
ServiceTypeSecurityTest	7	อ่านได้ทุกคนที่ล็อกอิน, เขียนได้เฉพาะ ADMIN, ราคาติดลบได้ 400
OrderWebControllerTest	9	render หน้าเว็บจริง, ข้อความผิดพลาดภาษาไทย, ข้ามแถวว่าง, ปุ่มยกเลิกเฉพาะ RECEIVED, ลูกค้าเปิดบอร์ดได้ 403, ปุ่มขั้นถัดไปมาจาก State

เพื่อยืนยันว่าเทสต์สิทธิ์ตรวจจับข้อผิดพลาดได้จริง ได้ลองเปลี่ยนเงื่อนไขเจ้าของใน OrderApiController ให้ผ่านเสมอชั่วคราว เทสต์ล้ม 3 ข้อ (สร้างแทนคนอื่น, ดูรายการคนอื่น, พนักงานใช้ DELETE) แล้วเปลี่ยนกลับ

4.10 ผลการทดสอบกับระบบจริง

ทดสอบกับแอปและ PostgreSQL ใน Docker ผ่าน REST API และหน้าเว็บ

กรณี	ผลที่ได้
ลูกค้าสร้างออเดอร์ด่วน 2 รายการ (2 กก. ซักธรรมดา + 1.5 กก. ซักผ้าห่ม)	201 ยอดรวม 152.50 บาท
ส่งออเดอร์ที่ไม่มีรายการ	400 พร้อม fieldErrors ที่ items
ลูกค้าสร้างหรือดูออเดอร์ของลูกค้าคนอื่น	403
ลูกค้าเรียกเลื่อนสถานะหรือเปิดบอร์ดพนักงาน	403
พนักงานเลื่อนสถานะ RECEIVED → WASHING	200 และมีแจ้งเตือน "ออเดอร์ #N สถานะ: WASHING" ในตาราง notifications
ยกเลิกหลังเริ่มซัก	400 "cannot be cancelled once it is WASHING" สถานะไม่เปลี่ยน
แก้ไขออเดอร์ตอน RECEIVED	ยอดคำนวณใหม่ รายการเดิมถูกลบด้วย orphanRemoval
ชำระเงินออเดอร์ผ่าน CheckoutFacade ด้วย QR จำลอง	Payment สถานะ PAID ยอดตรงกับออเดอร์ และมีแจ้งเตือนการชำระ
ประเภทบริการ: ชื่อซ้ำ (ต่างตัวพิมพ์) / ลูกค้าสร้าง / ปิดใช้งานแล้วสั่งออเดอร์	409 / 403 / 400 "Service type N is not available"

4.11 ผลการทดสอบโมดูล Self-Service

ผลทดสอบล่าสุดก่อน merge PR #34 หลังรวม develop f9c11a4: Maven รัน 369 รายการ ผ่าน 333 ข้าม 36 ไม่มี failure/error และ BUILD SUCCESS โดยโหลด Byte Buddy agent ล่วงหน้าเพื่อหลีกเลี่ยงข้อจำกัดการ attach บน Windows รอบนี้ไม่ได้เปิด LAUNDRY_DB_TESTS จึงไม่ใช่หลักฐานว่าชุด PostgreSQL ล่าสุดผ่านทั้งหมด ตัวเลขตรงกับผลรวมใน doc/test-report/test-report.md ส่วน CI ของ PR #34 ผ่านทั้ง Test และ Docker build

หลักฐานทดสอบ PostgreSQL รอบก่อน วันที่ 10 ตุลาคม 2569 หลังรวม develop f7c82e1 (PR #26) พร้อมลิงก์ชำระเงินจากประวัติและเทสต์ใหม่ มี 357 รายการ Failures 0 Errors 0 Skipped 0 และ BUILD SUCCESS โดยเปิด LAUNDRY_DB_TESTS=true ตัวเลขเป็นจำนวนทั้งโปรเจกต์ ไม่ใช่จำนวนเทสต์ที่ปฏิภาณ มะนิลทิพย์เขียนทั้งหมด Log อยู่ใน code/target/pond-pr26-checkout-tests.log ซึ่งเป็น build output ไม่ได้ commit เข้ารายงาน

ประเภท	หลักฐานที่ตรวจ
Pricing	สูตรราคา ค่าขอบเขต 10/180 นาที และปฏิเสธค่าที่ผิด
BookingValidator/Repository	เวลาซ้อน ติดขอบ เวลาอดีต เครื่องเสีย และ association กับฐานข้อมูล
State/Service	ความสามารถแต่ละ state เจ้าของ และการเปลี่ยนทั้ง Session/Machine
MVC Security	ADMIN CRUD, STAFF status, owner history, validation และตัวตนที่ client ปลอมไม่ได้
PostgreSQL concurrency	ตรวจว่าคำขอทั้งสองรอ lock จริง แล้วเกิดการจอง/เริ่มสำเร็จเพียงหนึ่งรายการ
Transaction rollback	บังคับ notification insert ล้มเหลวแล้วสถานะ Session/Machine ไม่เปลี่ยน
HTTP integration	จองราคา 65 บาท ⟶ overlap 409 ⟶ non-owner 403 ⟶ COIN PAID ⟶ start ⟶ finish ⟶ AVAILABLE

เทสต์ฐานข้อมูลสร้าง schema ชื่อสุ่มแยกจาก public รัน Flyway ใน schema นั้น และลบเฉพาะ schema ของเทสต์เมื่อจบ ไม่แก้ข้อมูลใช้งานจริง เทสต์ concurrency ใช้ transaction แยกใน worker thread เพื่อให้ตรวจการแข่งขันจริง ส่วนเทสต์ HTTP เชื่อม Security, Controller, Service, Repository, Payment และ Notification จริงโดยไม่ mock service

4.12 ผลการรีวิวและข้อจำกัดของหลักฐาน

PR #22 มี review และ approval จากภีมเดช กลั่นกิ่งก่อน merge เข้า main แต่ใช้ base ผิดจาก workflow ของทีม จากนั้น PR #23 ส่งงานเข้า develop ที่ a21cb2a และ PR #25 ส่งหน้าเว็บ/diagrams เข้า develop ที่ 11fe6af โดยภีมเดช กลั่นกิ่ง approve และ CI Test + Docker build ผ่าน งานลิงก์ชำระเงินเพิ่มเติม merge ผ่าน PR #28 ที่ 5070373 โดยภีมเดช กลั่นกิ่ง approve และ CI Test + Docker build ผ่าน การผ่านเทสต์ในเครื่องที่เปิดฐานข้อมูลไม่เท่ากับ CI ทดสอบ PostgreSQL เพราะ workflow ไม่เปิด opt-in database suite รายละเอียดอยู่ใน doc/self-service-api.md และ doc/test-report/session-lifecycle-pond.md

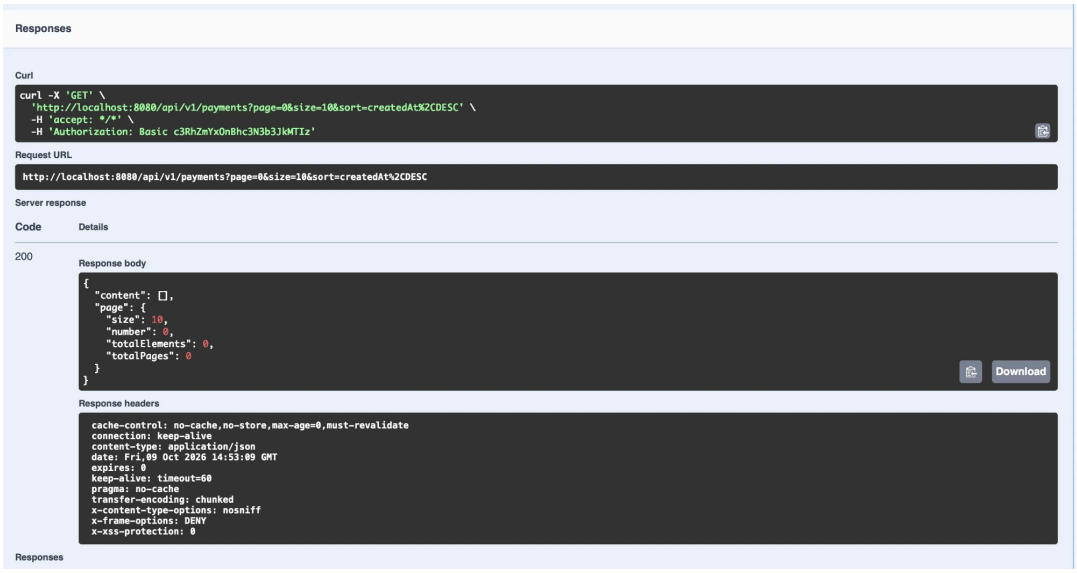
PR #34 แก้ตัวคั่นข้อความ Sequence และลำดับ note ของสถานะ RESERVED เพื่อให้ Mermaid แสดงผลได้ โดยคง flow เดิม ผ่าน review จากภีมเดช กลั่นกิ่งและ CI Test + Docker build แล้ว merge เข้า develop ที่ 9a11daa

4.13 ผลทดสอบกับฐานข้อมูลจริง

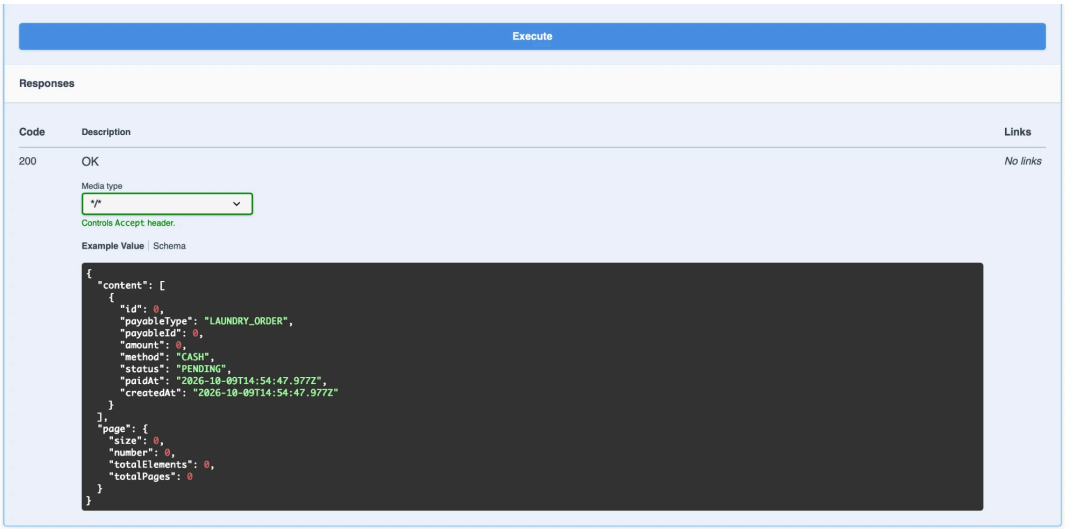
PaymentRepositoryDbTest 13 ข้อผ่านทั้งหมด ยืนยันว่า constraint ทำงานจริง: จ่ายซ้ำออเดอร์/รอบใช้เครื่อง → ถูกปฏิเสธด้วย payments_order_id_key / payments_session_id_key, ผูกทั้ง order และ session หรือไม่ผูกเลย → chk_payment_target, ออเดอร์ที่ไม่มี → fk_payment_order, ยอดติดลบ → payments_amount_check, วิธีชำระแปลก → chk_payment_method, แจ้งเตือนให้ผู้ใช้ที่ไม่มี → fk_notification_user

4.14 ผลทดสอบแอปจริง (22 กรณี)

สรุปจาก test-report.md หัวข้อ 3.4: ไม่ล็อกอิน → 401, ลูกค้าเรียกรายการทั้งหมด → 403, พนักงาน → 200, status ผิด → 400, sort ผิด → 400, ดูแจ้งเตือนของตัวเอง/ของคนอื่น → 200/403, body ว่าง → 400 พร้อม fieldErrors, ยืนยันของที่ไม่มี → 404 ฯลฯ และทดสอบวิธีชำระด้วยเหรียญกับรอบใช้เครื่อง (201 PAID) เงินสดที่ต้องรอพนักงานยืนยัน (PENDING → PAID) การจ่ายซ้ำ (409) และแจ้งเตือนที่ถูกสร้างหลังชำระ ทุกกรณีตอบเป็น ApiErrorResponse เดียวกัน



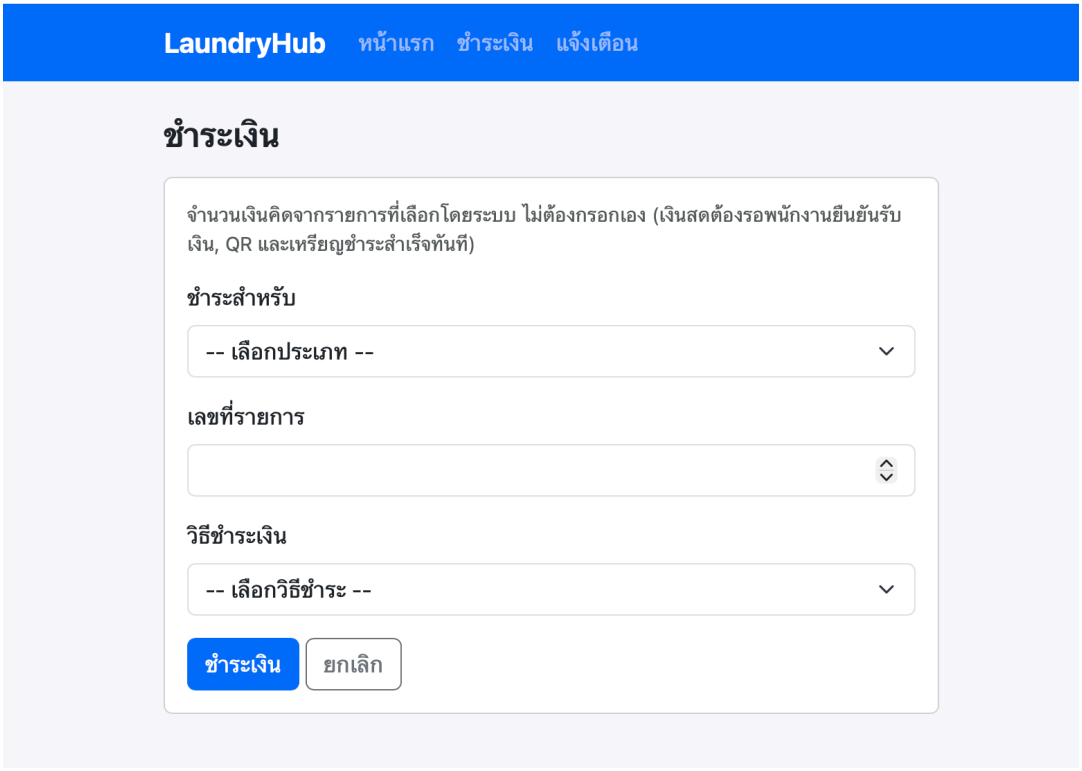
ภาพที่ 22 Swagger UI: GET /api/v1/payments ผลลัพธ์ 200 พร้อมพารามิเตอร์แบ่งหน้า page, size, sort



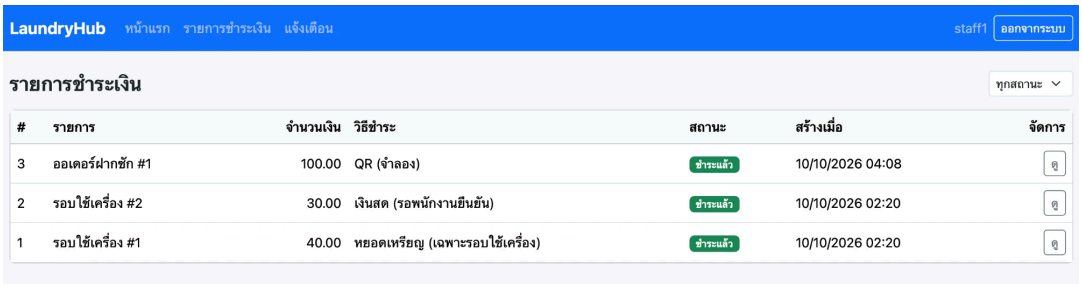
ภาพที่ 23 Swagger UI: โครงสร้างตัวอย่างของ PaymentResponse

4.15 ผลหน้าเว็บชำระเงินและแจ้งเตือน

หน้าเว็บเรียก `CheckoutFacade` ตัวเดียวกับ REST API จึงใช้กฎสิทธิ์เดียวกัน (เจ้าของหรือพนักงาน) ประกอบด้วยฟอร์มชำระเงิน, หน้ารายละเอียดการชำระ, รายการชำระเงินของพนักงานพร้อมปุ่มยืนยันรับเงินสด และหน้าแจ้งเตือนของผู้ใช้ (กรองยังไม่อ่าน, ทำเครื่องหมายอ่านแล้ว) ทดสอบด้วยเทสต์ระดับ controller 20 ข้อ และเรียกหน้าจริงด้วยบัญชีตัวอย่าง ผลคือสิทธิ์ถูกต้อง (ลูกค้าเปิดหน้าพนักงาน ⟶ 403), ข้อความผิดพลาดแสดงบนฟอร์ม และการชำระผ่านหน้าเว็บทำให้เกิดแจ้งเตือนให้ผู้ใช้จริง (รายละเอียดดู `test-report.md` หัวข้อ 3.7)



ภาพที่ 24 ฟอร์มชำระเงิน (ลูกค้า)



ภาพที่ 25 รายการชำระเงิน (พนักงาน)



ภาพที่ 26 แจ้งเตือนของฉัน (ลูกค้า)

บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

ส่วนของวงศธร ธน.ยอด

โครงงาน LaundryHub พัฒนาระบบจัดการร้านซักรีดที่รองรับการฝากซักและการซักเอง พร้อมผู้ใช้ 3 บทบาท ตามสถาปัตยกรรมแบบชั้นที่ไม่มี Controller เรียก Repository โดยตรง ใช้ SOLID และ Design Pattern ที่ตรวจตำแหน่งในโค้ดได้ มีฐานข้อมูล 10 ตารางที่จัดการด้วย Flyway ระบบ deploy แล้วที่ https://laundry-hub-1ltc.onrender.com และมี CI ที่ตรวจเทสต์ทุก Pull Request ส่วนของผู้เขียนซึ่งเป็นโครงสร้างพื้นฐาน ผ่านการทดสอบครบ ทั้งเทสต์อัตโนมัติ 29 ข้อ และการทดสอบกับระบบจริง

ส่วนของภีมเดช กลั่นกิ่ง

โมดูลชำระเงิน แจ้งเตือน และคุณภาพ API พัฒนาครบตามขอบเขต P0 ของโปรเจค ผ่านการทดสอบทั้งระดับหน่วย ระดับฐานข้อมูล และการใช้งานจริงบนแอป

5.2 อภิปรายผล

ส่วนของวงศธร ธน.ยอด

- **ยืนยันตัวตนสองช่องทาง**: ฟอร์มกับเซสชันเหมาะกับหน้าเว็บ ส่วน HTTP Basic ทำให้ทดสอบ API ผ่าน Swagger ได้ง่ายโดยไม่ต้องมี endpoint เข้าสู่ระบบแยก ข้อแลกเปลี่ยนคือ Basic ส่งรหัสผ่านทุกคำขอ จึงควรใช้ผ่าน HTTPS เสมอ ซึ่งเว็บที่ deploy ใช้อยู่
- **กระจายกฎสิทธิ์ไปที่ Controller**: ลดการแก้ไฟล์กลางพร้อมกันหลายคนและทำให้กฎอยู่ใกล้ endpoint ที่ใช้ ข้อแลกเปลี่ยนคือกฎกระจายอยู่หลายไฟล์ ต้องมีเทสต์สิทธิ์ครอบคลุมจึงจะมั่นใจว่าไม่มี endpoint ใดตกหล่น
- **แยกฐานข้อมูลออกจากแอป**: ข้อมูลอยู่รอดเมื่อ deploy ใหม่ แต่ต้องจัดการการเชื่อมต่อแบบเข้ารหัสและตัวแปรสภาพแวดล้อมเพิ่ม
- **บังคับผ่าน CI ก่อน merge**: ป้องกันโค้ดที่เทสต์พังไปถึงการ deploy ได้ โดยเฉพาะเมื่อ Docker build ข้ามเทสต์ ข้อแลกเปลี่ยนคือถ้า CI ขัดข้องจากฝั่งผู้ให้บริการ ต้องปรับกฎชั่วคราวจึงจะ merge ได้

- **ตรวจเอกสารด้วยเครื่องมือ**: การตรวจเลขบรรทัดด้วยสคริปต์พบว่าเลขเพี้ยนได้ง่ายเมื่อมีผู้แก้โค้ดต่อ จึงควรตรวจซ้ำทุกครั้งก่อนส่ง

ส่วนของภีมเดช กลั่นกิ่ง

- **Observer แบบ synchronous**: เลือกเพราะต้องการความสอดคล้องของข้อมูล ข้อแลกเปลี่ยนคือแจ้งเตือนที่ล้มเหลวกระทบการชำระเงิน หากต้องแยกขาด ต้องใช้ `@TransactionalEventListener(AFTER_COMMIT)` ร่วมกับธุรกรรมใหม่
- **Factory แบบ registry**: เพิ่ม `CoinProcessor` ได้เป็นคลาสเดียวโดยไม่แก้ Factory/Service (Open/Closed) พิสูจน์ด้วยเทสต์ แต่ไม่ใช่ GoF Factory Method แท้
- **ข้อจำกัดของเทสต์ standalone**: `@PreAuthorize` ไม่ทำงานใน MockMvc แบบ standalone จึงมีเทสต์แยก `PaymentNotificationSecurityTest` (13 ข้อ) ที่เปิด Spring Security และ Thymeleaf จริง (service เป็น mock) ครอบสิทธิ์ของ REST และหน้าเว็บ รวมทั้งตรวจว่า dropdown ทุกอันมีข้อความ

5.3 ผลลัพธ์การเรียนรู้

ส่วนของวงศธร ธน.ยอด

ผู้เขียนได้เรียนรู้ว่ากฎสิทธิ์ที่ถูกต้องต้องตรวจทั้งในโค้ดและด้วยเทสต์อัตโนมัติ เพราะ error บางชนิดเกิดนอกตัวจัดการ error ของ Controller (เช่น 401 จากตัวกรอง) และมองไม่เห็นจากเทสต์ชั้น Service การทดสอบกับระบบจริงจึงพบปัญหาที่เทสต์จำลองไม่พบ นอกจากนี้การทำ Code Review ระหว่างเพื่อนสมาชิกช่วยจับข้อบกพร่องที่ผู้เขียนมองไม่เห็นเอง และการทำ CI ตั้งแต่ต้นช่วยให้มั่นใจเมื่อรวมงานของหลายคน

ส่วนของภีมเดช กลั่นกิ่ง

การทดสอบและการรีวิวโค้ดโดยเพื่อนช่วยจับบั๊กที่มองไม่เห็นเอง (เช่น พฤติกรรมของ `@RestControllerAdvice` กับ error ที่เกิดก่อนเลือก controller) และการพิสูจน์กฎกับฐานข้อมูลจริงให้ความมั่นใจมากกว่าการทดสอบด้วย mock เพียงอย่างเดียว

5.4 ปัญหาที่พบในการดำเนินงาน

ส่วนของวงศธร ธน.ยอด

ดูตารางข้อบกพร่อง 9 ข้อในบทที่ 4 นอกจากนี้ มีปัญหาด้านการประสานงานคือโมดูลของสมาชิกแต่ละคนต้องรอโครงสร้างพื้นฐานและ Shared

Contracts จากส่วนของผู้เขียนก่อนจึงเริ่มได้ ระยะเวลาโครงงานสั้นเพียงสามวัน จึงต้องเรียงลำดับงานให้ส่งมอบโครงโปรเจคและ deploy ก่อน และการ deploy บนแผนฟรีมีข้อจำกัดเรื่องหน่วยความจำและ CPU ทำให้สตาร์ทช้าตามที่ระบุในบทที่ 4

ส่วนของคมซาญ น้อยเนียม

- **เวอร์ชัน JDK ไม่ตรงกับทีม**: เครื่องที่ใช้มี Java 26 ซึ่ง Lombok เวอร์ชันในโปรเจกต์ยังไม่รองรับ ทำให้ getter/setter ไม่ถูกสร้างและ compile ไม่ผ่าน แก้โดยติดตั้ง JDK 17 และตั้ง JAVA_HOME ให้ตรงกับที่ทีมกำหนด
- **พอร์ตฐานข้อมูลชนกัน**: PostgreSQL ที่ติดตั้งในเครื่องใช้พอร์ต 5432 เดียวกับ Docker แอปจึงต่อผิดฐานข้อมูล ทีมแก้โดยเปลี่ยนพอร์ตของ Docker เป็น 5433
- **ปัญหา N+1 query**: พบจากการเปิด log SQL ของหน้ารายการแบบแบ่งหน้า แก้ด้วย @BatchSize เพราะการใช้ @EntityGraph กับ collection พร้อม pagination ทำให้ Hibernate ตัดหน้าในหน่วยความจำ
- **ข้อมูลผิดรูปแบบได้ 500**: ผู้รีวิวพบว่าราคาที่เป็น null ทำให้เกิด NullPointerException จึงย้ายการตรวจไปไว้ใน compact constructor ของ FullServicePricingInput ให้ได้ 400
- **รหัสสถานะไม่ตรงกันในเอกสาร**: brief ระบุว่าเปลี่ยนสถานะผิดลำดับได้ 409 แต่ shared contract ระบุ BusinessRuleException → 400 จึงยึดตาม contract ที่ทั้งทีมใช้

ส่วนของภีมเดช กลั่นกิ่ง

ดูตารางข้อบกพร่อง 9 ข้อในบทที่ 4 นอกจากนี้มีเรื่องการประสานงานข้ามโมดูล เช่น การเปลี่ยนพอร์ตฐานข้อมูลในโมดูลหนึ่งกระทบทุกคน และการเชื่อมโมดูลชำระเงินกับรอบใช้เครื่องผ่าน SessionPayableProvider ของโมดูลเครื่องซัก ซึ่งเสร็จช่วงท้ายโครงงานและทดสอบแล้ว

5.5 ข้อจำกัด

ส่วนของวงศธร ธน.ยอด

- บัญชีตัวอย่างใน V2__seed_data.sql (admin, staff1, customer1) ใช้รหัสผ่านตัวอย่างเดียวกัน และอยู่ใน repository ที่เป็นสาธารณะ จึงใช้เพื่อการทดสอบและนำเสนอเท่านั้น ไม่เหมาะกับการใช้งานจริง

- ระบบไม่มีการล็อกบัญชีเมื่อกรอกรหัสผิดหลายครั้ง ไม่มีการจำกัดอัตราการเรียก ไม่มีการลืมรหัสผ่านหรือยืนยันอีเมล
- ไม่มีหน้าจัดการบัญชีผู้ใช้สำหรับผู้ดูแลระบบ (ผู้ดูแลระบบดูข้อมูลผู้ใช้รายคนได้ผ่าน API) และไม่มีรายงานยอดขาย
- ปิดการตรวจ CSRF เฉพาะ `/api/**` เพราะ API ออกแบบให้ยืนยันตัวตนด้วย Basic ทุกคำขอ แต่ถ้าเบราว์เซอร์มีเซสชันจากการล็อกอินผ่านฟอร์มอยู่ เซสชันนั้นก็ใช้เรียก API ได้เช่นกัน ซึ่งเบราว์เซอร์รุ่นใหม่ลดความเสี่ยงด้วยการไม่ส่ง `cookie` ข้ามเว็บสำหรับคำขอแบบ POST โดยปริยาย
- เอกสาร API (`/v3/api-docs`, `/swagger-ui.html`) เปิดเป็นสาธารณะเพื่อให้ตรวจงานได้ ควรปิดเมื่อใช้งานจริง
- บริการแผนฟรีบน Render จะหยุดทำงานเมื่อไม่มีผู้ใช้ การเปิดครั้งแรกหลังหยุดใช้เวลา 2 นาทีขึ้นไป
- เทสต์ของบางโมดูลที่ต้องใช้ฐานข้อมูลจริงเป็นแบบเลือกรัน จึงไม่ถูกรันใน CI ปัจจุบัน

ส่วนของคมชาญ น้อยเนียม

- ออเดอร์ที่ถูกยกเลิกแล้วยังชำระเงินได้ เพราะ `Payable` ไม่มีข้อมูลสถานะให้ `CheckoutFacade` ตรวจ
- การเลื่อนสถานะเป็น `PICKED_UP` ไม่ได้ตรวจว่าชำระเงินแล้วหรือยัง
- ไม่มีการล็อกแบบ optimistic (`@Version`) และคำขอเลื่อนสถานะไม่ได้ส่งสถานะที่ผู้ใช้เห็นมาตรวจ ถ้าพนักงานกด "ขั้นถัดไป" ซ้ำสองครั้งติดกัน หรือพนักงานสองคนกดออเดอร์เดียวกันในเวลาใกล้กัน ออเดอร์อาจเลื่อนไปสองขั้น และถ้าสองคำขออ่านสถานะเดิมพร้อมกัน ลูกค้าจะได้แจ้งเตือนซ้ำ
- ข้อความผิดพลาดบนหน้าเว็บออเดอร์มีสองภาษาปนกัน ข้อผิดพลาดของฟอร์ม (เช่น ไม่ได้เลือกสาขา น้ำหนักไม่ถูกต้อง) แสดงเป็นภาษาไทย แต่ข้อผิดพลาดที่มาจากกฎธุรกิจใน Service แสดงเป็นภาษาอังกฤษตามข้อความของ exception เช่น "Order 9 cannot be cancelled once it is WASHING" หรือ "Service type 4 is not available" เพราะหน้าเว็บนำข้อความเดียวกับที่ REST API ส่งกลับมาแสดงตรงๆ

- หน้าเว็บยังไม่มีการแก้ไขออเดอร์และการจัดการประเภทบริการของผู้ดูแลระบบ ทำได้ผ่าน REST API เท่านั้น และพนักงานสร้างออเดอร์แทนลูกค้าได้ผ่าน API เท่านั้น
- ฟอร์มฝากซักมีช่องรายการคงที่ 3 แถว และบอร์ดพนักงานแสดงรหัสลูกค้าแทนชื่อ

ส่วนของภีมเดช กลั่นกิ่ง

- ไม่มีเทสต์การชำระซ้ำพร้อมกันจริง (race) ตัว UNIQUE ที่กันไว้พิสูจน์แล้ว แต่สถานการณ์แข่งกันยังไม่ได้ทดสอบ
- เทสต์สิทธิ์ mock service จึงไม่ได้ตรวจกฎเจ้าของใน CheckoutFacade ร่วมกับฐานข้อมูลจริง (ตรวจด้วย CheckoutFacadeTest และการทดสอบ manual)
- เทสต์ที่ต่อฐานข้อมูลจริงเป็นแบบ opt-in และ CI ปัจจุบันไม่ได้รัน
- ข้อความผิดพลาดบนหน้าเว็บชำระเงินบางกรณี (จ่ายซ้ำ, ไม่พบรายการ) ยังเป็นภาษาอังกฤษ เพราะแสดงข้อความจาก service ตรงๆ
- สัญญา Payable ไม่มีข้อมูลสถานะ CheckoutFacade จึงตรวจไม่ได้ว่ารายการยกเลิกแล้วหรือไม่ ออเดอร์ที่ยกเลิกแล้วยังชำระผ่าน API ได้ (ข้อจำกัดของสัญญาที่ใช้ร่วมกันหลายโมดูล)
- ฟอร์มชำระเงินผ่านเมนูยังให้พิมพ์เลขที่รายการเอง (มีปุ่มชำระเงินจากหน้าออเดอร์และหน้าประวัติรอบใช้งานแล้ว), ยังไม่มีรายงานยอดขาย และยังไม่ได้วัด code coverage

5.6 ข้อเสนอแนะและแนวทางพัฒนาต่อ

ส่วนของวงศธร ธน.ยอด

1. เปลี่ยนรหัสผ่านของบัญชีตัวอย่างและตัดบัญชีตัวอย่างออกจากการ deploy จริง (เพิ่ม migration ใหม่) พร้อมเพิ่มการล็อกบัญชีและการยืนยันอีเมล
2. เพิ่มบริการ PostgreSQL ใน CI เพื่อรันเทสต์ที่ใช้ฐานข้อมูลจริงทุก Pull Request
3. เพิ่มหน้าจัดการผู้ใช้สำหรับผู้ดูแลระบบ และหน้ารายงานยอดขาย
4. ใช้ token (เช่น JWT) แทนการส่ง Basic ทุกคำขอ หากต้องรองรับแอปมือถือหรือไคลเอนต์ภายนอก
5. ย้ายไปแผนที่ไม่หยุดทำงานเมื่อไม่มีผู้ใช้ และเพิ่มการเฝ้าระวัง (monitoring) เมื่อใช้งานจริง

ส่วนของคมชาญ น้อยเนียม

- เพิ่มเมธอดตรวจว่าชำระได้ (เช่น isPayable()) ใน Payable ให้ทุกโมดูลกันการชำระรายการที่ยกเลิกแล้ว
- เพิ่มกฎให้รับผ้าคืน (PICKED_UP) ได้เมื่อชำระเงินแล้ว
- เพิ่มหน้าเว็บแก้ไขออเดอร์และหน้าจัดการประเภทบริการสำหรับผู้ดูแลระบบ โดยใช้ Service เดิม

ส่วนของภีมเดช กลั่นกิ่ง

เพิ่มบริการ PostgreSQL ใน CI เพื่อรันเทสต์ฐานข้อมูลทุก PR, ตั้งชื่อ CHECK ใน migration ถัดไปให้ชัด, ดึงโค้ดตั้งค่า schema ชั่วคราวของเทสต์เป็นคลาสแม่, จำกัด max-page-size ของการแบ่งหน้า, ทำหน้าเว็บ และเชื่อมวิธีชำระด้วยเหรียญกับรอบใช้เครื่องให้ครบวงจร

5.7 สรุปผลโมดูลออเดอร์

โมดูลออเดอร์ฝากซักครอบคลุมตั้งแต่ลูกค้าสร้างออเดอร์ พนักงานเลื่อนสถานะทีละขั้น จนถึงการชำระเงินผ่านโมดูลชำระเงิน มีทั้ง REST API และหน้าเว็บ ใช้ State, Strategy, Builder และ Observer (ฝั่งผู้ยิง) ในจุดที่แก้ปัญหาจริง กฎธุรกิจอยู่ใน Service ที่เดียวและทั้ง API กับหน้าเว็บใช้ร่วมกัน ทดสอบด้วยเทสต์อัตโนมัติ 78 ข้อและการทดสอบกับฐานข้อมูลจริง

5.8 สรุปโมดูลที่ทำแล้ว

ส่งมอบ Entity/Repository, Strategy, State, BookingValidator, MachineService/SessionService, DTO/Mapper, REST API เครื่องและรอบใช้งาน, SessionPayableProvider และเทสต์กฎธุรกิจ/สิทธิ์/ฐานข้อมูล การจองคิดราคาและเวลาใน server ใช้ lock ป้องกันคำขอพร้อมกัน และเปลี่ยนสถานะเครื่องกับรอบใช้งานใน transaction เดียวกัน หน้าเว็บเครื่อง จอง ประวัติ และพนักงานเปลี่ยนสถานะใช้ layout ของทีมและ Service เดียวกับ API พร้อม Sequence Diagram สาม scenario และ State Diagram ของ Machine ทั้งหมด merge เข้า develop แล้วผ่าน PR #25

5.9 ข้อจำกัด งานตรวจรับที่เหลือ และแนวทางพัฒนาต่อ

งานลิงก์ชำระเงินจากประวัติพร้อมเทสต์และรายงานล่าสุด merge ผ่าน PR #28 ที่ 5070373 โดยภีมเดช กลั่นกิ่ง approve และ CI Test + Docker build ผ่าน ส่วนหน้าเว็บและ diagrams ชุดหลัก merge แล้ว ทดสอบ flow จอง/ชำระ QR จำลอง/start/finish บน deployment ด้วยบัญชีตัวอย่างแล้ว ดู doc/report/pond-deploy-demo.md Start ตรวจสถานะแต่ไม่บังคับช่วงเวลาเริ่มหรือการชำระก่อนใช้งาน Machine event ยังไม่มี listener

และ API อ่านเครื่องใช้ข้อกำหนดล็อกอินของทีม หน้าชำระเงินของภีมเดช กลั่นกิ่งอยู่ใน develop ผ่าน PR #26 ลิงก์จากประวัติส่งเฉพาะประเภทกับเลขรอบและซ่อนสำหรับรอบยกเลิก หลัง PR #37 controller เรียก PaymentService เพื่ออ่าน payment IDs ครั้งเดียวต่อหน้าและซ่อนปุ่มเมื่อมีรายการชำระเงินแล้ว เดโมใหม่ตรวจว่าปุ่มหายหลัง QR สำเร็จ โดย backend ยังคงปฏิเสธการชำระซ้ำ

ก่อนปิดงานควรตกลง HTTP status ของ lifecycle ให้ตรง brief ทุกส่วน รวมหลักฐานเดโม UI ล่าสุดและปรับ citation/เอกสารรวมตาม branch ที่ส่งมอบล่าสุด แนวทางขยายคือกำหนดนโยบายเริ่มก่อน/หลังเวลาจองและ session เกินเวลา รวมทั้งเลือกผู้รับแจ้งเตือนเครื่องหยุดบริการก่อนเพิ่ม listener

เอกสารอ้างอิง

Bauer, C., King, G., & Gregory, G. (2015). Java Persistence with Hibernate (2nd ed.). Manning.

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress.

Docker Documentation. https://docs.docker.com

Fielding, R., Nottingham, M., & Reschke, J. (2022). HTTP Semantics (RFC 9110). IETF.

Flyway Documentation. https://flywaydb.org

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Fowler, M. (2006). Continuous Integration. https://martinfowler.com/articles/continuousIntegration.html

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

GitHub Docs: GitHub Actions. https://docs.github.com/actions

Hibernate ORM. (2024). Hibernate ORM User Guide: Fetching. สืบค้นจาก https://docs.jboss.org/hibernate/orm/6.6/userguide/html_single/Hibernate_User_Guide.html

Martin, R. C. (2003). Agile Software Development, Principles, Patterns, and Practices. Prentice Hall.

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. Linux Journal, 2014(239).

OWASP Foundation. Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org

OpenAPI Specification. https://www.openapis.org

Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. Proceedings of the USENIX Annual Technical Conference.

Spring Boot Reference Documentation. https://spring.io

Spring Framework and Spring Boot Reference Documentation. https://spring.io

Spring Security Reference. https://spring.io

Spring. (2024). Spring Data JPA Reference Documentation. สืบค้นจาก https://docs.spring.io/spring-data/jpa/reference/

ภาคผนวก

ส่วนของวงศธร ธน.ยอด

- **ภาคผนวก ก**: คู่มือการติดตั้ง รัน และทดสอบระบบ (README.md)
- **ภาคผนวก ข**: Use Case Diagram และ Use Case Description (doc/diagrams/use-case.md)
- **ภาคผนวก ค**: Component Diagram และ Deployment Diagram (doc/diagrams/component-diagram.md, doc/diagrams/deployment-diagram.md)
- **ภาคผนวก ง**: ส่วนของผู้เขียนในเอกสาร SOLID และ Design Pattern (doc/sections/solid-analysis-oak.md, doc/sections/design-patterns-oak.md) และฉบับรวม (doc/solid-analysis.md, doc/design-patterns.md)

ส่วนของคมชาญ น้อยเนียม

- Class Diagram ทั้งระบบ 4 รูป: doc/diagrams/class-diagram.md
- State Diagram และตารางสถานะ ↔️ คลาส: doc/diagrams/state-order.md
- SOLID พร้อมไฟล์และเลขบรรทัด: doc/sections/solid-analysis-peach.md
- Design Patterns พร้อมเหตุผล: doc/sections/design-patterns-peach.md

ส่วนของปฏิภาณ มะนิลทิพย์

- โค้ดหลัก: code/src/main/java/com/laundryhub/service/impl/SessionServiceImpl.java และ controller/api/MachineApiController.java, SessionApiController.java
- เอกสาร API: doc/self-service-api.md
- รายงานทดสอบ: doc/test-report/session-lifecycle-pond.md
- วิเคราะห์ส่วนบุคคล: doc/sections/solid-analysis-pond.md และ design-patterns-pond.md

- เทสต์ HTTP: test/java/com/laundryhub/service/SessionPaymentIntegrationTest.java
- เทสต์การแข่งขัน: test/java/com/laundryhub/service/SessionLifecycleIntegrationTest.java
- หน้าเว็บ: code/src/main/java/com/laundryhub/controller/web/SelfServiceWebController.java และ templates/machines/, templates/sessions/
- เทสต์เว็บ: test/java/com/laundryhub/security/SelfServiceWebSecurityTest.java และกรณี realWebFormBooksForAuthenticatedOwnerAndLifecyclePersists ใน SessionPaymentIntegrationTest
- Sequence Diagram: doc/diagrams/sequence-order-create-pond.md, sequence-session-booking-pond.md, sequence-checkout-pond.md
- State Diagram: doc/diagrams/state-machine-pond.md
- ผลตรวจความสอดคล้องกับทีม: doc/report/pond-integration-check.md

ส่วนของปฏิภาณ มะนิลทิพย์

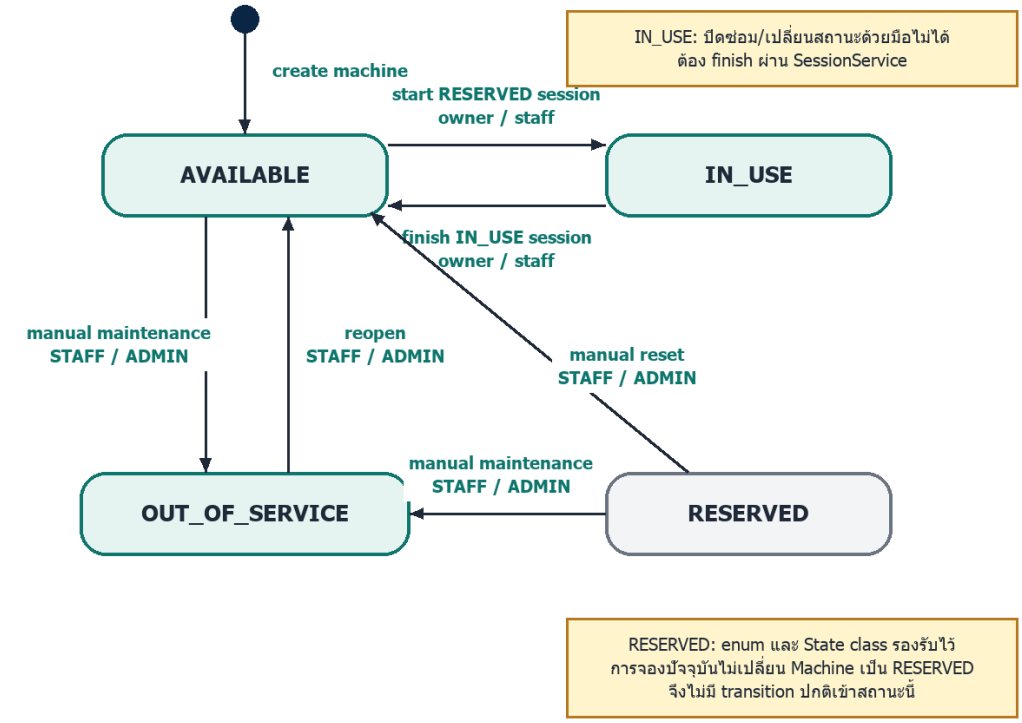
หลักฐาน browser/API จริงและภาพอยู่ใน doc/report/pond-deploy-demo.md; citation ที่ต้องให้วงศธร ธน.ยอดรวมอยู่ใน pond-citation-check.md; contract 2 ข้อที่รอข้อตกลงอยู่ใน pond-contract-check.md; บทซ้อมอยู่ใน pond-presentation-rehearsal.md เดโมนี้ไม่ใช่การชำระเงินจริงหรือ concurrency test และไม่ยืนยัน deployment SHA

ส่วนของปฏิภาณ มะนิลทิพย์

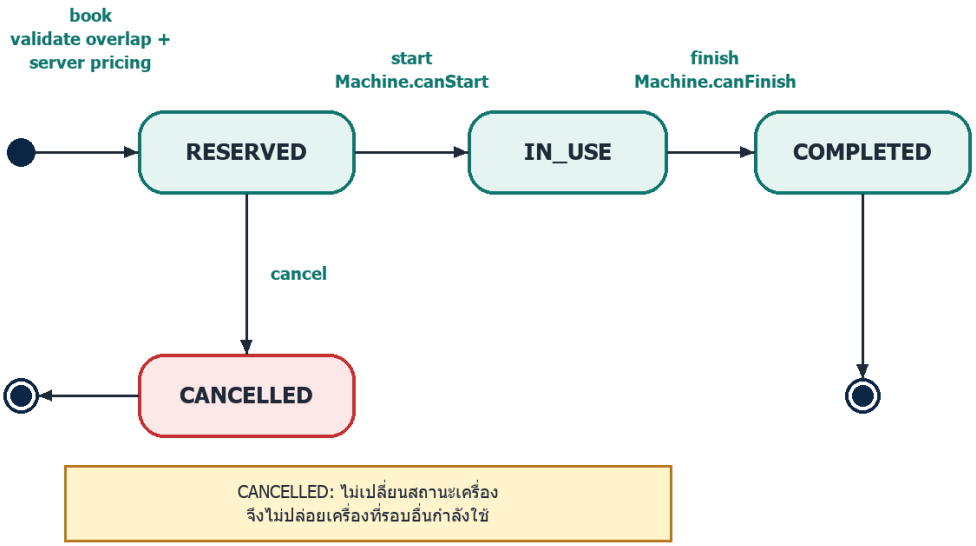
10 ตุลาคม 2569 รวม develop bf9d636 แล้วเดโมรอบใหม่ 5/Payment 4 ครบจอง–QR จำลอง–เริ่ม–จบ ตรวจสถานะภาษาไทย ปุ่มชำระหาย และแจ้งเตือนครบ ดู pond-deploy-demo.md ภาพและสไลด์ใช้รอบใหม่นี้

Maven หลังรวม develop รัน 376 รายการ ผ่าน 340 ข้าม PostgreSQL 36 ไม่มี failure/error และ BUILD SUCCESS เวลา 22:13 น. รอบนี้ไม่ได้เปิด LAUNDRY_DB_TESTS ไม่ใช้แทนหลักฐาน DB suite

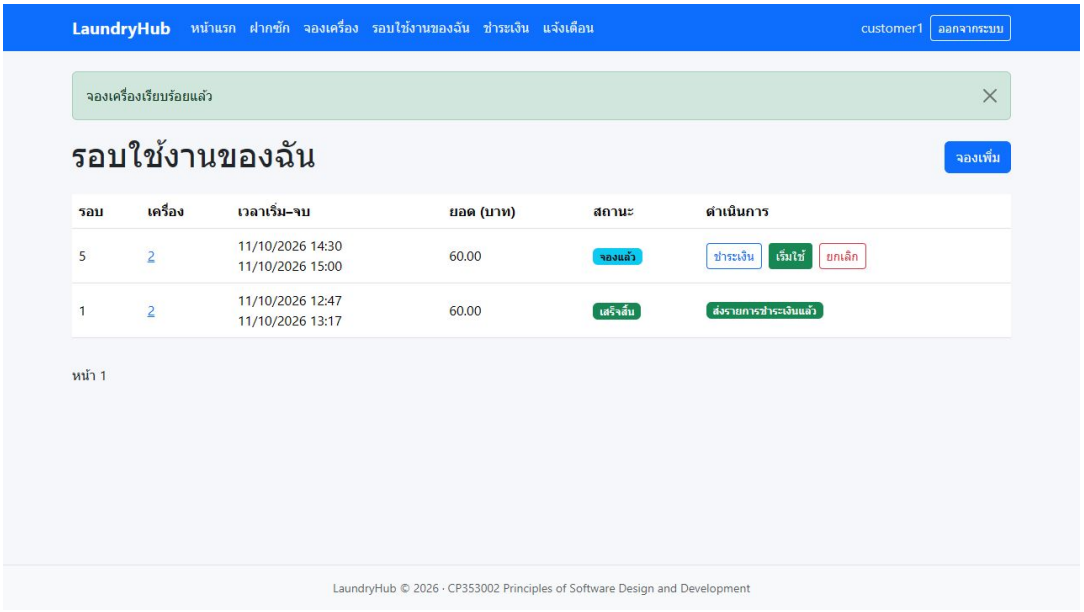
แผนภาพและภาพหลักฐานเพิ่มเติมของปฏิภาณ มะนิลทิพย์



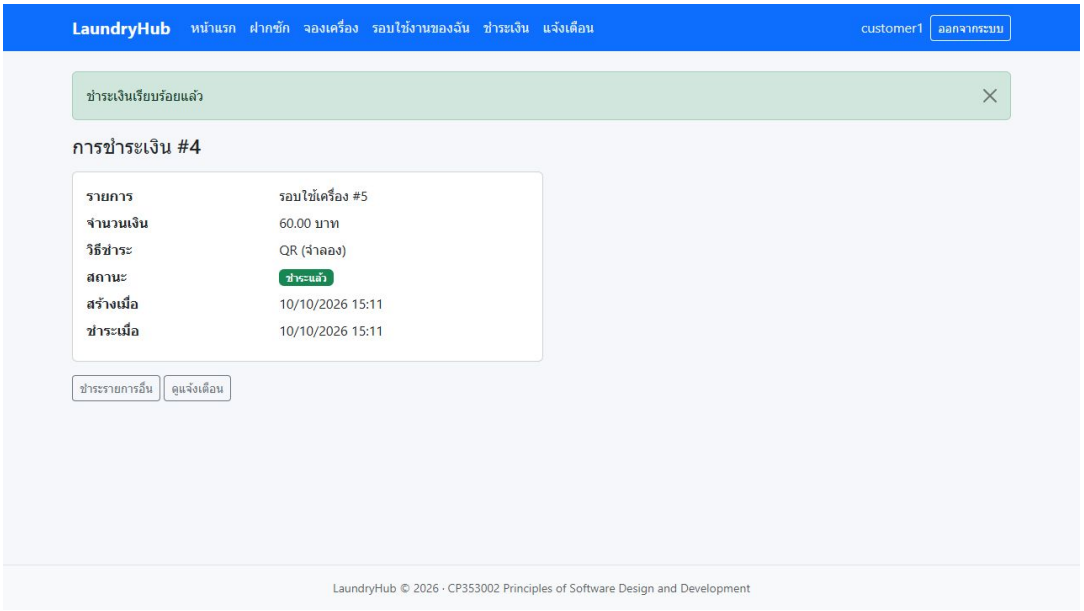
ภาพที่ 27 State Diagram ของเครื่อง (Machine)



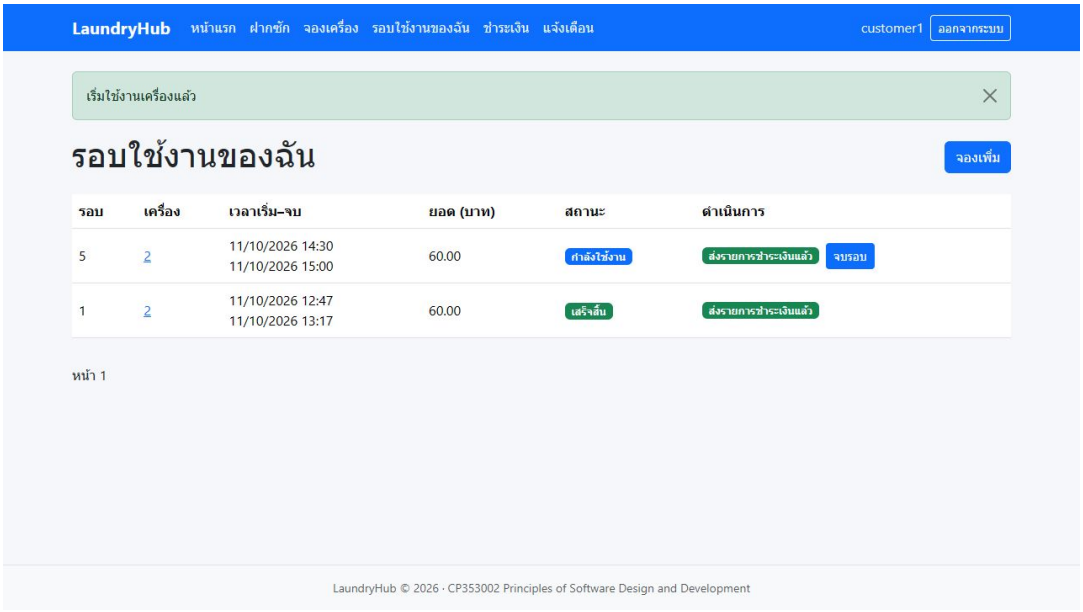
ภาพที่ 28 State Diagram ของรอบใช้งาน (Session)



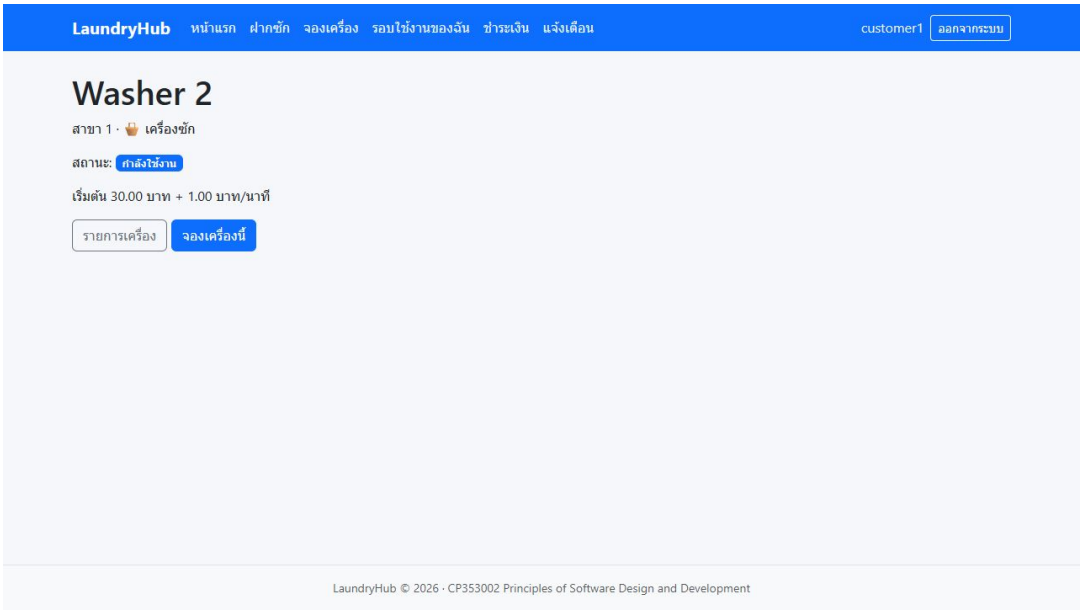
ภาพที่ 29 เดโมบน Render: จองสำเร็จ



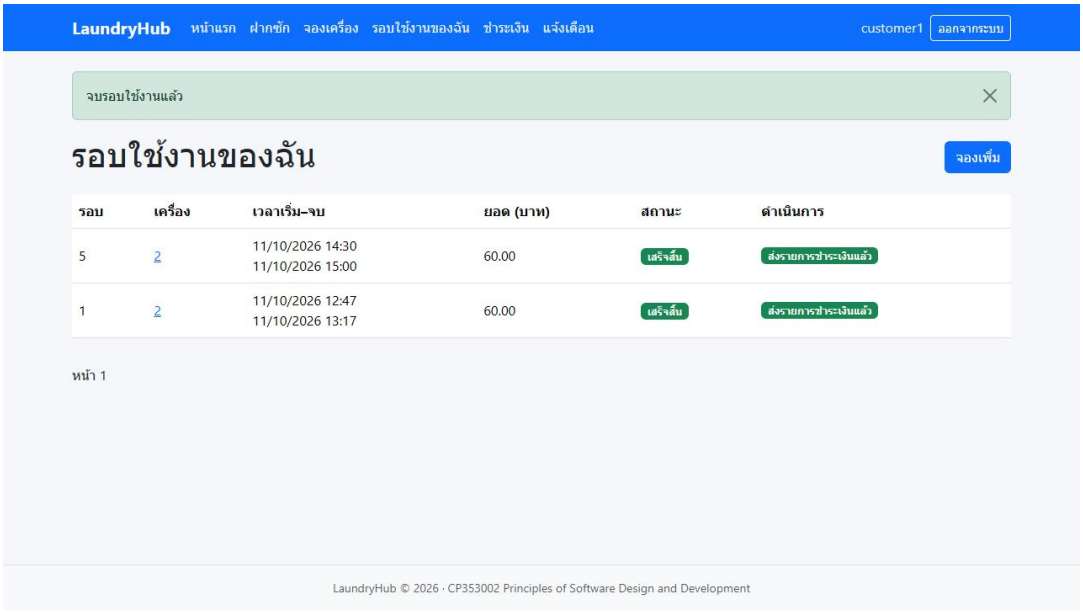
ภาพที่ 30 เดโมบน Render: ชำระ QR จำลองสำเร็จ



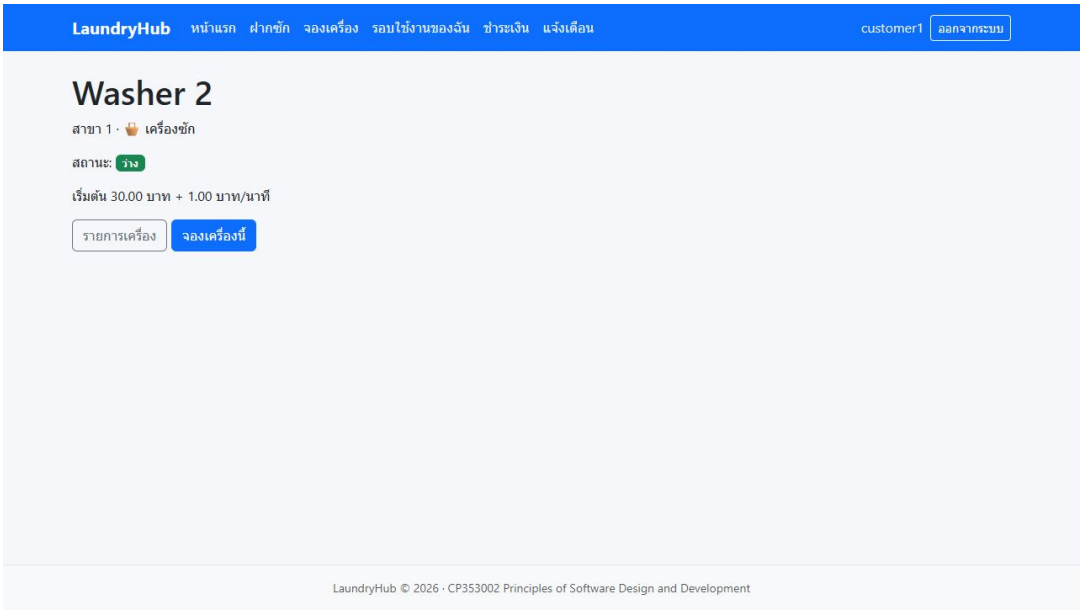
ภาพที่ 31 เดโมบน Render: รอบกำลังใช้งาน



ภาพที่ 32 เดโมบน Render: เครื่องกำลังใช้งาน



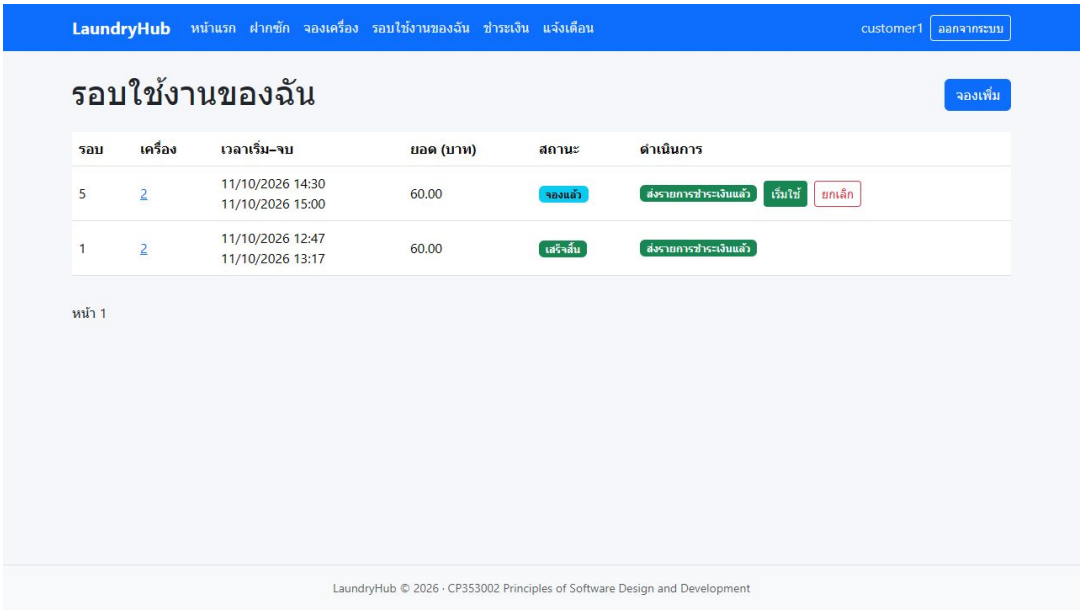
ภาพที่ 33 เดโมบน Render: รอบเสร็จสิ้น



ภาพที่ 34 เดโมบน Render: เครื่องกลับมาว่าง



ภาพที่ 35 เดโมบน Render: แจ้งเตือนครบทุกเหตุการณ์



ภาพที่ 36 เดโมบน Render: ปุ่มชำระเงินหายหลังชำระแล้ว

ส่วนของภีมเดช กลั่นกิ่ง

- **ภาคผนวก ก**: Test Report ฉบับเต็ม (`doc/test-report/test-report.md`)
- **ภาคผนวก ข**: Data Dictionary (`doc/data-dictionary.md`) และ ER Diagram (`doc/diagrams/er-diagram.md`)

- **ภาคผนวก ค**: วิธีรันโปรเจคและรันเทสต์ (README.md, คำสั่ง LAUNDRY_DB_TESTS=true mvn test -Dtest=PaymentRepositoryDbTest)